

Isomera

Manual visual e estudo teórico

Detecção de tabelas duplicadas com grafos, benchmarks, modelos e
interpretabilidade

Cayo Oliveira

Centro de Informática – UFPE

MoDCS Research Group

Julho de 2026

Contents

Como ler este manual	4
1 O problema: duplicidade em arquiteturas de dados	5
1.1 O que o Isomera tenta responder	6
2 Da tabela ao grafo	8
2.1 Como interpretar grafo e matriz lado a lado	10
3 Do grafo ao tensor VMamba-Mesh	12
3.1 Leitura dos canais	13
3.2 O que entra e o que sai do modelo	14
4 Mapa geral do aplicativo	16
4.1 Home	16
4.2 Jornadas principais	17
4.3 Rotas principais	17
4.4 Cobertura visual deste manual	18
5 Benchmark & Examples	20
5.1 Article Reproducibility	20
5.2 Run Benchmark	22
5.3 Concepts	24
6 Scenario Studio	27
6.1 Fluxo vertical do Scenario Studio	28
6.2 Model Training no Scenario Studio	30
7 Study Lab	32
7.1 Deep Learning Workbench	32
7.2 Original VMamba	33
7.3 Run SS2D Demo	34
7.4 VMamba-Mesh Changes	35
7.5 Official Runtime	36
7.6 Train Model Adapter	37
7.7 Model Reports	39
7.8 Model Interpretability	41
7.9 Knowledge Base	43

8 Model Lab	46
9 Research Reports	49
10 Admin	52
10.1 Backend Store	53
10.2 Scenario Warehouse	55
10.3 Neo4j e Settings	57
11 Logs	61
12 Help e About	64
12.1 Help	64
12.2 About	67
13 Métricas e resultados	70
14 Famílias de modelos no Isomera	72
14.1 VF2	72
14.2 Node Match	72
14.3 GNN e GIN	73
14.4 Vanilla VMamba	73
14.5 VMamba-Mesh adapter	73
14.6 VMamba-T	74
14.7 VMamba-Mesh-T	74
15 Treinamento, hiperparâmetros e ablation	75
15.1 Dataset supervisionado	75
15.2 Loss e otimizador	75
15.3 Device: CPU, MPS e auto	76
15.4 Batching	76
15.5 Ablation	76
16 Playbooks de uso	78
16.1 Playbook 1: primeira abertura	78
16.2 Playbook 2: executar um benchmark pronto	78
16.3 Playbook 3: explicar VMamba-Mesh	79
16.4 Playbook 4: treinar modelo	79
16.5 Playbook 5: analisar interpretabilidade	79
16.6 Playbook 6: criar cenário novo	79
16.7 Playbook 7: usar em uma empresa	80
17 Glossário operacional	81
18 Catálogo visual tela a tela	83
18.1 Home	84
18.2 Benchmark: reprodução de campanha	85
18.3 Benchmark: execução direta	86

18.4 Benchmark: conceitos	87
18.5 Scenario Studio: visão geral	88
18.6 Scenario Studio: fonte relacional	89
18.7 Scenario Studio: validação de pares	90
18.8 Scenario Studio: treinamento	91
18.9 Study Lab: Deep Learning Workbench	92
18.10 Study Lab: Original VMamba	93
18.11 Study Lab: SS2D Demo	94
18.12 Study Lab: mudanças VMamba-Mesh	95
18.13 Study Lab: runtime oficial	96
18.14 Study Lab: treino VMamba	97
18.15 Study Lab: relatórios de modelo	98
18.16 Study Lab: interpretabilidade	99
18.17 Study Lab: Knowledge Base	100
18.18 Model Lab	101
18.19 Research Reports	102
18.20 Admin: Overview	103
18.21 Admin: Backend Store	104
18.22 Admin: Scenario Warehouse	105
18.23 Admin: Neo4j	106
18.24 Admin: Settings	107
18.25 Logs: Session Log	108
18.26 Logs: Terminal Logs	109
18.27 Help: Presentation	110
18.28 Help: Tech Docs	111
18.29 About	112
19 Interpretabilidade	113
20 Como aplicar em uma empresa	115
20.1 Roteiro corporativo recomendado	115
20.2 Cuidados antes de produção	116
21 Checklist de demonstração	117
21.1 Checklist de estudo individual	117
22 Referências internas e materiais de apoio	118
22.1 Referências conceituais recomendadas	118

Como ler este manual

Este documento foi escrito para funcionar como manual de ferramenta e material de estudo. A leitura começa com o problema de negócio, passa pela teoria de grafos e modelos, e depois entra na navegação prática do Isomera. A intenção é que um engenheiro de dados, um pesquisador ou um avaliador consiga abrir a ferramenta, entender cada rota e reproduzir a lógica central do estudo.

O manual usa capturas reais do Isomera. As caixas “Onde clicar” indicam a navegação dentro do aplicativo. As caixas de estudo explicam por que aquela tela existe e como ela se conecta à teoria.

Este book não deve ser lido como um artigo científico. Ele é um guia de produto. O artigo aparece somente quando a ferramenta precisa explicar uma campanha de benchmark, uma métrica ou uma evidência de reprodutibilidade. O foco principal é outro: mostrar como o Isomera funciona, por que cada tela existe, como cada clique participa da jornada e o que acontece tecnicamente por trás da interface.

Há três formas naturais de leitura. Quem quer usar a ferramenta pela primeira vez deve ler os capítulos iniciais e depois seguir o mapa das rotas. Quem quer gravar uma demonstração deve ir direto para o checklist e voltar às seções de Benchmark, Study Lab e Research Reports. Quem quer estudar a metodologia deve ler com calma os capítulos sobre grafo, matriz, tensor, modelos, métricas e interpretabilidade antes de abrir as telas.

Dica! Escolha uma trilha antes de começar: demonstração, pesquisa ou implantação. Isso evita ler o manual como sequência obrigatória e ajuda a voltar às rotas certas depois.

Chapter 1

O problema: duplicidade em arquiteturas de dados

Em uma arquitetura de dados moderna, a mesma ideia analítica pode aparecer mais de uma vez. Um domínio cria uma tabela de resumo de clientes, outro domínio cria uma tabela parecida para um painel diferente, e uma terceira área cria uma agregação de vendas que já existia em outro lugar. Muitas vezes os nomes são diferentes, os caminhos de transformação não são idênticos e a documentação não mostra imediatamente que as tabelas são redundantes.

O Isomera trata esse problema como um problema de linhagem. Em vez de comparar apenas nomes de tabelas, ele olha para a vizinhança estrutural: de onde a tabela veio, quais transformações a produziram, em qual camada ela está e quais padrões de dependência aparecem ao redor dela.

Essa mudança de ponto de vista é pequena na frase, mas grande na prática. Uma tabela não é apenas um arquivo com colunas. Ela tem uma história: nasce de fontes operacionais, passa por transformações, é agregada, filtrada, juntada com outras tabelas e finalmente aparece como produto analítico. Quando duas tabelas contam praticamente a mesma história, existe uma chance real de redundância. O nome pode mudar, a ordem de algumas transformações pode mudar, mas a estrutura de dependência ainda carrega sinais.

Dica! Ao apresentar o Isomera, comece pela história da tabela, não pelo algoritmo. A duplicidade fica mais clara quando o usuário entende origem, transformação e consumo do dado.

Em uma empresa, esse tipo de redundância costuma ficar invisível porque cada time olha para o próprio domínio. Um time de vendas enxerga o painel de vendas. Um time de clientes enxerga o resumo de clientes. Um time financeiro enxerga agregações de margem. O problema aparece quando os dados começam a divergir, quando o custo de processamento cresce ou quando alguém precisa explicar por que duas tabelas supostamente diferentes respondem à mesma pergunta.

O Isomera nasce para apoiar esse tipo de investigação. Ele não substitui governança, catálogo, revisão humana ou conhecimento de domínio. Ele organiza essas peças em um fluxo rastreável: transformar arquitetura em grafo, gerar pares candidatos, executar detectores, medir resultados, salvar relatórios e permitir que o usuário volte ao par específico para entender por que ele foi marcado como duplicado.

Ideia prática. Se duas tabelas são duplicadas, elas podem não ser idênticas no SQL ou no nome. Mesmo assim, seus contextos de linhagem podem revelar que elas representam o mesmo produto de dados ou uma variação redundante do mesmo cálculo.

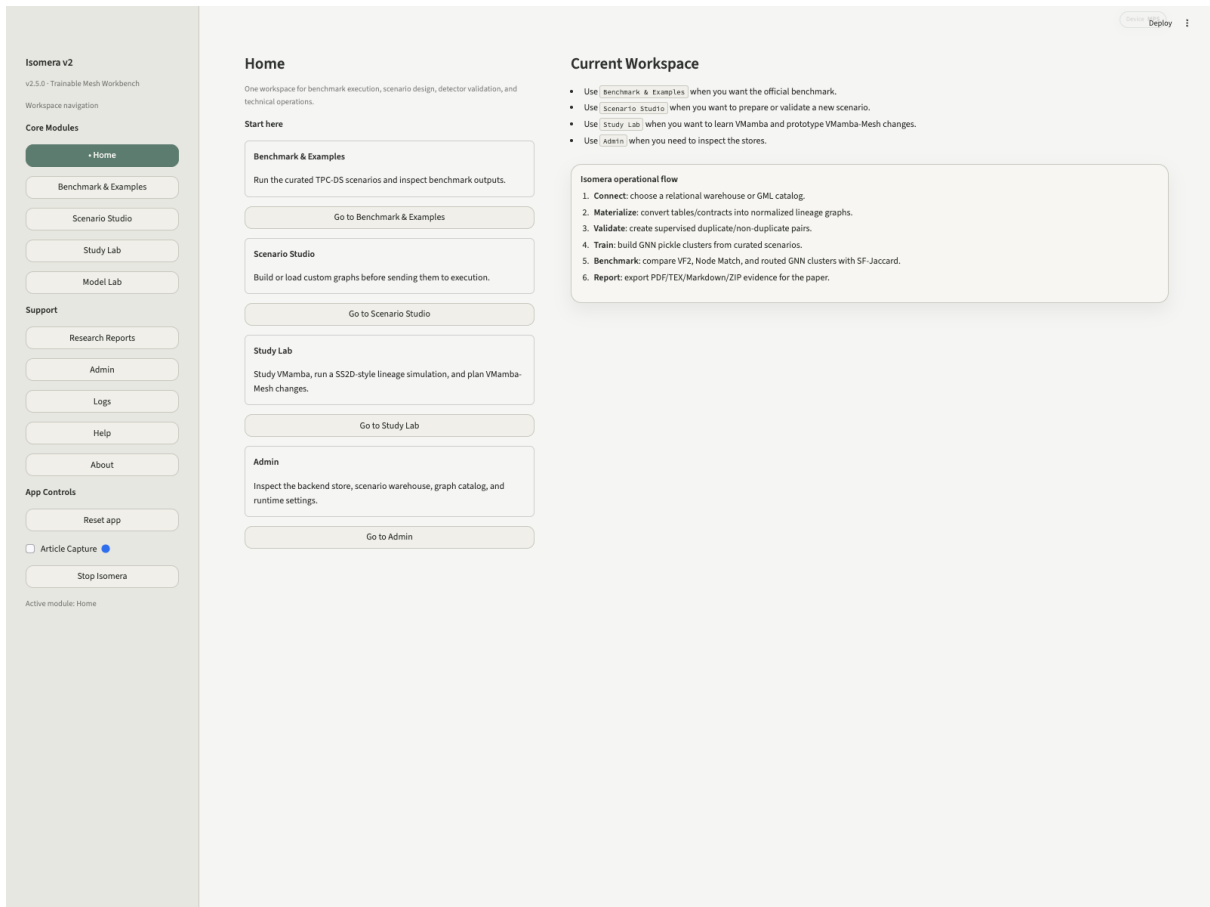


Figure 1.1: Tela inicial do Isomera. O menu lateral organiza o uso por módulos: benchmark, criação de cenários, estudo, modelos, relatórios, administração, logs, ajuda e informações do projeto.

1.1 O que o Isomera tenta responder

A pergunta operacional é simples: dado um par de tabelas, o par deve ser tratado como duplicado? A resposta, porém, precisa ser bem formada. Não basta dizer “sim” ou “não”. Um detector útil precisa informar qual benchmark foi usado, qual cenário estava ativo, qual modelo tomou a decisão, qual threshold foi aplicado, quais métricas foram obtidas e quais evidências visuais sustentam a análise.

No Isomera, essa resposta aparece em várias camadas. Na camada mais direta, o modelo retorna pares duplicados por meio do contrato `predict_pairs(graph)`. Na camada de benchmark, esses pares são comparados com o ground truth e viram métricas como Jaccard, SF-Jaccard e tempo de execução. Na camada de estudo, o usuário abre o par e observa grafo, matriz, canais e interpretabilidade. Na camada de relatório, a execução vira um pacote com CSV, JSON, Markdown, figuras e manifest.

Na prática: Quando estiver usando a ferramenta, pense sempre em quatro perguntas: qual grafo estou analisando, qual par estou comparando, qual modelo está decidindo e qual evidência eu consigo mostrar depois?

Chapter 2

Da tabela ao grafo

O Isomera representa a arquitetura como um grafo dirigido. Cada nó é uma entidade da arquitetura de dados, e cada aresta representa dependência. Nos benchmarks usados neste estudo, os nós geralmente aparecem como:

- **SOR**: origem operacional, extração ou tabela de sistema.
- **SOT**: transformação, tabela intermediária ou camada confiável.
- **SPEC**: produto semântico ou tabela analítica final.

Essa representação transforma uma pergunta difusa, “esta tabela é duplicada?”, em uma pergunta comparável: dado um par de nós ou subgrafos, o par representa uma duplicidade estrutural ou semântica?

Um grafo dirigido é uma forma natural de representar linhagem porque dependência tem direção. Se uma tabela analítica foi criada a partir de duas tabelas intermediárias, as arestas apontam das entradas para a saída. Quando isso se repete em várias camadas, o grafo registra o caminho de formação do dado. O Isomera usa essa estrutura para comparar contextos, não apenas objetos isolados.

Dica! Leia o grafo como evidência de linhagem: nós são entidades, arestas são dependências e a direção explica como o dado se forma.

Formalmente, o cenário pode ser tratado como um grafo dirigido rotulado $G = (V, E, \ell_V, \ell_E)$, em que V é o conjunto de tabelas ou entidades, $E \subseteq V \times V$ é o conjunto de dependências dirigidas, ℓ_V associa cada nó a metadados como camada, domínio e tipo, e ℓ_E pode representar o tipo de dependência ou transformação. Para um nó v , o grau de entrada $d^-(v)$ mede quantas fontes alimentam v , enquanto o grau de saída $d^+(v)$ mede quantos objetos dependem dele. Um caminho dirigido é uma sequência de nós conectados por arestas na mesma direção, e uma vizinhança de linhagem é o subgrafo usado para contextualizar uma tabela antes da comparação de duplicidade.

O ponto central é que a duplicidade não precisa acontecer no grafo inteiro. Em muitos casos, a redundância está em uma região: duas tabelas finais compartilham fontes, passam por transformações parecidas e chegam a produtos parecidos. Por isso a ferramenta trabalha com pares e subgrafos. O par indica quais nós estão sendo comparados; o subgrafo carrega o contexto necessário para a decisão.

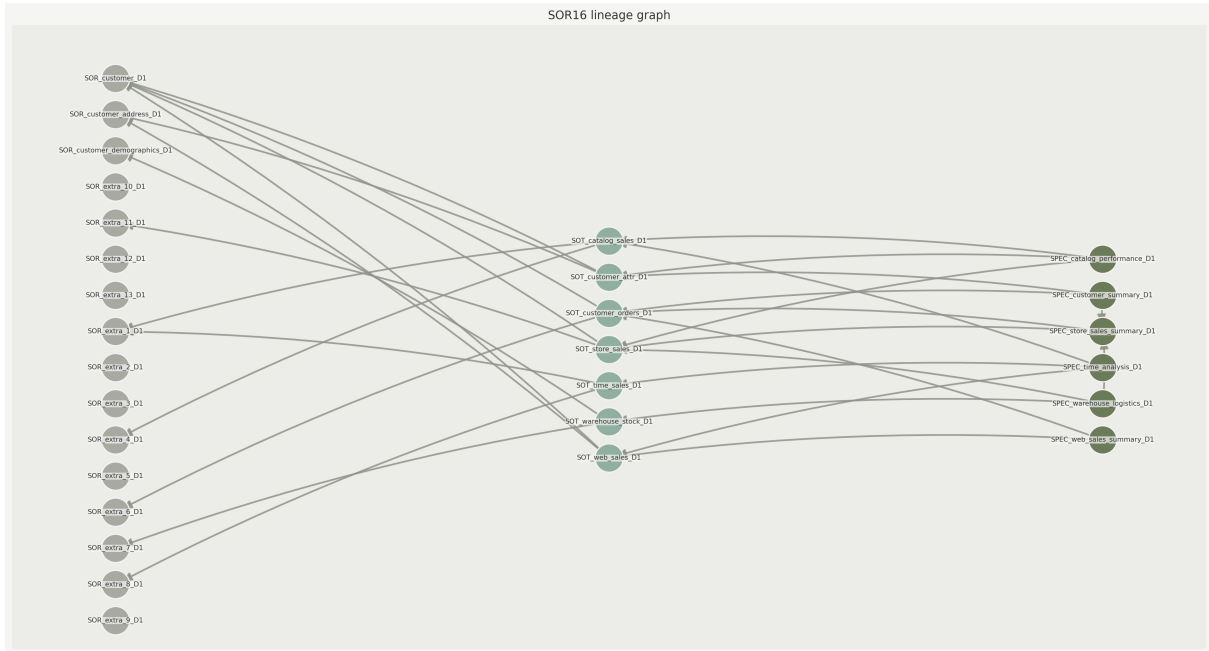


Figure 2.1: Grafo SOR16-D1 usado no estudo. O mesmo cenário aparece em várias telas do Isomera para manter a explicação consistente.

A matriz de adjacência é outra forma de ver o mesmo grafo. Cada linha e coluna representa um nó ordenado. Uma célula ativa indica dependência entre dois nós. Essa matriz é útil porque permite conectar teoria de grafos, imagens/tensores e modelos neurais.

Essa matriz é mais do que uma figura. Ela é um contrato computacional. Se os nós forem ordenados de qualquer forma, a matriz muda demais entre cenários e fica difícil aprender padrões. Por isso entra a ideia de ordenação canônica. O Isomera organiza os nós para que regiões da matriz tenham significado estável: fontes próximas de fontes, transformações próximas de transformações e produtos semânticos próximos de produtos semânticos. Essa estabilidade é essencial para comparar cenários e para alimentar modelos que leem tensores.

Fique atento! A matriz de adjacência depende da ordenação dos nós. Duas matrizes podem representar o mesmo grafo se diferirem apenas por uma permutação de linhas e colunas. Por isso, CanonSort não é detalhe visual: ele reduz variação artificial antes do modelo aprender padrões.

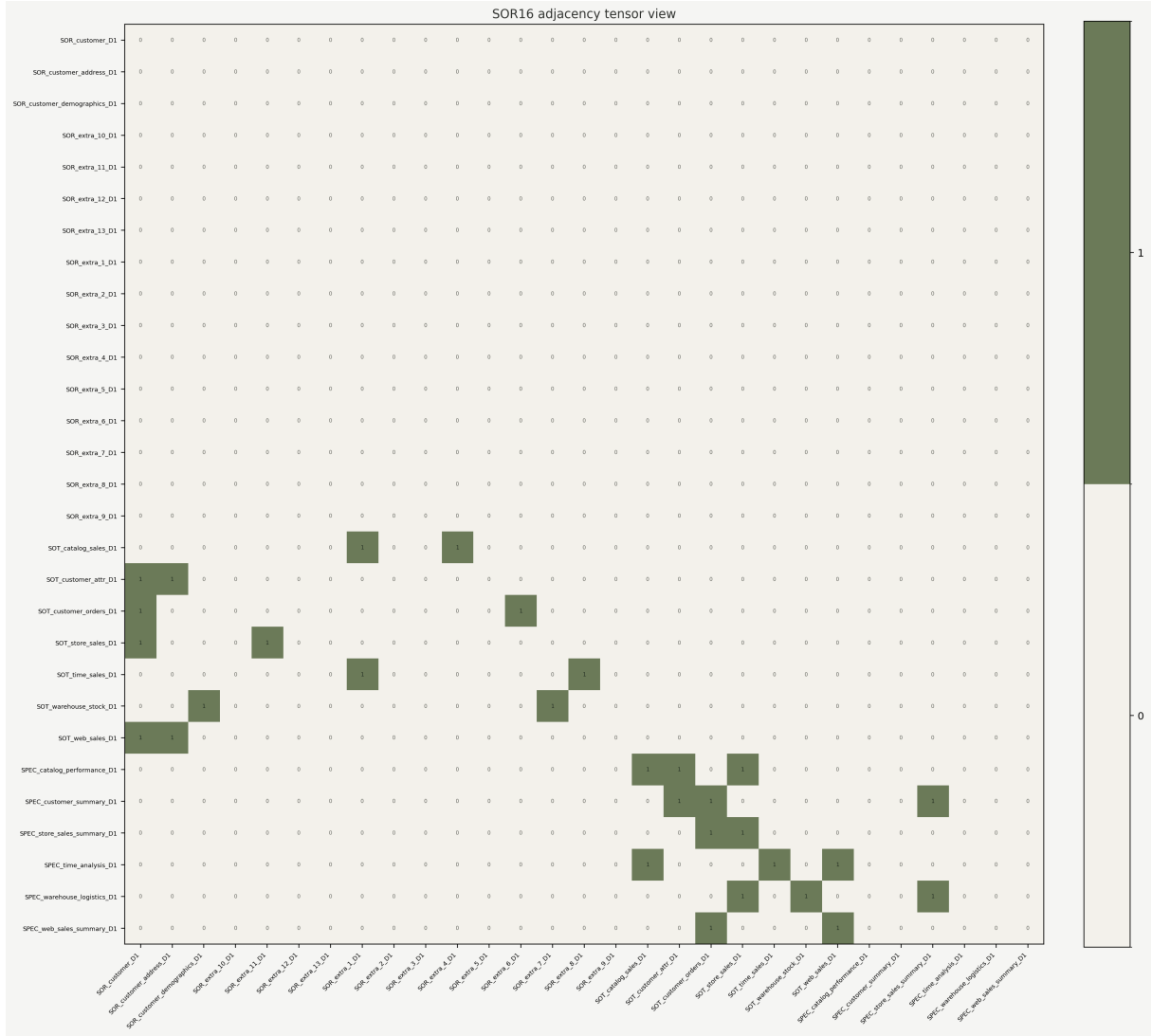


Figure 2.2: Matriz de adjacência canônica do SOR16-D1. Ela é a ponte entre grafo de linhagem e entrada matricial.

2.1 Como interpretar grafo e matriz lado a lado

No grafo, o olhar humano entende fluxo: esquerda para direita, camada por camada, fontes até produtos. Na matriz, o olhar técnico entende posição: linha, coluna, diagonal e densidade. As duas visões são complementares. O grafo ajuda a explicar o domínio. A matriz ajuda a transformar o domínio em entrada de algoritmo.

Quando a matriz tem muitos zeros, isso não significa ausência de informação. Significa que a arquitetura é esparsa, como costuma acontecer em linhagens reais. Poucas dependências são ativas entre todas as combinações possíveis de tabelas. Essa esparsidade precisa ser tratada com cuidado, porque um modelo ingênuo pode aprender mais sobre o vazio do que sobre o padrão estrutural. É por isso que canais como máscara esparsa e viés de rota existem no VMamba-Mesh.

Por trás do clique: O clique que mostra a matriz está conectado a uma transformação formal: o grafo $G = (V, E)$ vira uma matriz A , em que $A_{ij} = 1$ quando existe uma aresta dirigida do nó i para o nó j . A qualidade dessa matriz depende da ordem dos nós, por isso o CanonSort aparece antes dos canais.

Chapter 3

Do grafo ao tensor VMamba-Mesh

O VMamba original lê mapas bidimensionais. O VMamba-Mesh adapta essa ideia para grafos de linhagem. A pergunta é: se um grafo pode virar matriz, e se uma matriz pode virar tensor, então um backbone inspirado em VMamba pode aprender padrões de duplicidade em arquiteturas de dados?

O motivo para trazer uma arquitetura visual para esse problema é direto: uma matriz de adjacência é um campo 2D. Ela não é uma foto, mas possui linhas, colunas, regiões, diagonais, densidade e padrões espaciais. O VMamba foi criado para trabalhar com mapas bidimensionais usando blocos de espaço de estados visuais. A adaptação para Data Mesh não consiste em fingir que o grafo é uma imagem comum. A adaptação consiste em construir uma representação matricial que preserve a semântica da linhagem.

Por isso o VMamba-Mesh não usa apenas a matriz de adjacência. Ele adiciona canais. Cada canal responde a uma pergunta diferente sobre o mesmo grafo. Existe aresta direta? Existe dependência reversa? Qual camada está na diagonal? Qual é a assinatura de grau? Por onde a rota de linhagem tende a passar? Esta célula está vazia ou contém evidência estrutural? A rede passa a receber um tensor, não uma única imagem.

Dica! Não trate os canais como imagens separadas. Eles são perguntas diferentes feitas ao mesmo grafo: direção, camada, grau, rota e esparsidade.

Na linguagem de aprendizado supervisionado, a tensorização define uma função de representação $\phi(G, u, v)$, que transforma o grafo e um par candidato (u, v) em um tensor X . O modelo treinável aprende uma função $f_\theta(X) \rightarrow z$, em que z é o logit. A decisão final não está apenas no tensor, mas na composição entre representação, parâmetros aprendidos, sigmoid e threshold. Essa separação é central em aprendizado de máquina: features não são decisão; features são a forma como o problema é apresentado ao classificador.

Para isso, o Isomera usa uma representação com seis canais:

Canal	Função
C0	Adjacência direta. Preserva o sentido original das arestas.
C1	Adjacência reversa. Permite observar dependência no sentido oposto.
C2	Diagonal de camada. Marca posições SOR, SOT e SPEC.
C3	Impressão de grau. Expõe conectividade, fan-in e fan-out.
C4	Viés de rota de linhagem. Ajuda o scan a seguir trajetórias coerentes com SOR–SOT–SPEC.
C5	Máscara esparsa. Diferencia célula vazia de evidência estrutural real.

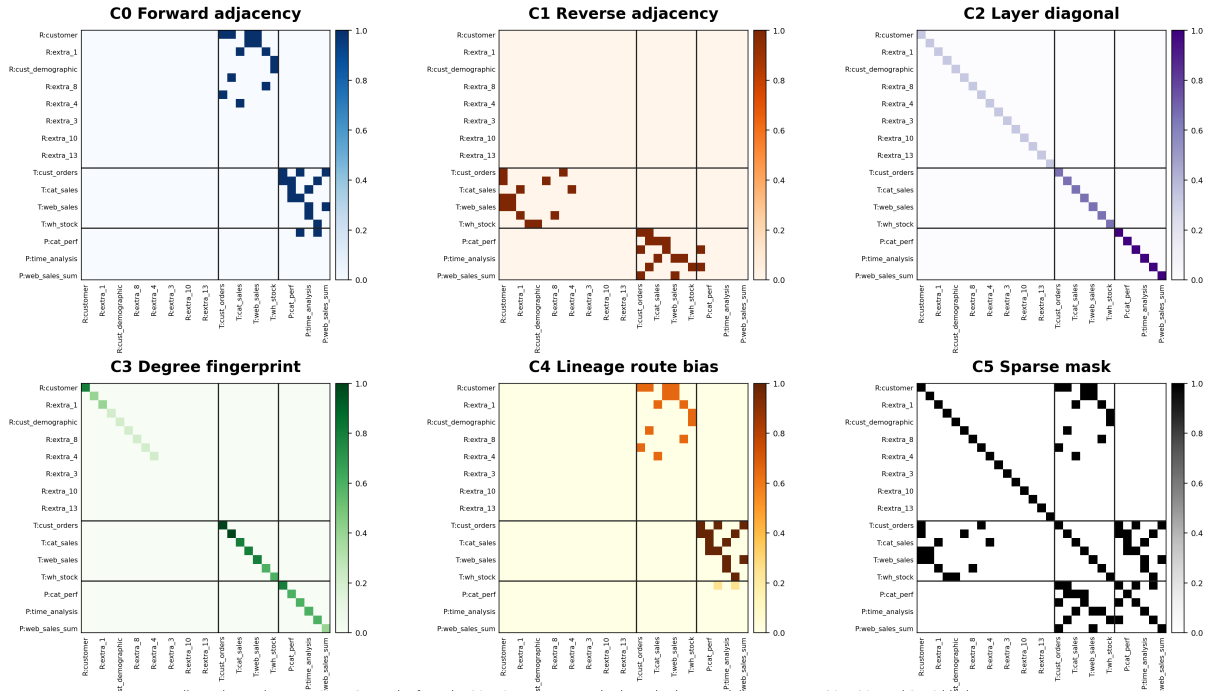


Figure 3.1: Canais C0–C5. O modelo deixa de olhar somente para zeros e arestas, e passa a receber direção, camada, grau, rota e máscara.

3.1 Leitura dos canais

O canal C0 é a adjacência direta. Ele preserva o caminho natural da linhagem. Se uma tabela A alimenta uma tabela B, a célula correspondente fica ativa. Sem esse canal, o modelo perderia o esqueleto principal do grafo.

O canal C1 é a adjacência reversa. Ele permite olhar o mesmo fluxo de trás para frente. Isso é importante porque, em duplicidade, às vezes a pergunta começa no produto final e volta para as fontes. Dois produtos podem parecer diferentes olhando para frente, mas revelar contexto parecido quando olhamos seus ancestrais.

O canal C2 marca a diagonal de camada. A diagonal é um espaço especial porque identifica posições dos próprios nós. Ao marcar SOR, SOT e SPEC, o tensor informa ao modelo onde estão fontes, transformações e produtos. Isso reduz ambiguidade entre cenários.

O canal C3 é a impressão de grau. Ele resume conectividade: quantas entradas e saídas aparecem ao redor de um nó. Tabelas com fan-in alto podem representar agregações; tabelas com fan-out alto podem representar fontes reutilizadas. Esses sinais ajudam a diferenciar coincidência de redundância.

O canal C4 é o viés de rota de linhagem. Ele orienta a leitura para caminhos coerentes com a progressão SOR–SOT–SPEC. Isso importa porque a matriz é 2D, mas a história do dado tem direção e camadas. O canal não cria uma aresta nova; ele informa ao modelo quais trajetórias são mais compatíveis com a arquitetura.

O canal C5 é a máscara esparsa. Ele separa vazios reais de evidência estrutural. Em matrizes de adjacência, a maioria das células está vazia. A máscara ajuda o modelo a não tratar todo zero como informação igualmente relevante.

Na prática: Quando apresentar a ferramenta, use o grafo para explicar o domínio, a matriz para explicar a entrada computacional e os canais para explicar por que VMamba-Mesh é diferente de usar uma imagem simples do grafo.

O caminho treinável usado nas versões VMamba-T e VMamba-Mesh-T é:

par de grafos/subgrafos

-> CanonSort

-> tensor C0-C5

-> patch embedding

-> blocos estilo VSS/SS2D

-> pooling

-> head neural de par

-> logit

-> sigmoid

-> score contínuo

-> threshold

-> duplicado / não duplicado

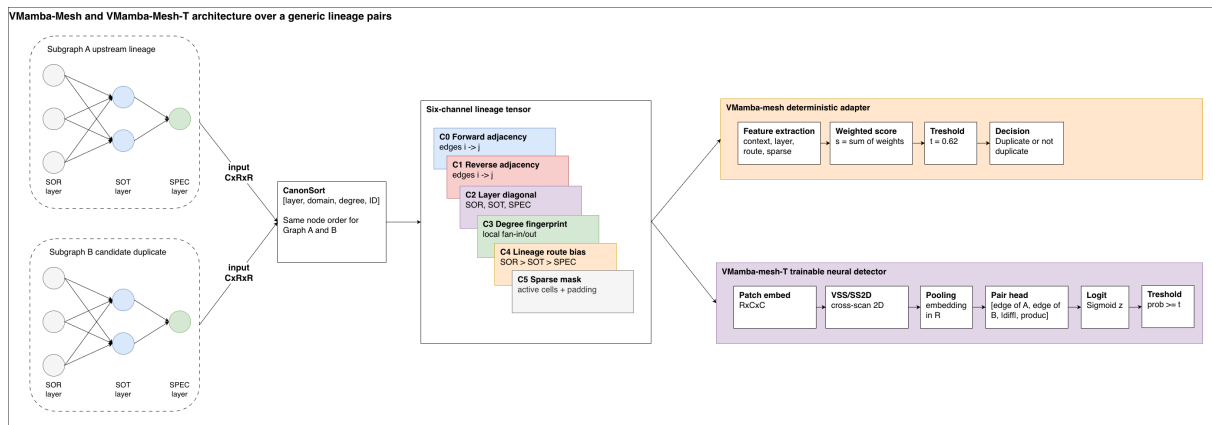


Figure 3.2: Arquitetura operacional: caminho determinístico VMamba-Mesh e caminho treinável VMamba-Mesh-T.

3.2 O que entra e o que sai do modelo

A unidade de decisão é sempre um par. O usuário pode olhar para dois nós, duas tabelas ou dois contextos de linhagem. O Isomera extrai o contexto do par, ordena os nós, monta os canais e entrega o tensor ao modelo. Na versão determinística, as features estruturais alimentam um score ponderado. Na versão treinável, os canais passam pelo backbone neural.

Na versão treinável, o fluxo é: tensor 2D, patch embedding, blocos VSS/SS2D, pooling e head neural. O head produz um logit. O logit passa por uma sigmoid e vira um score contínuo entre 0 e 1. Depois entra o threshold. Se o score for maior ou igual ao threshold, o par é classificado como duplicado. Se for menor, o par fica fora da lista de duplicidades.

Essa saída contínua é importante porque permite calibração. Dois pares podem ser positivos, mas com confiança diferente. Dois pares podem ficar perto do threshold e merecer

revisão humana. O Isomera preserva esse caminho porque, em governança de dados, a decisão precisa ser auditável.

Chapter 4

Mapa geral do aplicativo

4.1 Home

Onde clicar: Abrir o aplicativo e permanecer em Home.

A tela inicial resume o fluxo operacional. Ela mostra que o Isomera não é apenas uma tela de visualização: ele conecta materialização de grafos, validação de pares, treino de modelos, benchmark e geração de evidências.

A Home deve ser lida como uma mesa de controle. O menu lateral separa módulos de trabalho, e cada módulo representa uma etapa da vida de um estudo. Primeiro vem a escolha ou criação do cenário. Depois vem a execução dos detectores. Em seguida aparecem treino, análise, relatórios, administração e documentação. Essa organização evita que a ferramenta vire apenas uma coleção de scripts: cada função tem uma rota visível, uma intenção e um lugar para voltar.

Para um usuário novo, a Home responde a três perguntas. A primeira é onde começo. A resposta depende do objetivo: se quer executar algo pronto, comece em Benchmark; se quer criar um cenário, comece em Scenario Studio; se quer estudar VMamba-Mesh, comece em Study Lab. A segunda é onde verifico os resultados. A resposta é Research Reports e Model Reports. A terceira é onde vejo estado técnico e logs. A resposta é Admin e Logs.

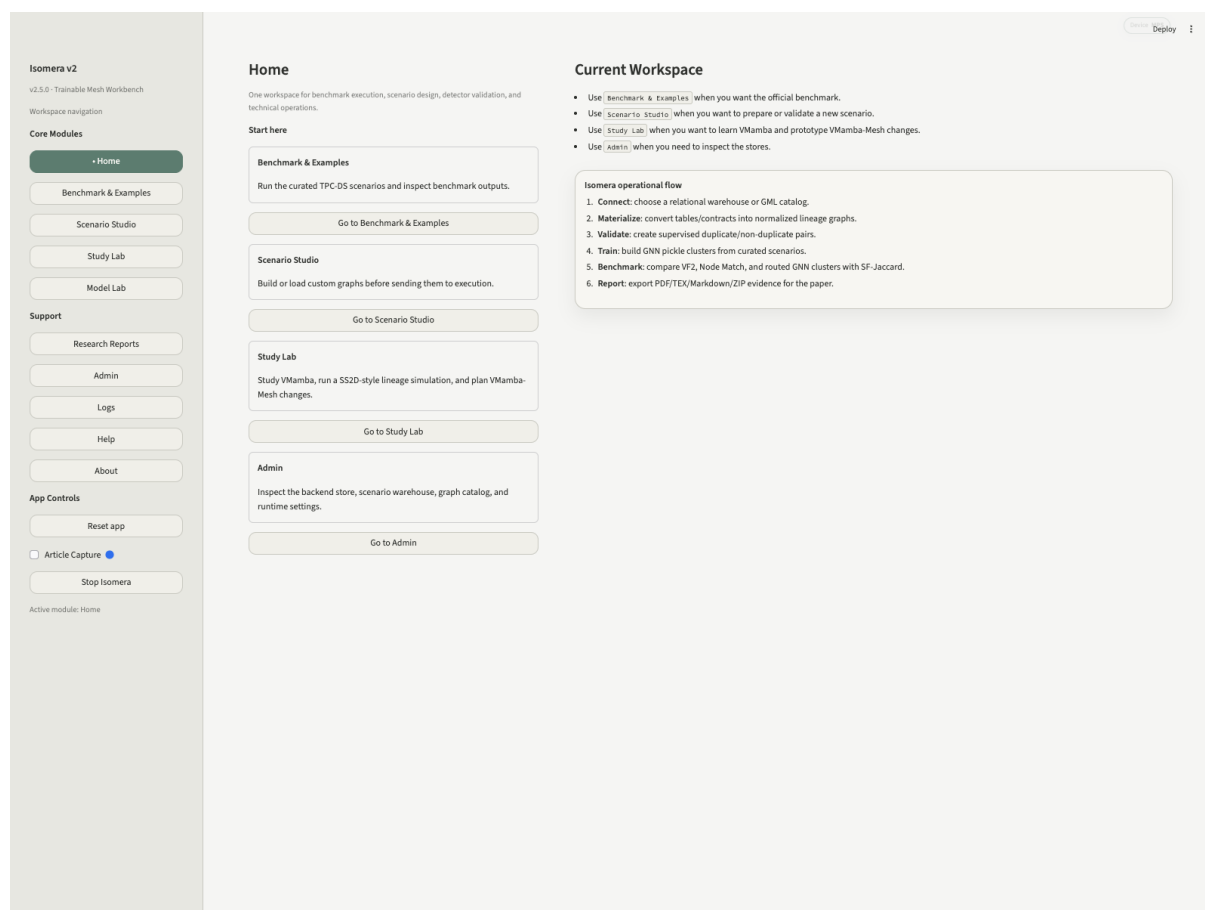


Figure 4.1: Home do Isomera. O menu lateral é a referência principal de navegação.

4.2 Jornadas principais

O Isomera pode ser usado por jornadas. A jornada de demonstração começa em Home, passa por Benchmark, abre Study Lab para explicar o modelo, mostra Model Interpretability e termina em Research Reports. A jornada de pesquisa começa em Scenario Studio, cria ou carrega cenários, valida pares, treina modelos, executa benchmarks e salva relatórios. A jornada de engenharia começa em Admin, confere conexões, abre Scenario Studio para materialização, registra modelos no Model Lab e configura execução no Benchmark.

Essa separação é importante porque a mesma interface atende públicos diferentes. Um professor ou avaliador quer ver evidência e método. Um engenheiro de dados quer saber como conectar a ferramenta a uma arquitetura real. Um pesquisador quer reproduzir números, alterar hiperparâmetros e comparar versões. O manual usa as mesmas telas, mas explica o que cada público deve observar.

Dica! Para uma primeira demonstração, siga uma linha simples: Home, Benchmark, Study Lab, Interpretability e Research Reports. Essa ordem conta o produto sem interromper a narrativa.

4.3 Rotas principais

Rota	Uso
Benchmark & Examples	Executar benchmarks, comparar detectores e reproduzir campanhas salvas.
Scenario Studio	Criar, carregar, validar e publicar cenários de grafo.
Study Lab	Estudar VMamba, VMamba-Mesh, canais, treino, relatórios e interpretabilidade.
Model Lab	Ver pickles, famílias registradas e roteamento de modelos por cenário.
Research Reports	Abrir relatórios, Markdown, PDFs, manifests e pacotes ZIP.
Admin	Ver backend, warehouse de cenários, Neo4j e configurações.
Logs	Auditar logs da sessão e logs de terminal.
Help	Abrir apresentação VMamba-Mesh e Tech Docs.
About	Ver versão, autores, contexto e notas de funcionalidades.

4.4 Cobertura visual deste manual

A tabela abaixo funciona como inventário de cobertura. Cada rota principal foi aberta no Isomera, capturada em PNG e associada a uma explicação operacional. Para páginas longas, o manual também mantém capturas com scroll. Isso é importante porque várias funcionalidades ficam abaixo da primeira dobra da tela, principalmente treino, relatórios, administração e documentação técnica.

Rota ou aba	PNG principal	Validação visual
Home	01_home.png	Menu lateral, cartões de jornada e acesso às áreas principais.
Benchmark / Reproducibility	02_benchmark_article_reproducibility.png	Manifesto, artigo, modo de execução e trace reprodutível.
Benchmark / Run	03_benchmark_run.png	Seletor de benchmark, modelos, política e execução.
Benchmark / Concepts	04_benchmark_concepts.png	Conceitos, cenários, pares reais e matriz de benchmark.
Scenario Studio	06_scenario_studio_overview.png	Entrada por warehouse, GML ou construtor manual.
Scenario Studio / scroll	07_scenario_studio_source_scroll.png; 08_scenario_studio_validation_scroll.png; 10_scenario_model_training.png	Etapas de origem, validação e treino no fluxo vertical.
Study / Workbench	11_study_deep_learning_workbench.png	Comparação lado a lado de modelos e pares.
Study / Original VMamba	12_study_original_vmamba.png	Conceitos do VMamba original e partes da arquitetura.
Study / Official Runtime	13_study_official_runtime.png	Runtime oficial como referência externa opcional.
Study / SS2D	14_study_ss2d_demo.png	Demonstração de scan 2D em linhaagem.

Study / Mesh Changes	15_study_mesh_changes.png	CanonSort, DiagFP, MeshSS2D, HierInit e SparseGate.
Study / Train	16_study_train_model_adapter.png	Treino, device, batches, loss, negative ratio e artefatos.
Study / Reports	17_study_model_reports.png	Campanhas, CSVs, figuras e manifests.
Study / Interpretability	18_study_model_interpretability.png	Grafo, matriz, canais, saliency e trace.
Study / Knowledge Base	19_study_knowledge_base.png	KBs internas sobre VMamba e VMamba-Mesh.
Model Lab	20_model_lab.png	Pickles, famílias de modelos e roteamento.
Research Reports	21_research_reports.png	Pacotes de relatório, Markdown, anexos e ZIP.
Admin	22_admin_overview.png— 26_admin_settings.png	Stores, warehouse, Neo4j e configurações.
Logs	27_logs_session.png, 28_logs_terminal.png	Auditoria da sessão e logs de terminal.
Help	29_help_presentation.png, 30_help_tech_docs.png	Apresentação e documentação técnica.
About	31_about.png	Versão, autores e notas de funcionalidades.

Chapter 5

Benchmark & Examples

Onde clicar: Menu lateral → Benchmark & Examples.

Essa área responde à pergunta: “como executo e comparo detectores nos benchmarks empacotados?” Ela possui três abas conceituais: **Article Reproducibility**, **Run Benchmark** e **Concepts**.

O Benchmark é o lugar onde o Isomera deixa de ser apenas exploratório e passa a ser mensurável. Uma hipótese de modelo só tem valor se puder ser testada contra cenários conhecidos. Por isso a tela começa pedindo um benchmark. O benchmark define quais grafos existem, quais pares são considerados verdadeiros, quais famílias de modelo podem rodar e quais resultados podem ser comparados.

Na prática, o usuário não deve abrir essa tela pensando apenas em apertar um botão. Ele deve ler a tela como um contrato experimental. O benchmark escolhido define o universo do teste. O modelo escolhido define a estratégia de detecção. A política de execução define se o app vai rodar algo rápido, algo completo ou algo já preparado para reprodutibilidade. O resultado só é comparável se essas três decisões forem explícitas.

Dica! Antes de executar, confirme três escolhas: benchmark, modelo e política de execução. Sem isso, o número final perde contexto.

Por trás do clique: Quando um benchmark roda, o Isomera carrega o grafo do cenário, enumera pares candidatos, chama o detector registrado e compara a lista retornada com o ground truth. A partir dessa comparação nascem TP , FP , FN , tempo de execução e métricas agregadas.

5.1 Article Reproducibility

Onde clicar: Benchmark & Examples → Article Reproducibility.

Aqui o usuário escolhe um manifesto de reprodução, seleciona modo rápido, evidência do artigo ou pacote completo, e executa uma reprodução com trace auditável. O trace registra parâmetros, seeds, arquivos lidos, arquivos escritos, métricas, valores esperados, tolerâncias e status de comparação.

Essa tela existe para resolver um problema comum em pesquisa aplicada: depois que um resultado aparece em gráfico ou relatório, alguém precisa conseguir repetir o caminho. O manifesto de reprodução é a peça que guarda essa memória. Ele diz qual campanha

deve ser rodada, quais cenários entram, quais modelos estão habilitados, quais seeds foram usadas, quais artefatos são esperados e quais tolerâncias fazem sentido.

A palavra “reproducibility” aqui não significa apenas gerar o mesmo número. Significa registrar o caminho. Se um cenário não estiver disponível, o status precisa mostrar isso. Se uma métrica ficar dentro da tolerância, o trace deve indicar. Se uma figura não existir, o pacote deve mostrar que ela não foi produzida. Esse comportamento é fundamental para a ferramenta ser usada em banca, laboratório ou empresa.

Na prática: Para uma demonstração curta, use o modo rápido com um cenário/par. Para validar um resultado de relatório, use o modo de campanha. Para fechar um pacote completo, use a reprodução completa e depois abra Research Reports.

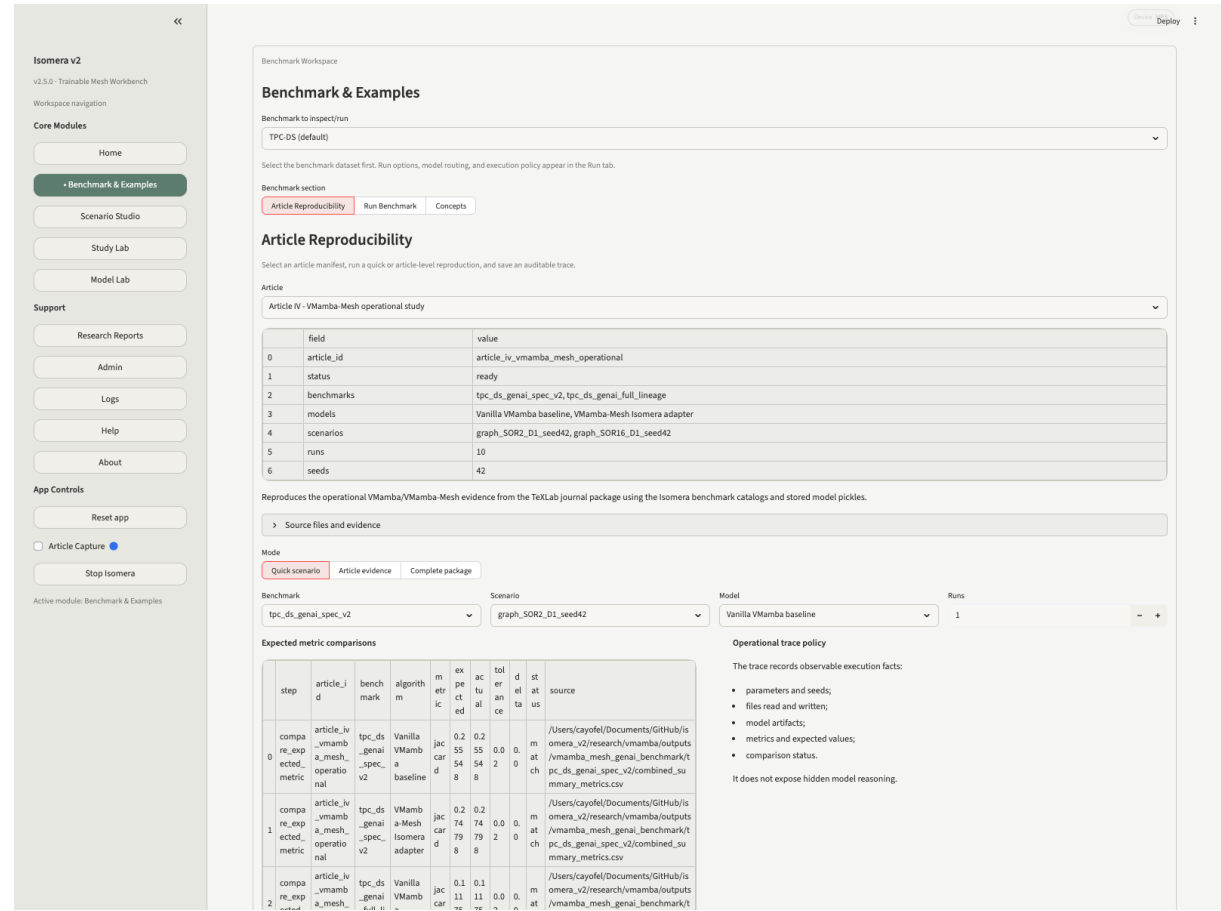


Figure 5.1: Article Reproducibility. A tela mostra manifesto, benchmarks, modelos, cenários, seeds e política de trace.

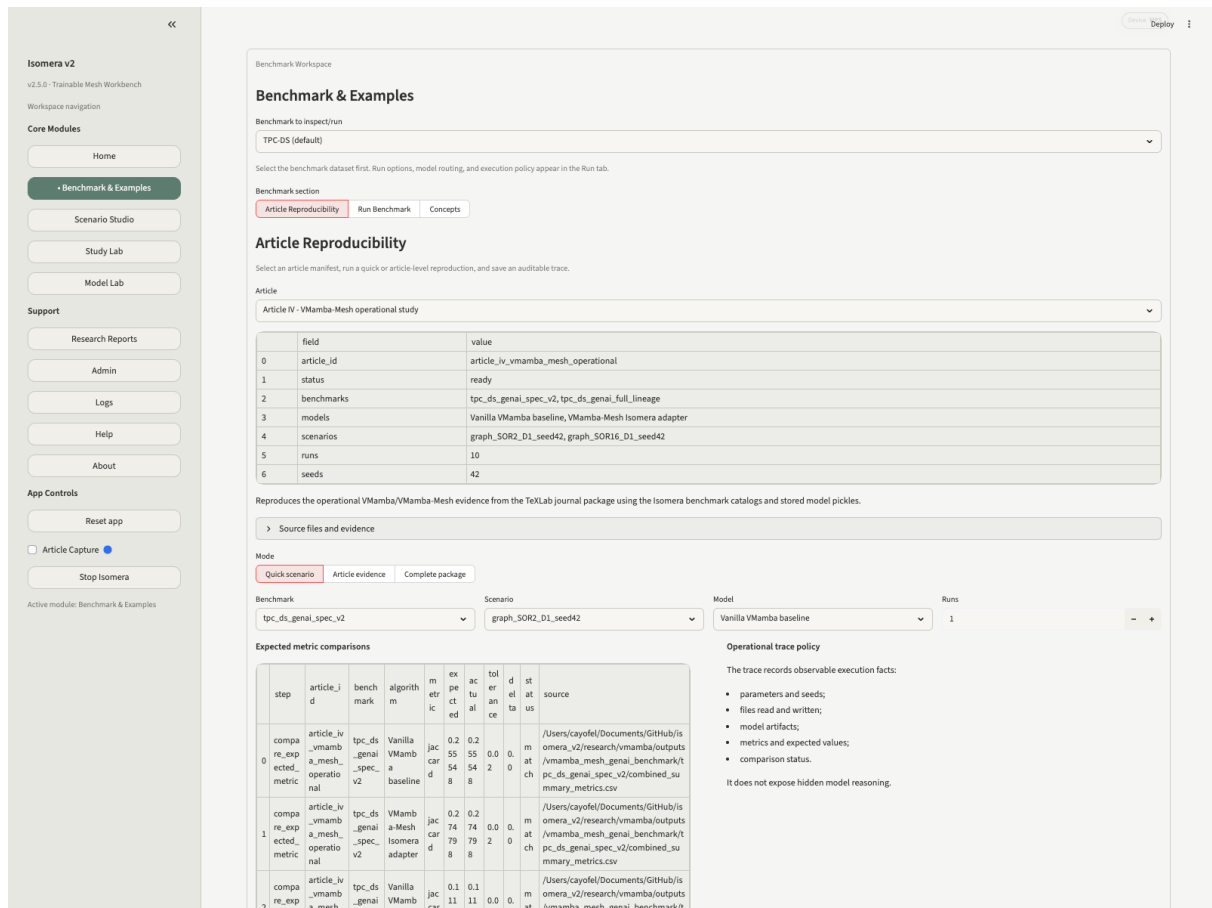


Figure 5.2: Article Reproducibility com scroll. A captura confirma que a rota mostra controles e evidências abaixo da primeira dobra da tela.

5.2 Run Benchmark

Onde clicar: Benchmark & Examples → Run Benchmark.

Essa aba executa detectores sobre o benchmark selecionado. O usuário escolhe dataset, política de execução, família de modelo e opções de roteamento. É a rota para medir Jaccard, SF-Jaccard, tempo mediano e métricas por cenário.

O Run Benchmark é a tela de execução direta. Ela é mais operacional do que a reprodução por manifesto. Aqui o usuário quer saber: com este benchmark e este modelo, o que acontece agora? A tela precisa deixar claro quais modelos estão disponíveis, quais dependências estão presentes, se o detector é determinístico ou treinável, e onde o resultado será registrado.

Os detectores têm naturezas diferentes. VF2 e Node Match são baselines determinísticos. Eles ajudam a entender casos estruturais exatos ou quase exatos. GNN e GIN são modelos treinados sobre representações de grafo. VMamba e VMamba-Mesh trabalham com tensorização matricial. VMamba-T e VMamba-Mesh-T usam rede neural treinável. O valor do Run Benchmark é colocar todos sob o mesmo contrato de saída: uma lista de pares considerados duplicados.

Por trás do clique: A interface não compara modelos por opinião. Ela transforma a decisão de cada detector em conjunto de pares e calcula métricas. Isso permite dizer que um modelo foi melhor em Jaccard, outro foi melhor em tempo, e outro teve melhor SF-Jaccard.

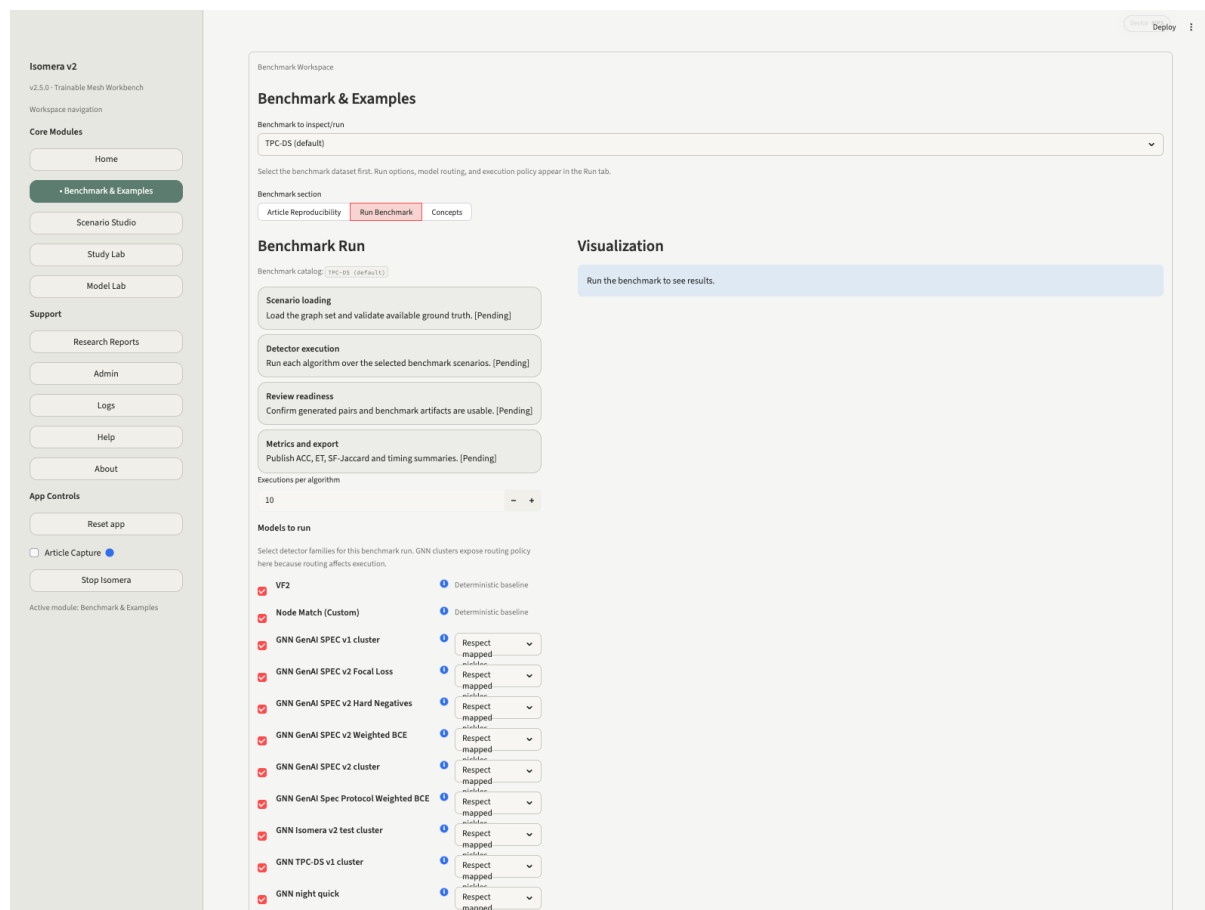


Figure 5.3: Run Benchmark. Esta rota concentra execução, roteamento de modelos e comparação operacional.

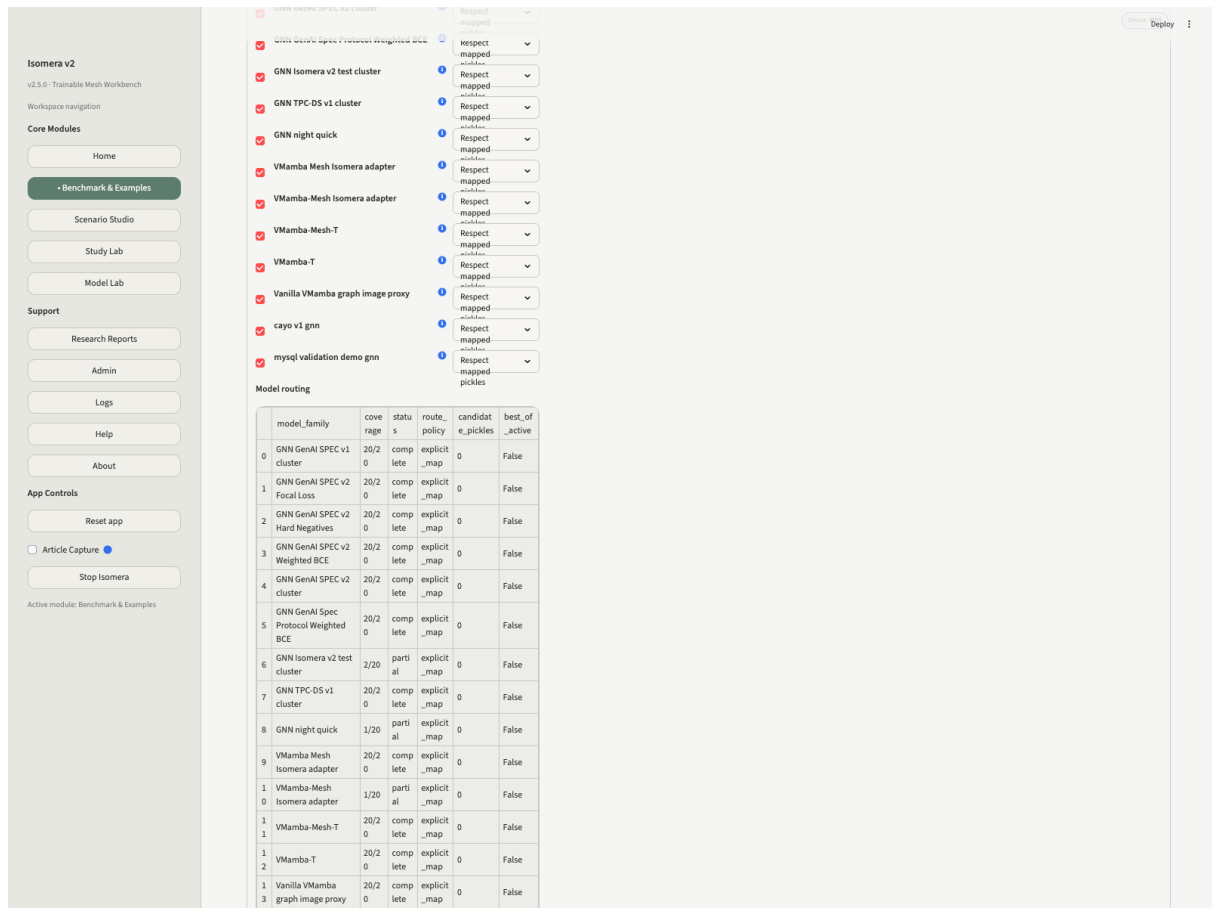


Figure 5.4: Run Benchmark com scroll. Use esta área para conferir opções adicionais de execução e o espaço de resultados.

5.3 Concepts

Onde clicar: Benchmark & Examples → Concepts.

A aba de conceitos documenta o que o benchmark representa, quais famílias de detectores existem e por que a comparação precisa registrar métricas e contexto.

Essa aba é útil antes de executar. Ela explica o que significa um nome de cenário, como `graph_SOR16_D1_seed42`. O número depois de SOR indica a quantidade de fontes. O número depois de D indica domínios. A seed permite reconstrução controlada. Quando alguém olha apenas para uma métrica final, perde essa informação. O Concepts existe para recuperar o contexto.

Também é nessa visão que o usuário entende cobertura de roteamento. Um modelo pode existir, mas não estar mapeado para todos os cenários. Um pickle pode cobrir um subconjunto. Uma família pode estar registrada, mas incompleta. A tabela de cobertura evita que o usuário confunda ausência de rota com resultado ruim de modelo.

Na prática: Antes de rodar um benchmark novo, abra Concepts. Verifique cenários, pares reais, matriz SOR x Domínios e cobertura de roteamento. Isso evita interpretar resultado parcial como resultado final.

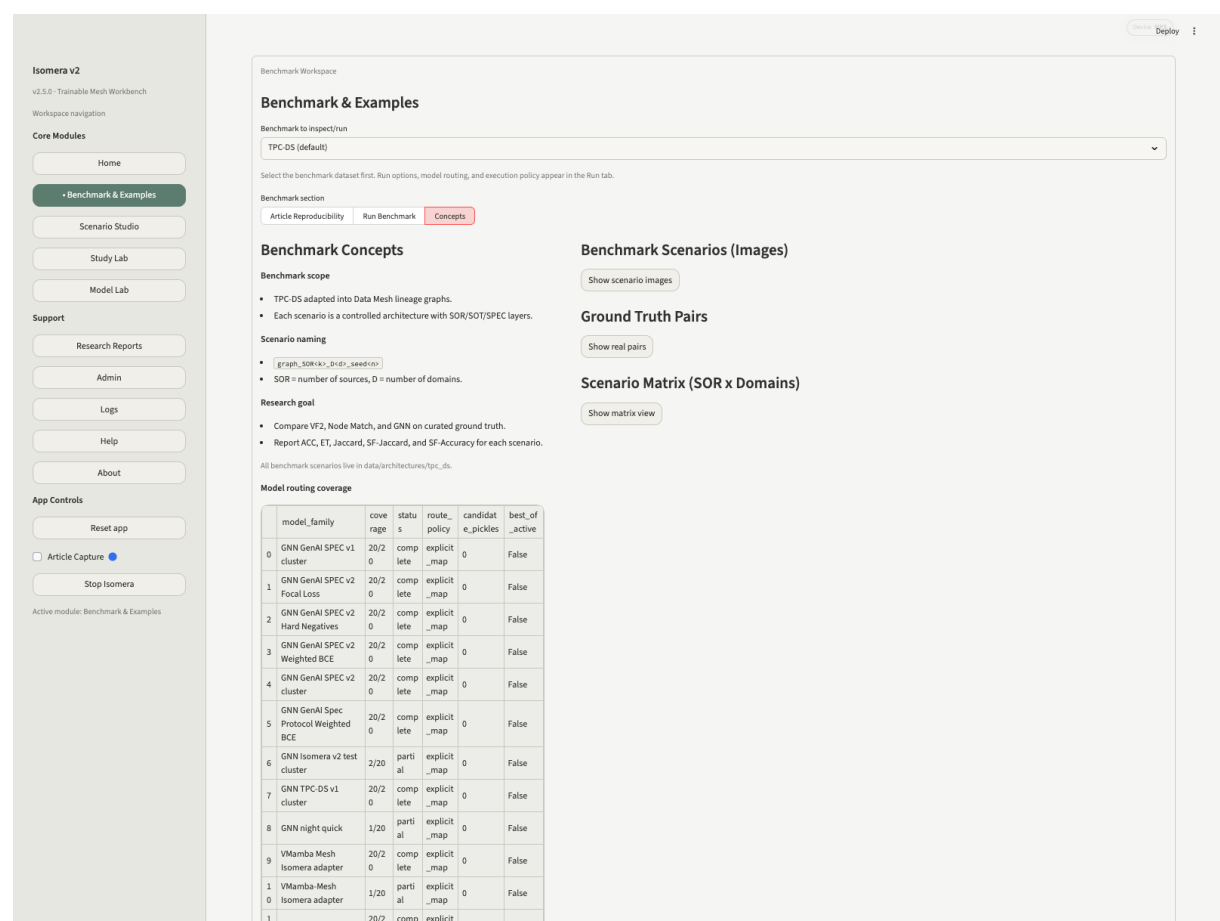


Figure 5.5: Concepts. O objetivo é explicar o benchmark antes de executar.

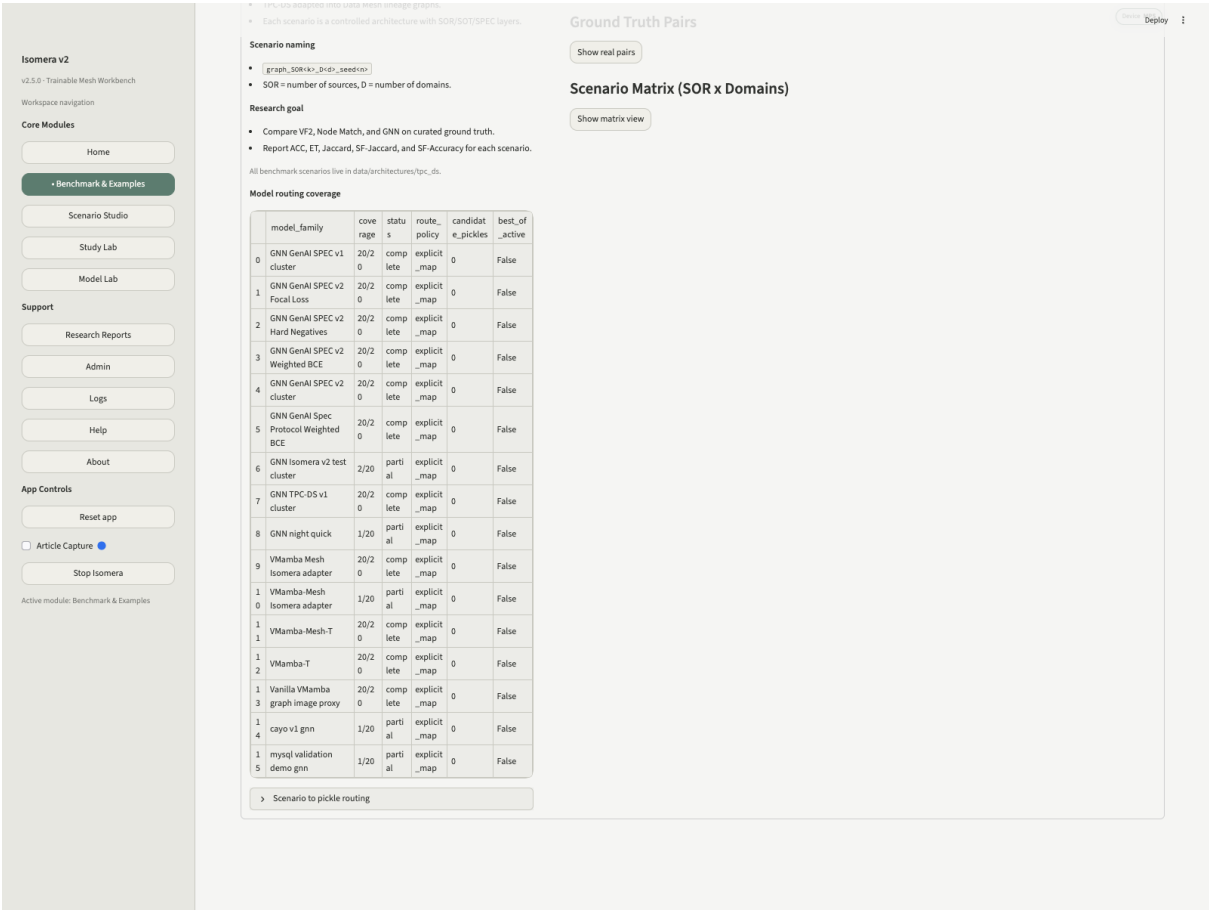


Figure 5.6: Concepts com scroll. A tela expande explicações de cenários, pares reais e matriz de benchmark.

Chapter 6

Scenario Studio

Onde clicar: Menu lateral → Scenario Studio.

O Scenario Studio é a rota para preparar cenários. Ele é importante porque nenhum detector é melhor do que o benchmark e os labels que o alimentam. Aqui entram fontes, grafos, pares candidatos, validação humana ou assistida por LLM e publicação para o benchmark.

Se o Benchmark é o lugar de medir, o Scenario Studio é o lugar de construir o que será medido. Ele organiza a entrada de dados, a materialização do grafo, a inspeção visual, a escolha de pares candidatos e o preparo para treino. Em uma empresa, essa etapa corresponderia ao momento em que a equipe conecta catálogo, warehouse, linhagem ou arquivo GML e decide quais tabelas precisam entrar no estudo.

Dica! Não avance para treino com um grafo mal validado. O Scenario Studio define o chão do experimento: fonte, pares, filtros e ground truth.

O usuário deve olhar essa tela com atenção porque ela controla o ground truth. Um benchmark com pares mal definidos pode favorecer o modelo errado, penalizar um modelo correto ou produzir uma conclusão que não se sustenta fora do laboratório. Por isso a jornada é vertical e explícita: primeiro entra o cenário, depois aparecem filtros, depois vem revisão, depois publicação e, quando fizer sentido, treino.

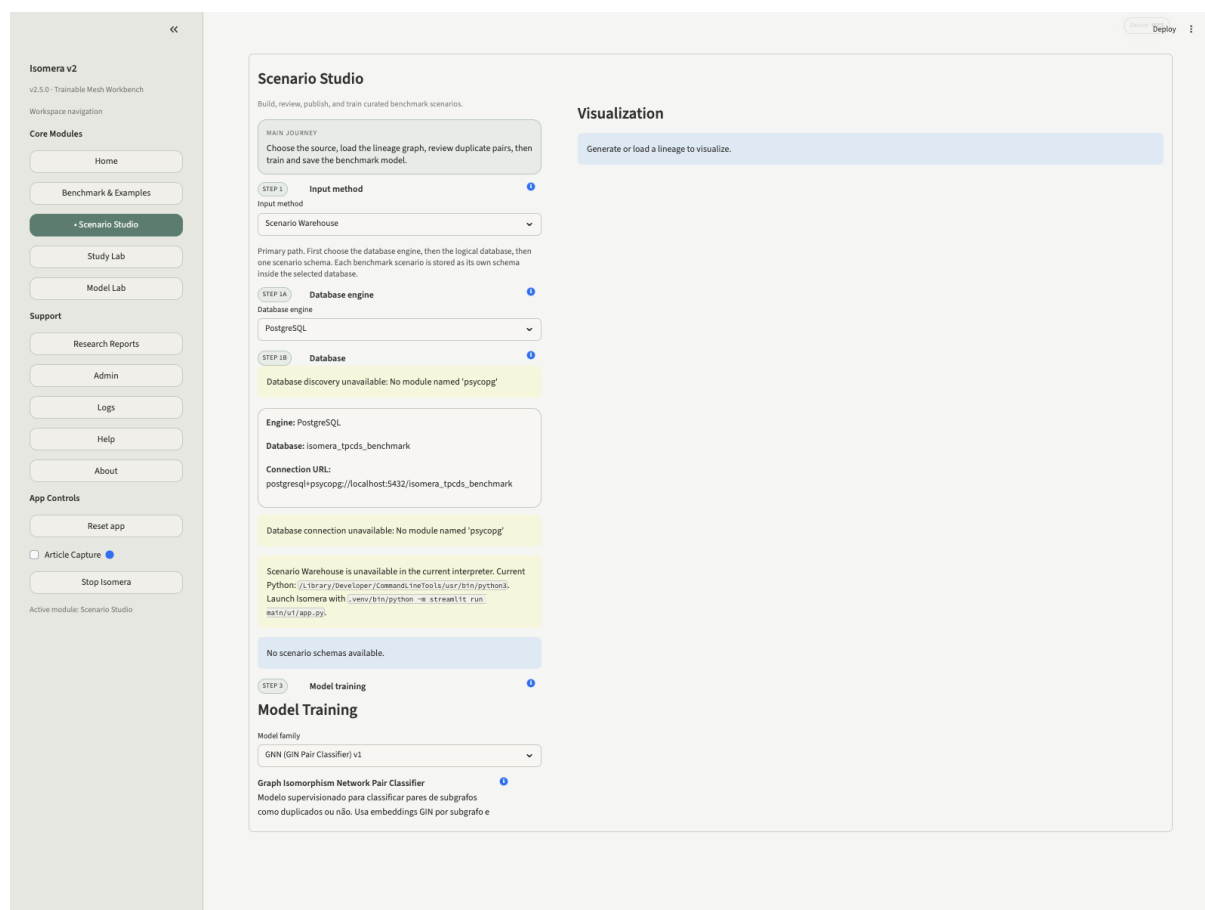


Figure 6.1: Scenario Studio. Espaço de criação e validação de cenários.

6.1 Fluxo vertical do Scenario Studio

O Scenario Studio não é um conjunto de abas horizontais. Ele funciona como uma jornada vertical. No topo, o usuário escolhe a fonte do cenário. No meio, valida o grafo, os pares candidatos e os filtros. No fim, registra treino e publicação. Por isso a cobertura visual usa capturas com scroll.

Essa escolha de design é intencional. Uma aba horizontal daria a sensação de etapas independentes. No Scenario Studio, as etapas são dependentes. Não faz sentido revisar pares antes de carregar o grafo. Não faz sentido treinar antes de publicar ou selecionar cenários. Não faz sentido medir um modelo sem saber de onde vieram os labels. A tela vertical força uma leitura sequencial.

Onde clicar: Scenario Studio → escolher Input method.

As opções principais são **Scenario Warehouse**, **GML Catalog** e **Manual Builder**. A primeira é o caminho de produção para benchmarks materializados em banco relacional. A segunda carrega grafos portáteis. A terceira é útil para criação manual ou demonstrações pequenas.

No modo **Scenario Warehouse**, o app conversa com um banco relacional. Cada cenário pode existir como schema, com tabelas e metadados próprios. Esse caminho é o mais próximo de um uso corporativo, porque preserva a ideia de que a arquitetura real vive em bancos, catálogos e contratos de dados. No modo **GML Catalog**, o grafo já está exportado

e pode ser carregado diretamente. No modo **Manual Builder**, o usuário cria ou ajusta um caso pequeno para estudo.

Por trás do clique: Ao carregar um cenário, o Isomera materializa um grafo dirigido. A materialização normaliza nós, arestas, camadas e metadados. Depois guarda esse grafo em sessão para que as etapas seguintes usem exatamente a mesma estrutura.

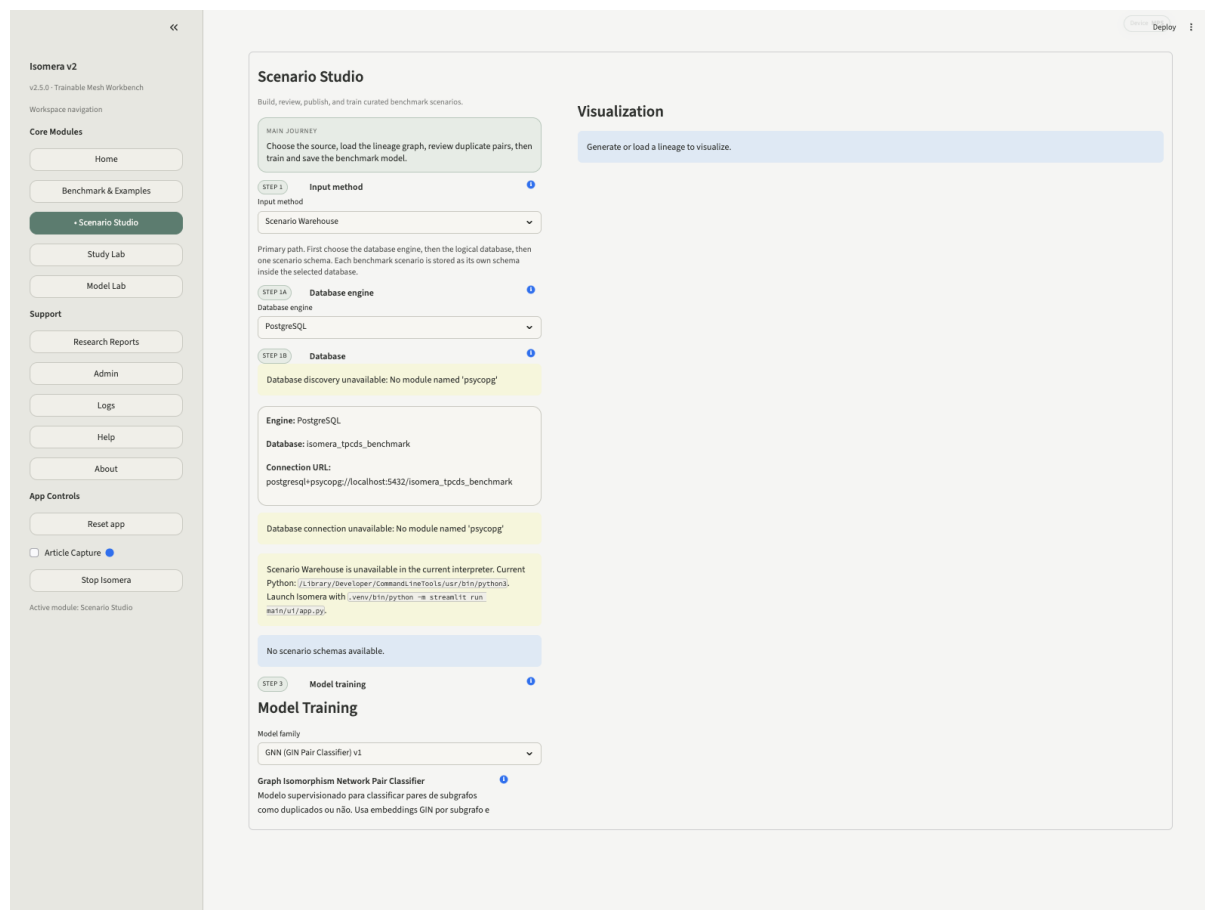


Figure 6.2: Scenario Studio com scroll na área de origem. A tela mostra engine, banco, schema e carregamento do cenário.

Onde clicar: Continuar descendo a página até filtros, pares candidatos e validação.

Depois que o cenário é carregado, o Isomera permite revisar pares candidatos. Essa parte é essencial para benchmarks confiáveis: o modelo só aprende corretamente se o ground truth for rastreável e se as regras de candidatura forem claras.

Os filtros de pares existem para reduzir ruído. Em um grafo com muitos nós, comparar todos contra todos pode gerar uma explosão de pares sem significado. O usuário pode restringir por camada, domínio, grau, assinatura de pais ou assinatura de filhos. Cada filtro representa uma hipótese. Se o filtro exige mesma camada, ele assume que duplicidade costuma ocorrer entre objetos do mesmo nível. Se exige mesmo domínio, assume que duplicidade é mais provável dentro de um recorte de negócio. Se permite SPEC completo, a ferramenta olha para o contexto upstream, não apenas para o nó final.

A revisão de pares é onde o conhecimento humano entra. A ferramenta sugere candidatos e organiza evidências, mas a validação final de ground truth precisa ser rastreável. Esse

cuidado é o que permite depois treinar modelos e dizer que as métricas medem algo real, e não apenas uma coincidência estrutural.

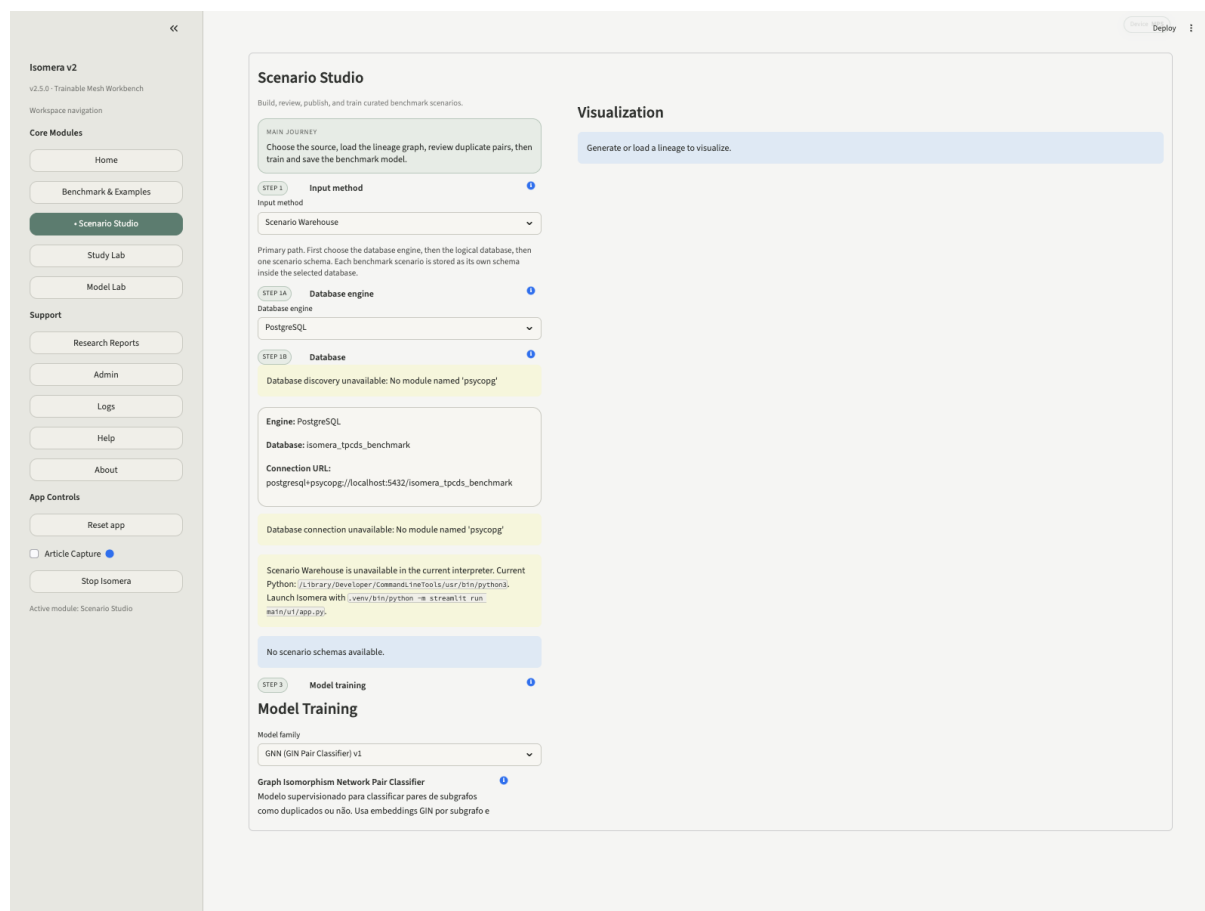


Figure 6.3: Scenario Studio na área de validação. O usuário controla camadas, filtros e fila de revisão de pares.

6.2 Model Training no Scenario Studio

Onde clicar: Scenario Studio → Model Training, quando disponível.

Essa rota concentra treinamento de artefatos `.pkl` e preparação de modelos para o Model Lab e o Benchmark. Quando as dependências estão instaladas, o usuário pode treinar modelos e registrar hiperparâmetros, progresso e artefatos.

O treino dentro do Scenario Studio foi pensado para modelos que dependem diretamente dos cenários publicados. O usuário escolhe família, parâmetros, loss, otimizador, split de treino, estratégia de balanceamento e device. A interface registra esses valores porque, sem hiperparâmetros, um pickle é apenas um arquivo. Com hiperparâmetros e manifest, ele vira um artefato reproduzível.

Em modelos de duplicidade, o desbalanceamento é uma preocupação constante. Normalmente existem muito mais pares não duplicados do que duplicados. Por isso aparecem opções de balanceamento, loss ponderada, negative ratio e hard negatives. Essas opções alteram o tipo de erro que o modelo aprende a evitar. Um negative ratio alto expõe o

modelo a mais exemplos negativos; hard negatives forçam atenção a pares que parecem positivos, mas não são.

Na prática: Para uso real, nunca salve um modelo sem registrar benchmark, cenários usados, seed, loss, otimizador, epochs, device e política de negativos. Esses dados precisam aparecer no relatório para que outra pessoa entenda o que foi treinado.

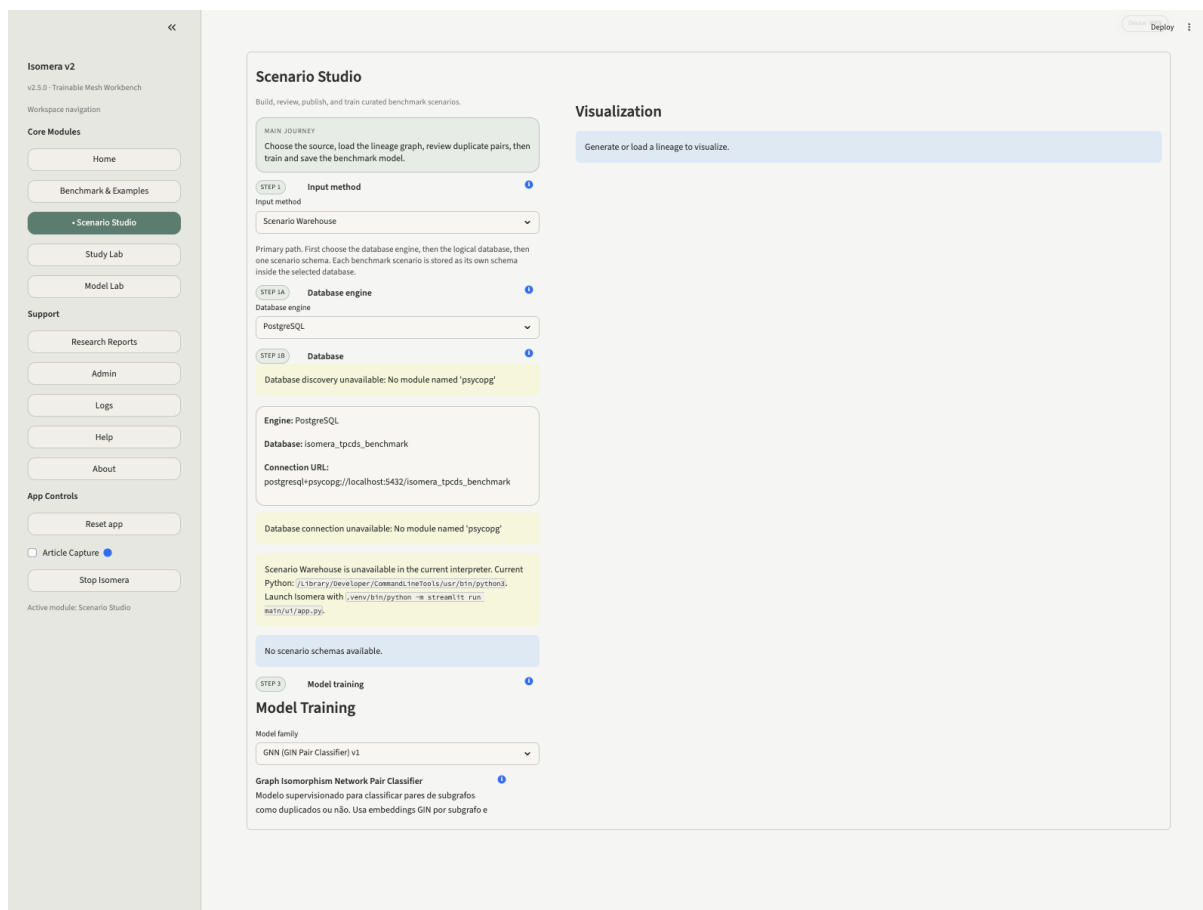


Figure 6.4: Model Training. Caminho prático para produzir artefatos treináveis.

Chapter 7

Study Lab

Onde clicar: Menu lateral → Study Lab.

O Study Lab é o laboratório de aprendizado e validação do VMamba-Mesh. Ele une explicação conceitual, workbench de comparação, treino, relatórios, interpretabilidade e base de conhecimento.

Essa é a área mais didática do Isomera. Ela foi criada para responder a uma pergunta que não aparece em uma métrica final: o que está acontecendo dentro do modelo? O Study Lab permite sair do resultado agregado e voltar ao par, ao cenário, ao tensor, ao canal, ao score e à explicação local. É onde a ferramenta deixa de ser uma caixa de execução e vira material de estudo.

Para quem está aprendendo, o caminho recomendado é começar pelo Deep Learning Workbench, passar por Original VMamba, rodar a demo SS2D, abrir VMamba-Mesh Changes, depois Train Model Adapter, Model Reports, Model Interpretability e Knowledge Base. Esse caminho reproduz a história da adaptação: primeiro comparar modelos, depois entender a inspiração, depois ver a leitura 2D, depois ver o que foi alterado, depois treinar, depois analisar.

Dica! Use o Study Lab para explicar o “porquê” antes do “quanto”. Primeiro mostre grafo, matriz e canais; depois mostre score, métricas e relatórios.

7.1 Deep Learning Workbench

Onde clicar: Study Lab → Deep Learning Workbench.

O Workbench permite selecionar benchmark, cenário, par e modelos A/B. Ele é útil para mostrar, em uma mesma tela, como a decisão nasce do grafo e como os modelos diferem.

O Deep Learning Workbench é uma bancada de comparação. O usuário escolhe um par e coloca dois modelos lado a lado. Isso permite observar diferenças que uma tabela final não mostra. Um modelo pode selecionar mais pares, outro pode ser mais conservador. Um pode ter score alto no par escolhido, outro pode ficar próximo do threshold. Um pode usar apenas C0/C1; outro pode usar C0–C5.

Esse tipo de comparação é útil em vídeo, aula e depuração. Em vez de falar abstratamente que VMamba-Mesh usa canais, o usuário aponta para o par, muda o modelo e observa a saída. Em vez de explicar apenas que um threshold existe, ele pode mostrar o

score contínuo e a decisão binária.

Por trás do clique: Internamente, o Workbench chama os mesmos contratos usados no benchmark. Isso é importante: a tela não deve ser uma simulação desconectada. Ela precisa refletir o comportamento dos modelos registrados, dos benchmarks disponíveis e dos artefatos salvos.

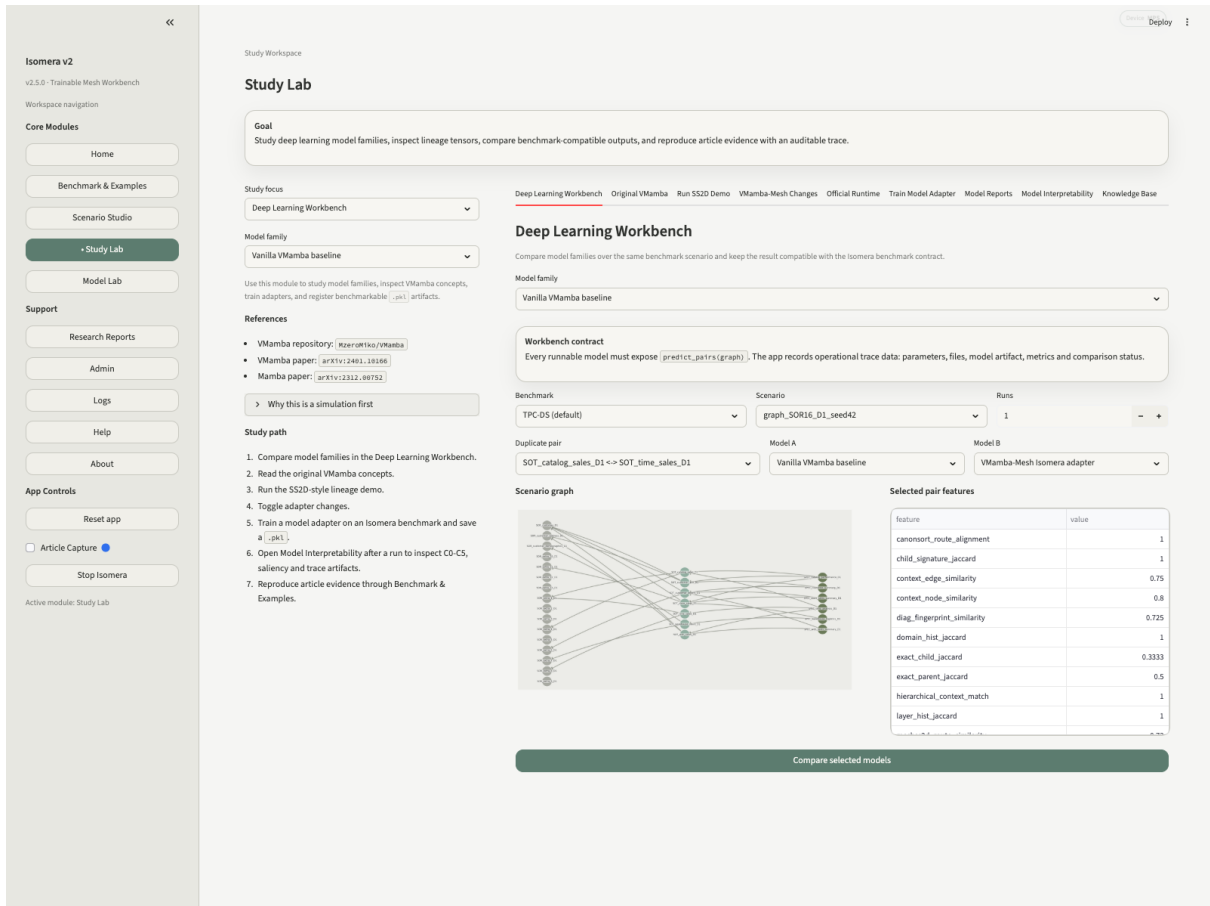


Figure 7.1: Deep Learning Workbench. Exemplo com benchmark, cenário, par, modelos e features do par selecionado.

7.2 Original VMamba

Onde clicar: Study Lab → Original VMamba.

Esta aba é a referência didática do VMamba original. Ela mostra o papel de patch embedding, VSS block, SS2D e head de classificação. O ponto mais importante para o Isomera é entender a analogia: no VMamba, a rede lê campos 2D; no Isomera, o grafo de linhagem vira campo 2D por meio de matriz de adjacência e canais estruturais.

O VMamba original trabalha sobre imagens ou mapas de features. A imagem entra como tensor, passa por patch embedding, atravessa blocos visuais de espaço de estados e chega a um head de classificação. A inspiração para o Isomera vem daí: se a linhagem pode ser representada como matriz, e se a matriz pode ser enriquecida com canais, então a leitura 2D pode ser reaproveitada em um domínio de governança de dados.

A adaptação, porém, não é copiar a rede e trocar a imagem por uma matriz qualquer. O grafo tem semântica. A diagonal tem papel. As camadas SOR, SOT e SPEC importam. A direção da aresta importa. A esparsidade importa. Por isso a aba Original VMamba aparece antes das mudanças: ela cria a base conceitual para entender o que precisa ser preservado e o que precisa ser alterado.

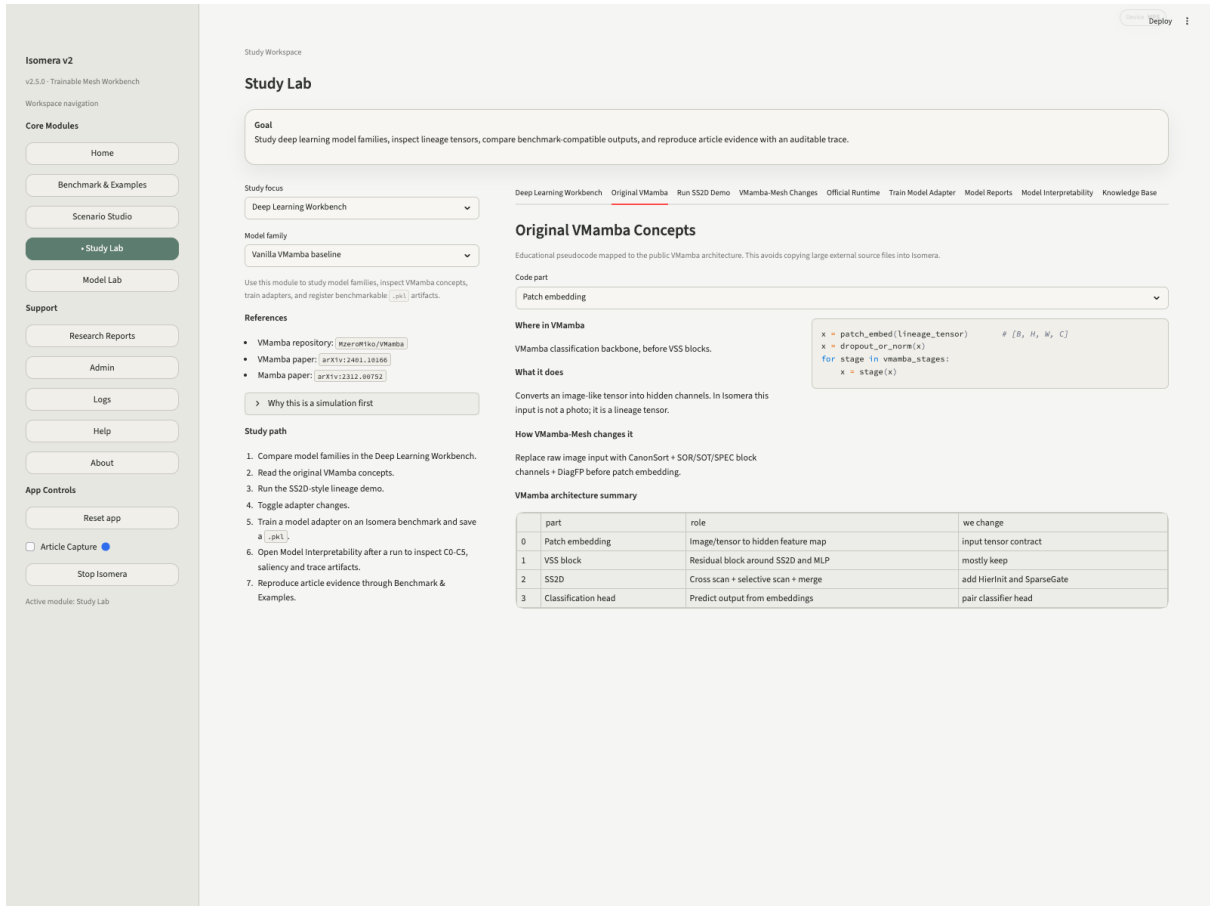


Figure 7.2: Original VMamba. A aba mostra pseudocódigo educacional e mapeia cada parte para a adaptação VMamba-Mesh.

7.3 Run SS2D Demo

Onde clicar: Study Lab → Run SS2D Demo.

Essa rota explica o papel do scan 2D. No VMamba original, o scan percorre imagens. No Isomera, a ideia é transferida para uma matriz/tensor de linhagem.

O scan 2D é uma forma de percorrer o campo em rotas. Em uma imagem, isso ajuda a capturar contexto horizontal, vertical e reverso. Em uma matriz de linhagem, a rota precisa respeitar o fato de que linhas e colunas representam nós ordenados. Quando o usuário muda a rota, o decaimento de memória, o ganho de entrada ou a força do SparseGate, ele está observando como o contexto se propaga pela matriz.

A demo SS2D é educacional, mas tem papel prático. Ela mostra por que a ordem dos nós muda a leitura, por que células vazias podem contaminar contexto e por que rotas múltiplas podem capturar padrões que uma varredura única perderia.

Na prática: Use esta aba quando precisar explicar para alguém técnico por que o Isomera não está simplesmente aplicando um classificador qualquer. A rota SS2D mostra a ponte entre matriz e leitura contextual.

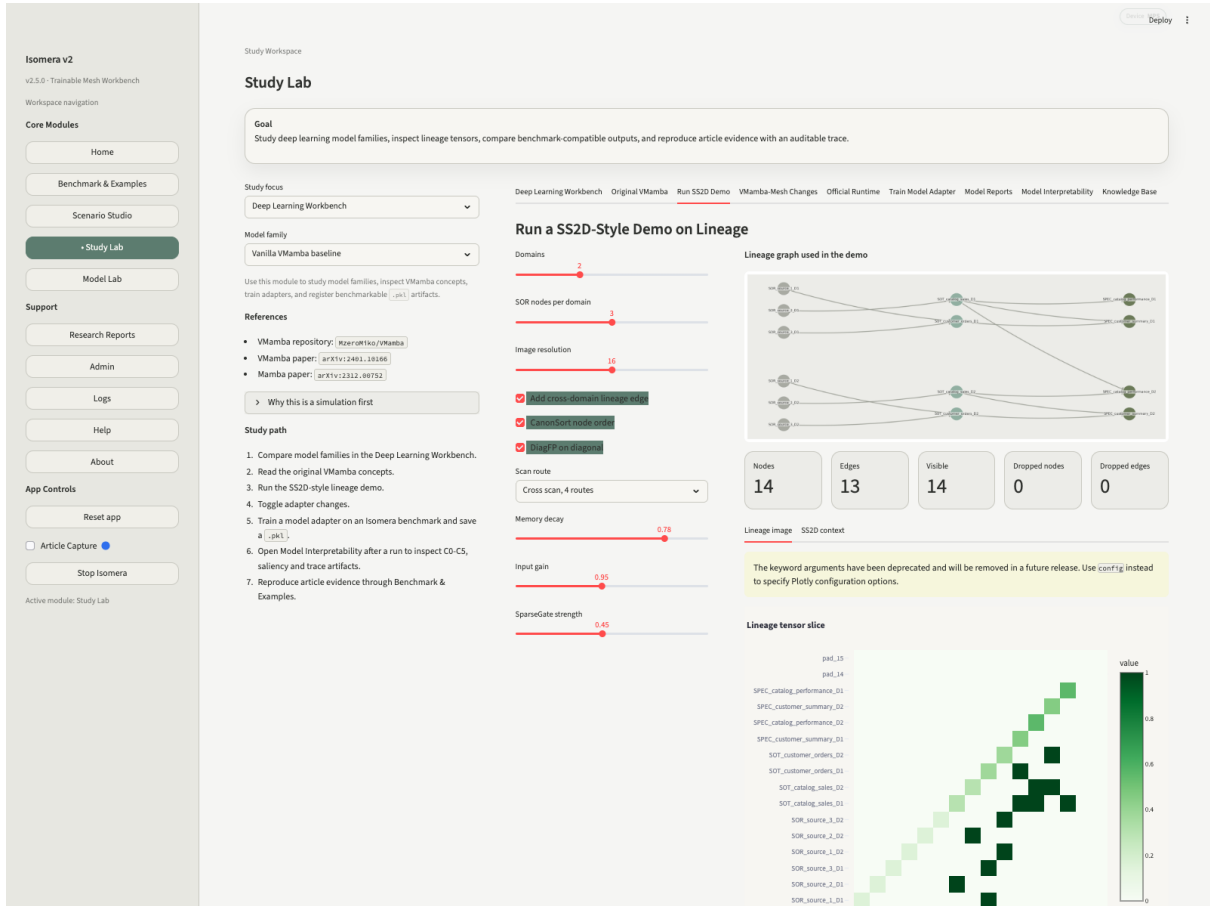


Figure 7.3: Run SS2D Demo. Demonstração didática de leitura 2D sobre representação de linhagem.

7.4 VMamba-Mesh Changes

Onde clicar: Study Lab → VMamba-Mesh Changes.

Essa aba documenta as adaptações: CanonSort, DiagFP, MeshSS2D, SparseGate e contrato de exportação. Ela serve como ponte entre teoria e implementação.

Cada mudança tem um papel. CanonSort estabiliza a ordem dos nós. DiagFP usa a diagonal para carregar sinais de identidade estrutural. MeshSS2D ajusta a leitura para rotas de linhagem. HierInit ajuda a inicializar o contexto respeitando camadas. SparseGate reduz a influência do vazio. O contrato de exportação garante que, depois de tudo isso, o modelo ainda possa ser chamado pelo benchmark como qualquer outro detector.

Essa aba é importante porque evita uma confusão comum: VMamba-Mesh não é apenas “VMamba aplicado em grafo”. É um conjunto de adaptações para que o grafo se torne um tensor informativo e para que o tensor continue compatível com o ecossistema do Isomera.

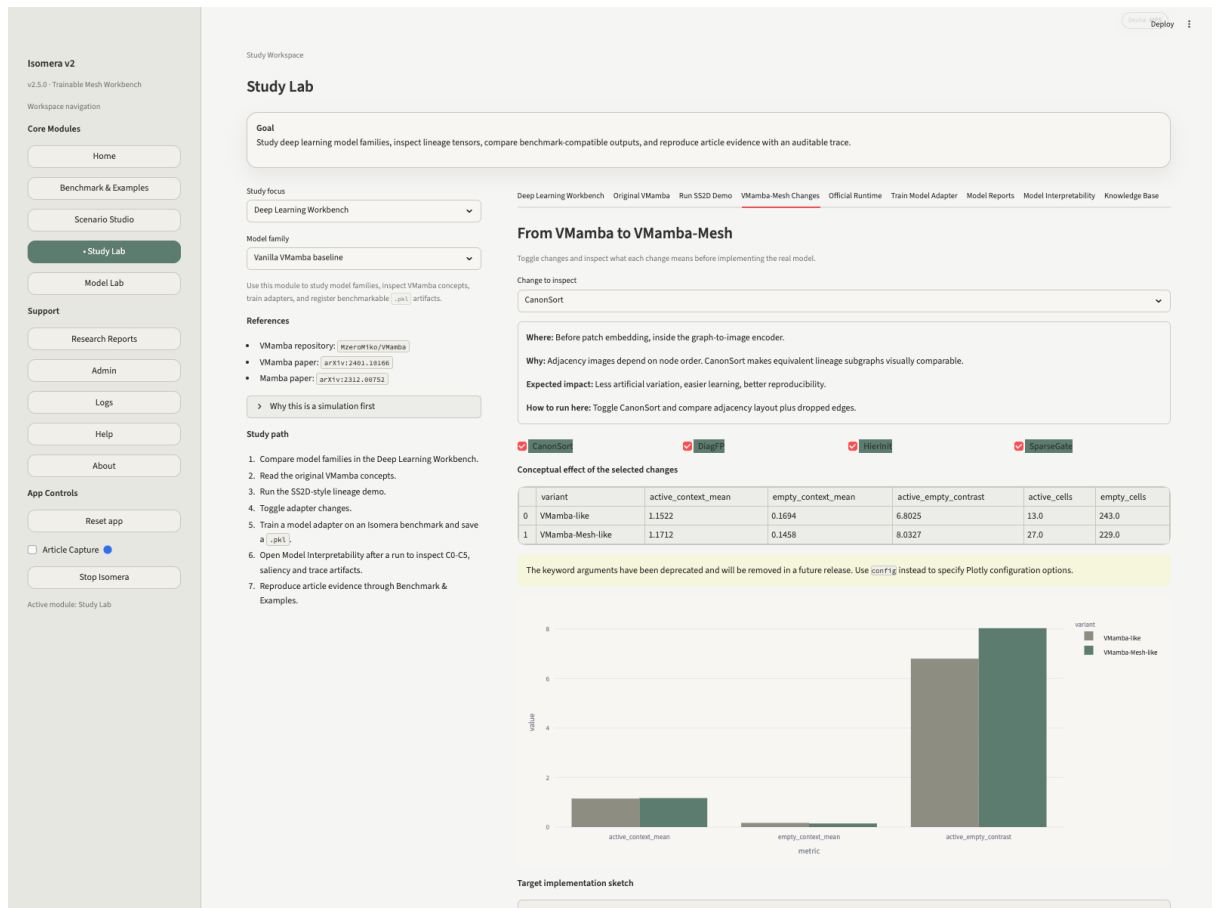


Figure 7.4: VMamba-Mesh Changes. Registro das mudanças necessárias para adaptar VMamba ao Isomera.

7.5 Official Runtime

Onde clicar: Study Lab → Official Runtime.

O runtime oficial é mantido como referência de estudo e comparação. Ele não é obrigatório para executar o benchmark do Isomera, porque o Isomera usa contratos próprios de modelo, pickles e `predict_pairs(graph)`. Mesmo assim, essa tela é importante para deixar claro de onde veio a inspiração técnica e onde consultar o código aberto original do VMamba.

Em máquinas com Linux e CUDA, o runtime oficial pode ser estudado com seus kernels e dependências próprias. No Mac, o Isomera usa PyTorch e pode resolver device como CPU ou MPS/Metal quando disponível. Essa distinção precisa ficar clara: a ferramenta implementa um caminho neural treinável real, mas não depende do kernel CUDA/Triton oficial para ser executável no ambiente local.

O valor dessa aba é separar referência de produto. A referência oficial ajuda a estudar a arquitetura original. O produto Isomera precisa ser estável para benchmark, relatório e demonstração. Por isso os dois mundos aparecem conectados, mas não misturados de forma frágil.

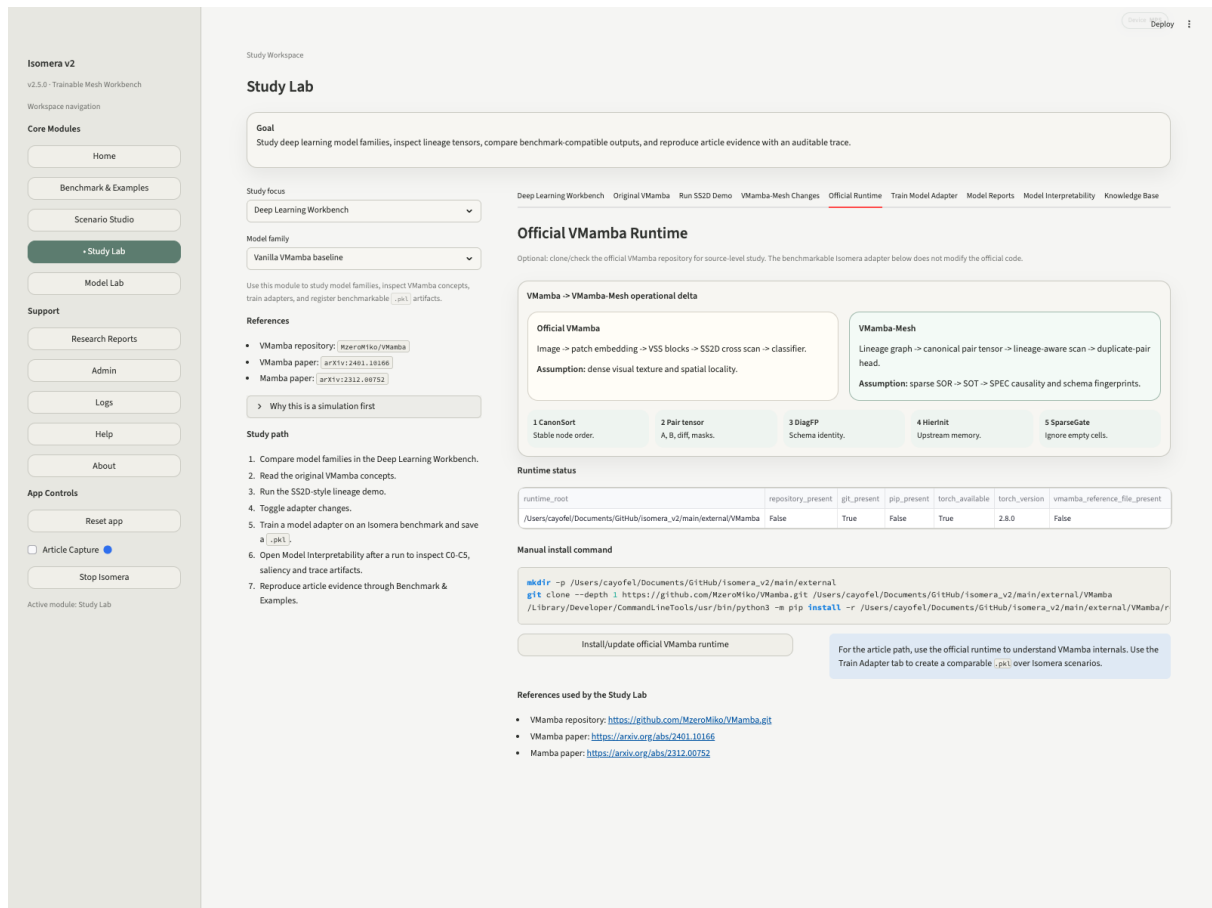


Figure 7.5: Official Runtime. Rota opcional para checar ou instalar a referência oficial do VMamba.

7.6 Train Model Adapter

Onde clicar: Study Lab → Train Model Adapter.

Essa rota permite configurar treino e registrar artefatos. O usuário deve observar hiperparâmetros, benchmark escolhido, dispositivo, número de épocas, loss, negative ratio e local de saída.

O Train Model Adapter é a tela onde a hipótese vira artefato. Antes dela, temos teoria e simulação. Depois dela, temos um modelo salvo que pode ser chamado pelo Benchmark. Essa transição exige disciplina. Um treino sem seed não é reproduzível. Um treino sem manifest não é auditável. Um treino sem informação de device não explica diferença de tempo. Um treino sem política de negativos não explica comportamento de precisão e recall.

As opções de device têm leitura prática. CPU costuma ser estável e pode ser mais rápida em cenários pequenos, porque evita overhead de despacho. MPS pode ajudar quando batches são grandes o suficiente para aproveitar o Metal. Por isso a tela expõe device, inference batch e encoder batch. O usuário consegue testar se o gargalo está na tensorização, no encoder, no head ou no overhead.

As opções de loss e balanceamento também mudam a história do modelo. Weighted BCE dá mais peso à classe minoritária. Focal loss tenta concentrar aprendizado em exem-

plos difíceis. Negative ratio controla quantos negativos entram para cada positivo. Hard negatives aproximam o treino de casos confusos, nos quais o grafo parece semelhante, mas o label é negativo.

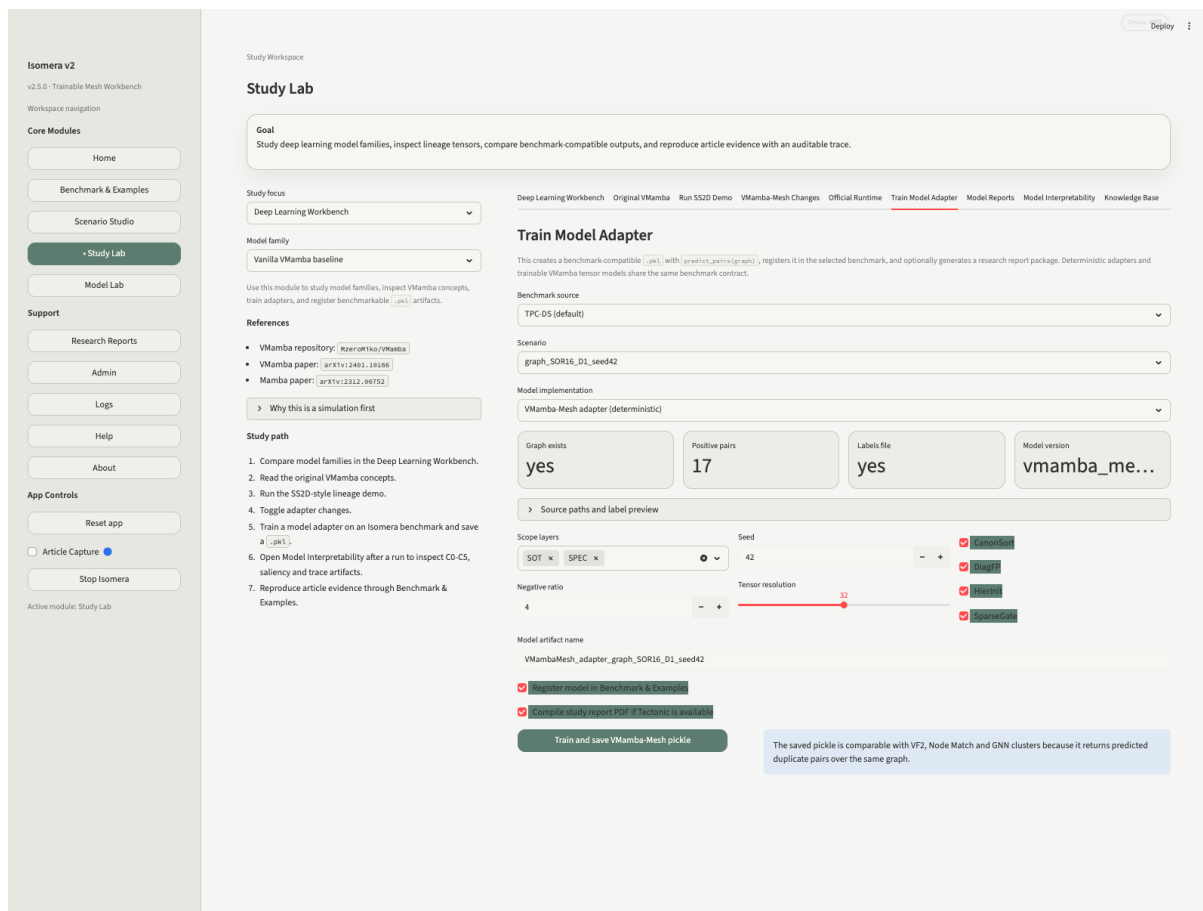


Figure 7.6: Train Model Adapter. Rota para treino e criação de artefatos compatíveis com benchmark.

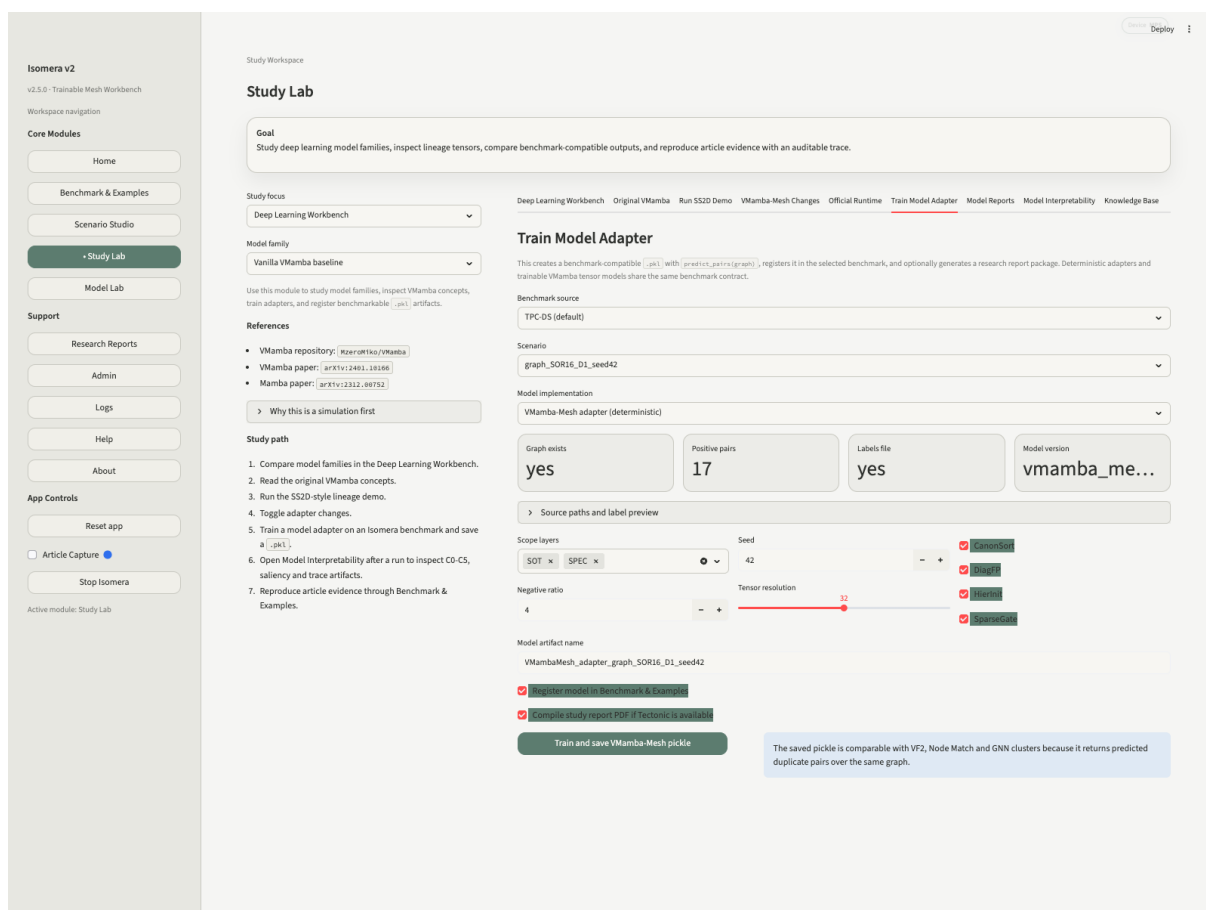


Figure 7.7: Train Model Adapter com scroll. A captura valida que a tela inclui hiperparâmetros, device, batches, loss, hard negatives e saída de artefatos.

7.7 Model Reports

Onde clicar: Study Lab → Model Reports.

Model Reports centraliza evidências de campanhas. É a rota certa para mostrar resultados sem retreinar modelos longos durante uma apresentação.

Em uso real, nem todo resultado deve ser recalculado ao vivo. Treinos podem demorar, benchmarks completos podem ser pesados e relatórios precisam preservar o estado da campanha. Model Reports resolve isso: ele permite abrir o que já foi executado, conferir tabelas, gráficos, manifests e decisões sem depender de uma nova execução.

Essa rota também ajuda a separar treino de evidência. Treinar é produzir o artefato. Rodar benchmark é medir o artefato. Abrir relatório é explicar o que aconteceu. Quando essas etapas ficam misturadas, a apresentação perde clareza. Quando ficam separadas, o usuário consegue mostrar a ferramenta com segurança.

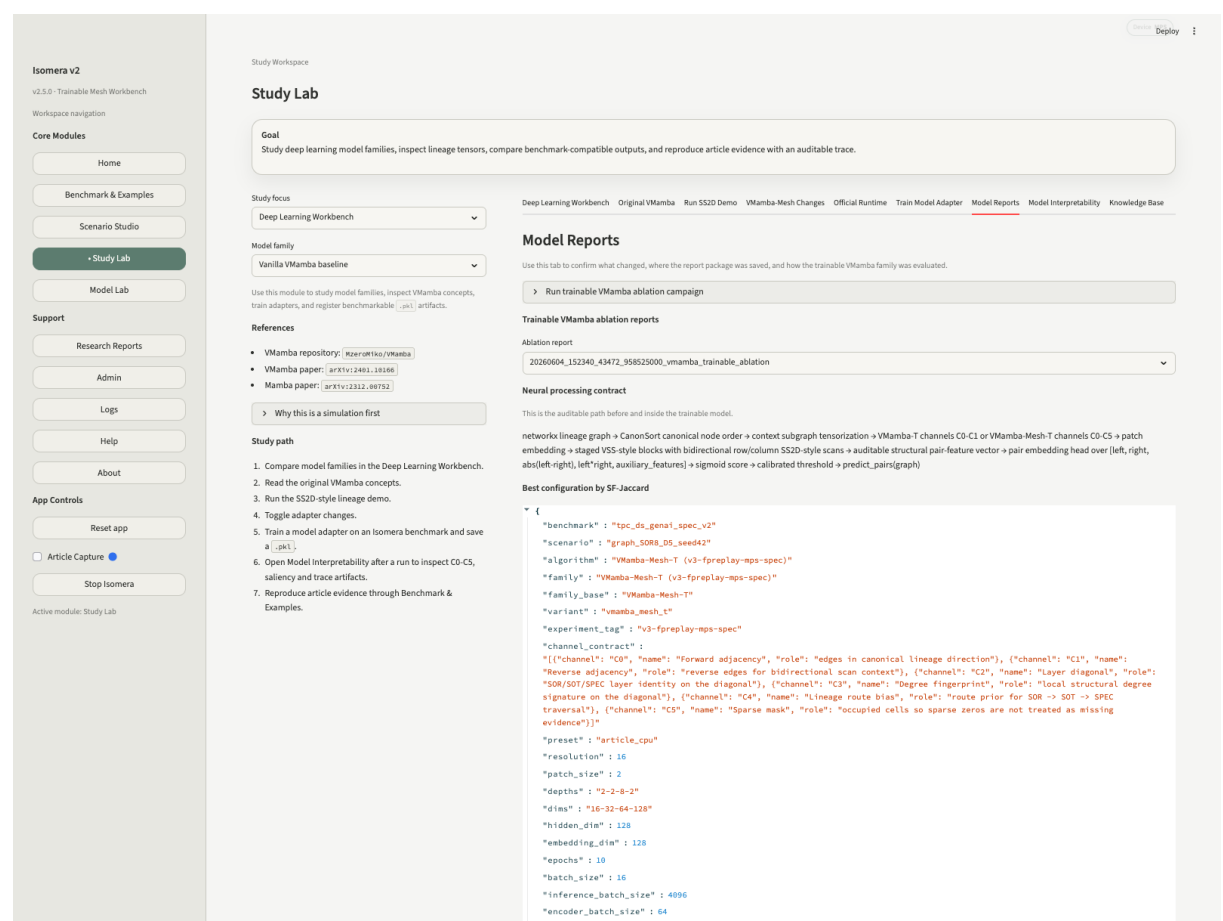
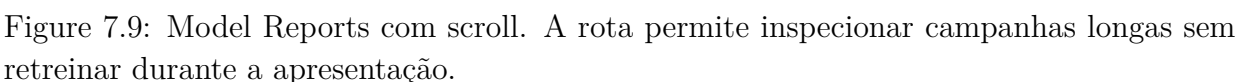


Figure 7.8: Model Reports. Relatórios e campanhas salvas.



Onde clicar: Study Lab → Model Interpretability.

No Isomera, interpretabilidade é local. Isso significa que ela explica um par específico, em um cenário específico, usando um artefato específico. A saliency não prova causalidade universal. Ela mostra sensibilidade local: quais regiões do tensor influenciaram o score daquele par naquela execução. Essa distinção é importante para evitar exagero na leitura.

Dica! Explique saliency como sensibilidade local, não como prova causal. Ela ajuda a revisar uma decisão específica de par, modelo e cenário.

41

Por trás do clique: A saliency é calculada a partir do gradiente do score em relação ao tensor de entrada. Em termos práticos, ela pergunta: se eu alterasse levemente esta célula/canal, quanto o score mudaria? Cores mais fortes indicam maior sensibilidade local.

Como saliency baseada em gradiente depende de $\partial f_{\theta}(X)/\partial X$, ela mede sensibilidade local ao redor da entrada analisada. Essa leitura tem limites: gradientes podem ser pequenos por saturação da sigmoid, regiões fortes podem refletir escala dos canais, e mapas semelhantes não equivalem a explicação causal. Por isso, saliency deve ser usada junto com grafo, matriz, canais, score, threshold e análise de erros.

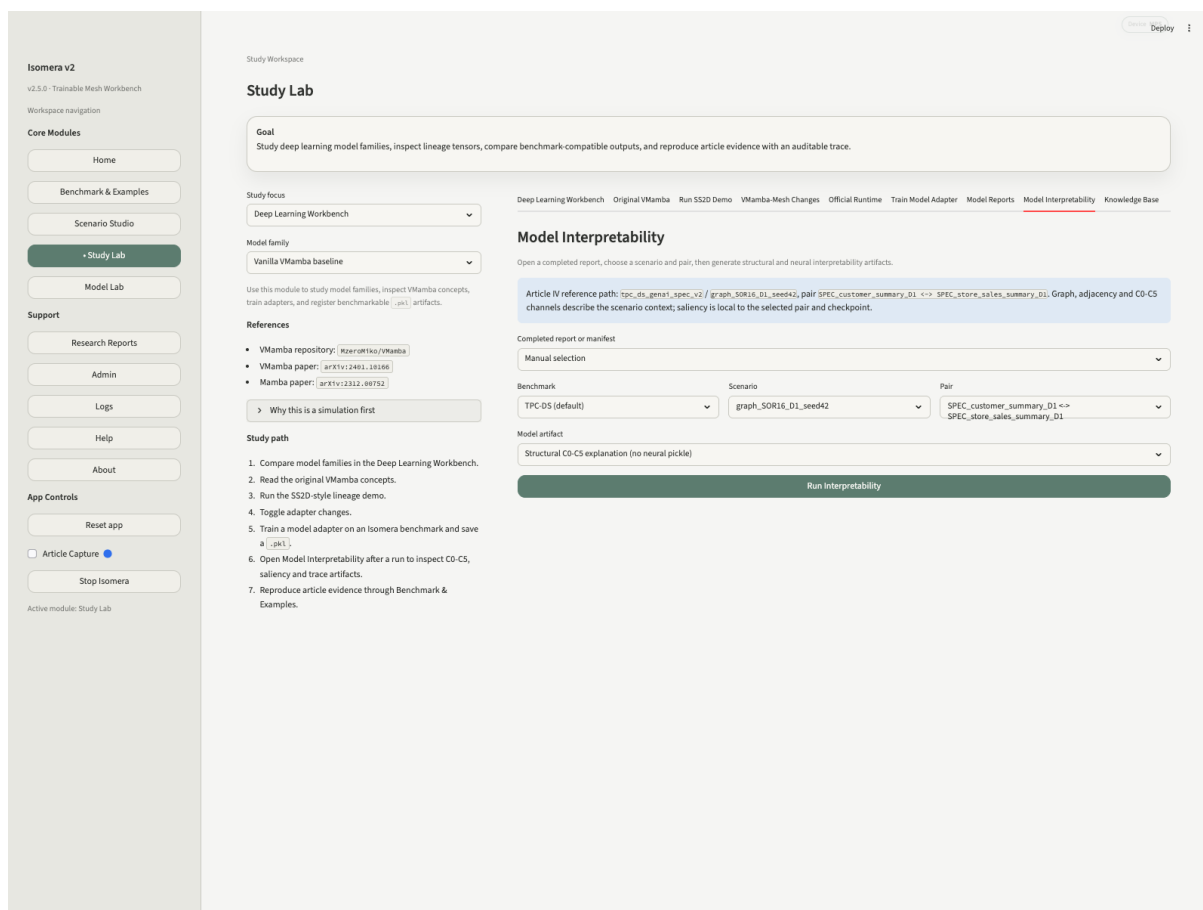


Figure 7.10: Model Interpretability. Caminho para inspecionar decisão de par com saliency e trace.

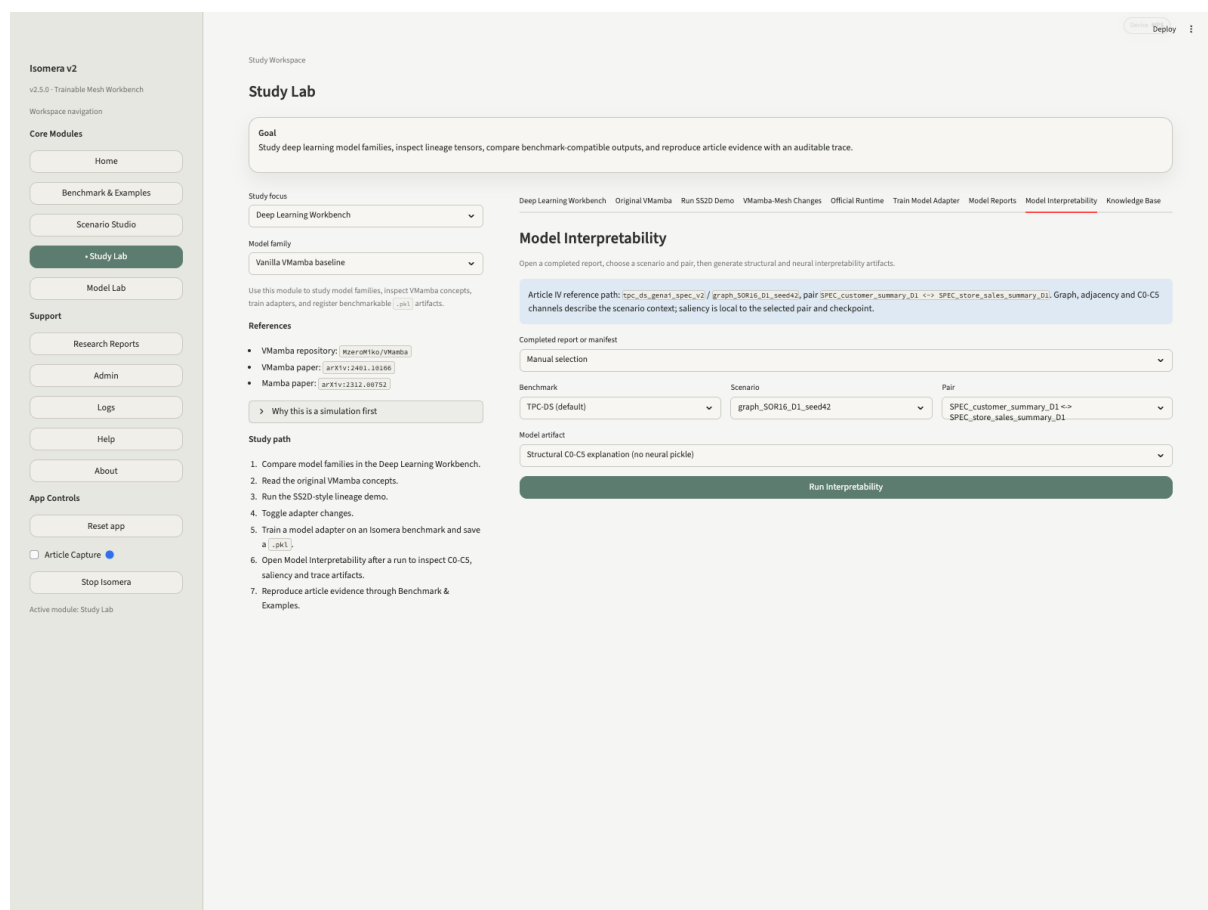


Figure 7.11: Model Interpretability com scroll. A captura confirma os controles de benchmark, cenário, par, modelo e artefatos de interpretabilidade.

7.9 Knowledge Base

Onde clicar: Study Lab → Knowledge Base.

A base de conhecimento resume VMamba, VMamba-Mesh, encoding, reprodutibilidade e interpretabilidade. Ela serve para manter a teoria dentro do aplicativo, não apenas nos PDFs.

Essa tela evita que o conhecimento fique espalhado. Quando o usuário esquece o papel de um canal, abre a Knowledge Base. Quando precisa lembrar como a reprodução foi organizada, abre a Knowledge Base. Quando quer revisar Mamba, VMamba, SS2D ou interpretabilidade, abre a Knowledge Base. A documentação fica perto da ação.

Em produto de pesquisa, essa proximidade importa. Se a explicação está apenas em um PDF externo, a ferramenta vira caixa-preta. Se a explicação está dentro do app, o usuário consegue alternar entre teoria e prática sem sair do fluxo.

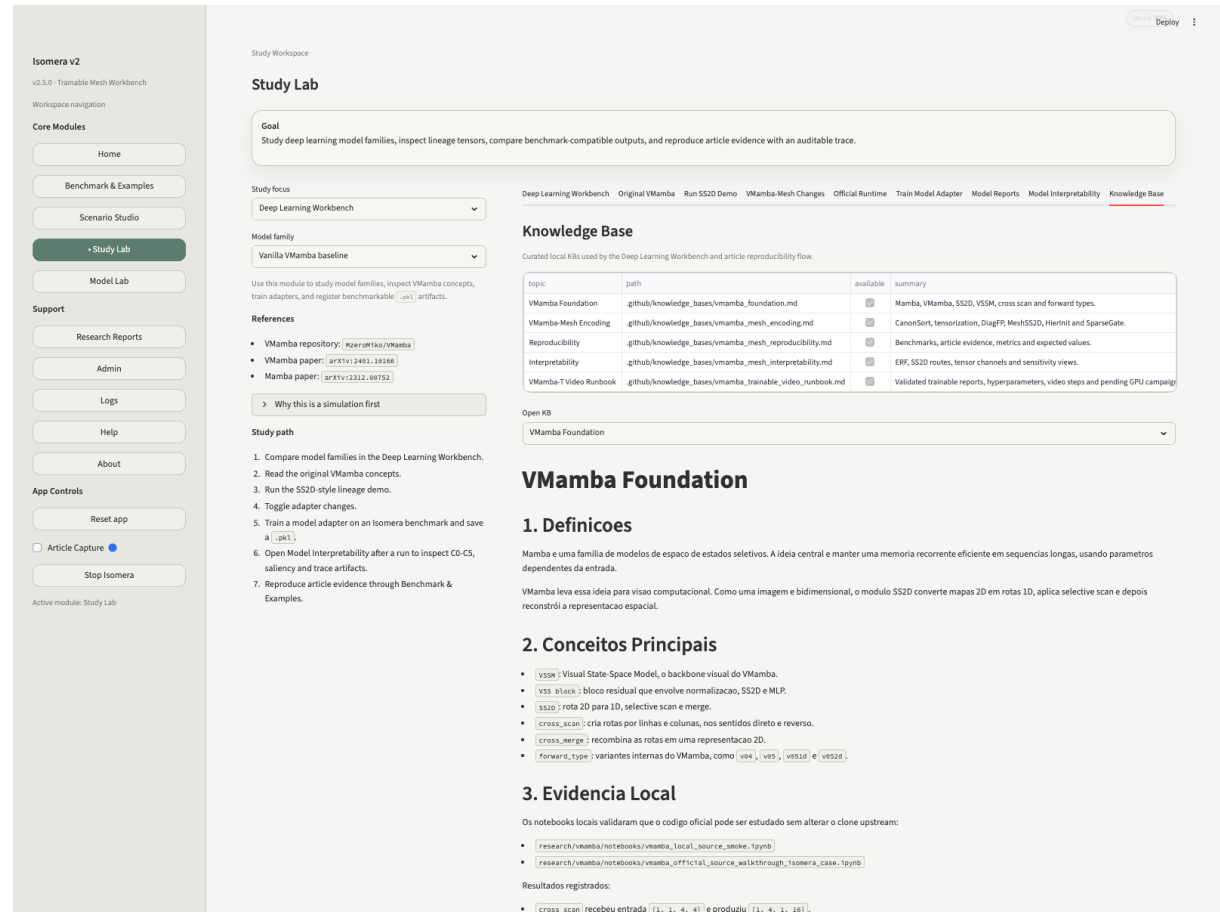


Figure 7.12: Knowledge Base. Material teórico embutido no Study Lab.

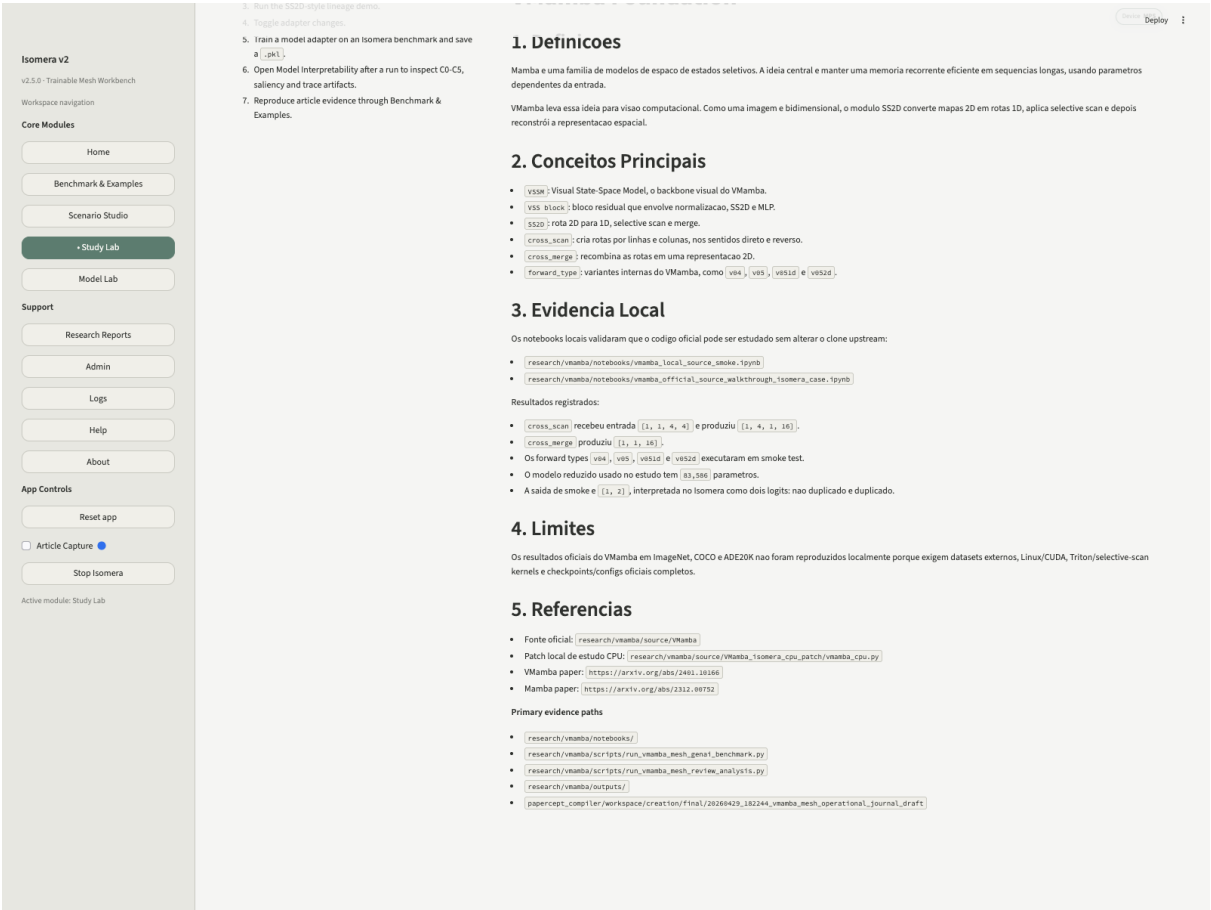


Figure 7.13: Knowledge Base com scroll. As KBs ficam navegáveis dentro do aplicativo para consulta durante estudo ou banca.

Chapter 8

Model Lab

Onde clicar: Menu lateral → Model Lab.

O Model Lab mostra os detectores registrados e os pickles disponíveis. Em um ambiente corporativo, essa é uma das rotas mais importantes: ela mostra quais artefatos podem ser reutilizados, quais cenários estão cobertos e como o benchmark roteia cada modelo.

O Model Lab é o inventário técnico dos modelos. Em uma ferramenta pequena, talvez bastasse uma pasta com arquivos. Em uma ferramenta de governança, isso não é suficiente. O usuário precisa saber quais modelos existem, quais famílias eles representam, quais benchmarks cobrem, qual política de roteamento está ativa e se o pickle carrega corretamente.

Essa rota é especialmente importante quando existem modelos especializados por cenário. Um GNN pode ter sido treinado para uma família de grafos. Um VMamba-Mesh-T pode ter sido treinado com canais C0–C5 e threshold específico. Um modelo pode estar mapeado para 20 cenários, outro para apenas um. Se o benchmark rodar sem esse controle, o resultado fica difícil de interpretar.

Dica! Quando um resultado parecer estranho, confira o Model Lab antes de culpar o modelo. O problema pode estar no roteamento, no pickle ou na cobertura do cenário.

Por trás do clique: O roteamento liga benchmark, cenário e artefato. Na execução, o Isomera pergunta qual modelo deve ser usado para aquele cenário. Se houver rota explícita, usa o pickle mapeado. Se houver política best-of, seleciona conforme configuração. Se não houver cobertura, o status precisa indicar que o modelo não está disponível para aquele caso.

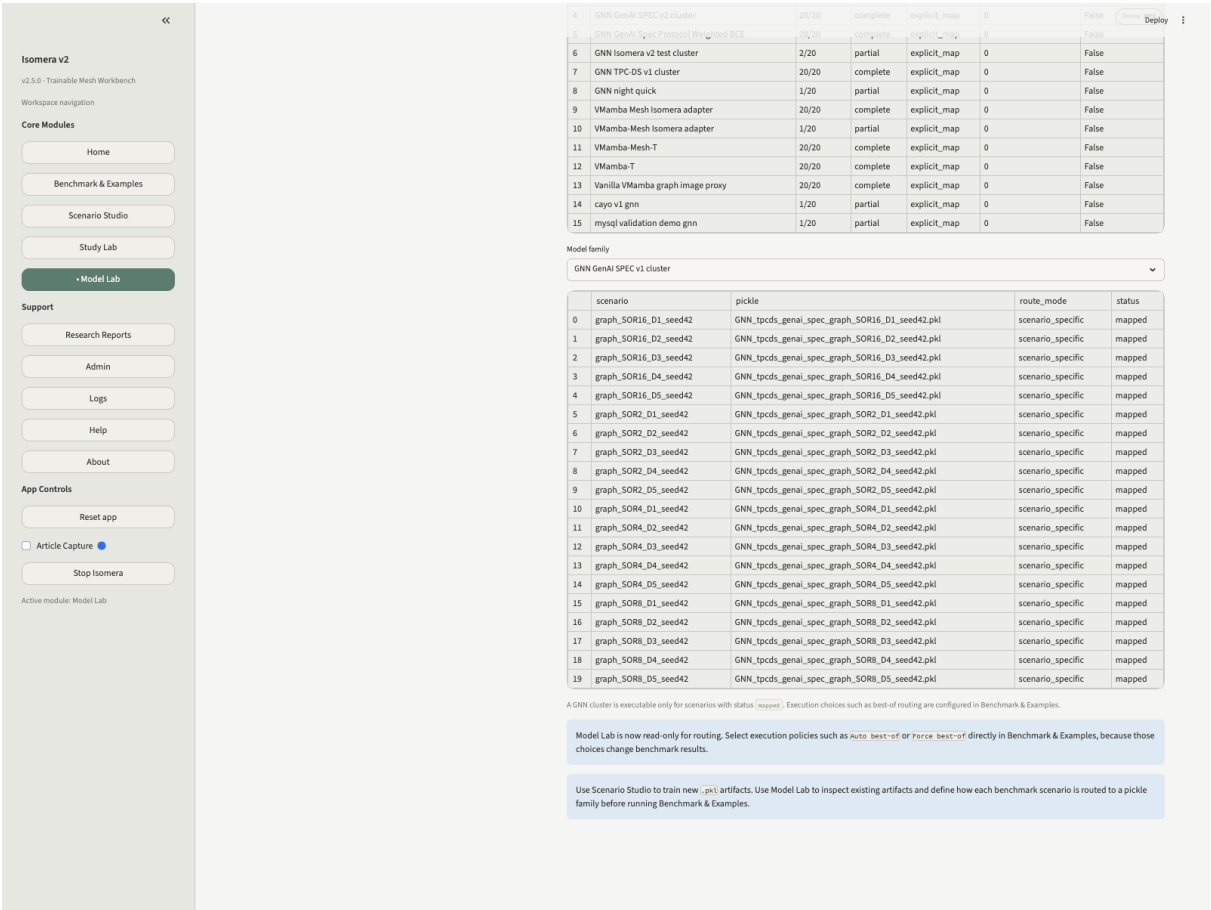


Figure 8.1: Model Lab. Registro e validação de pickles e famílias de modelo.

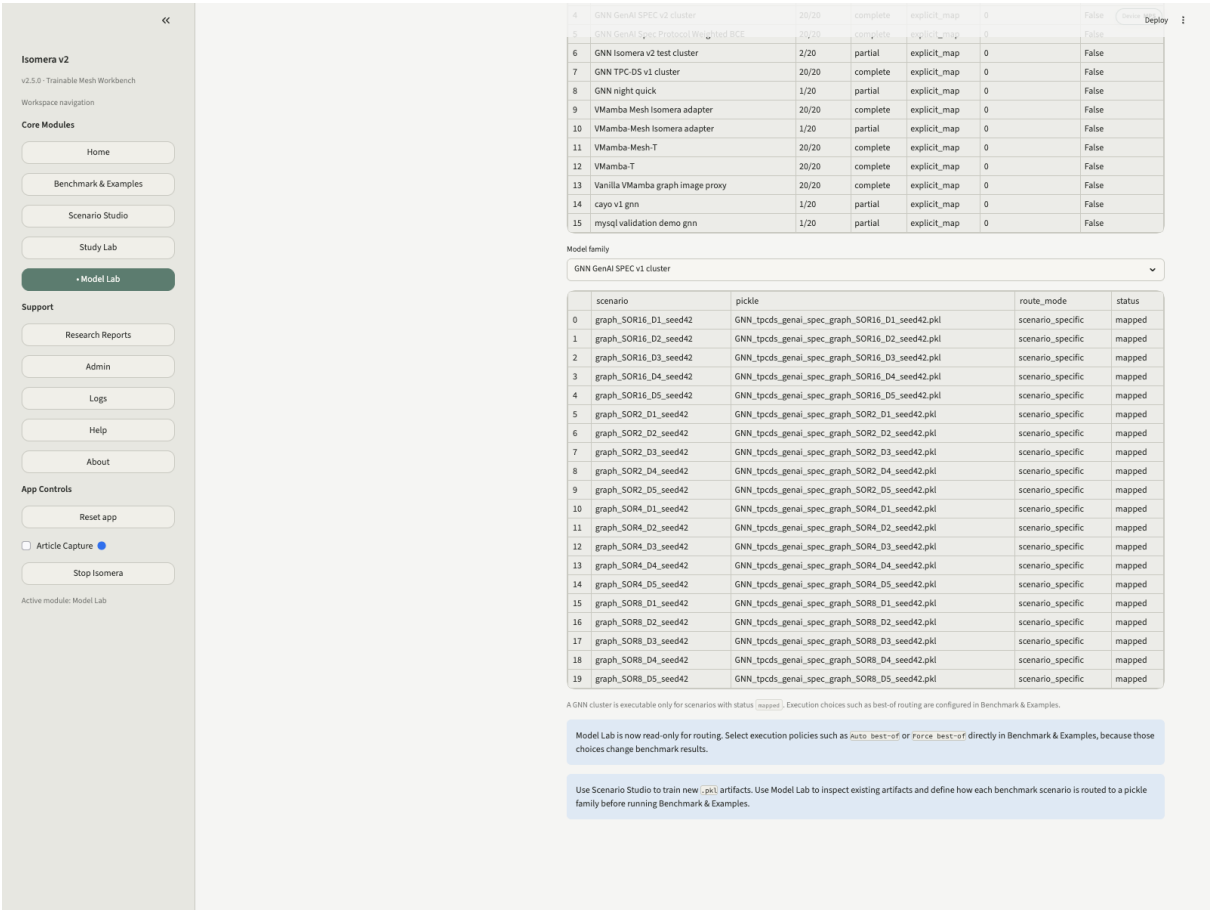


Figure 8.2: Model Lab com scroll. A imagem mostra que o usuário pode verificar roteamento e cobertura por benchmark.

Chapter 9

Research Reports

Onde clicar: Menu lateral → Research Reports.

Research Reports é o arquivo vivo do Isomera. Cada execução relevante pode gerar Markdown, CSV, JSON, figuras, manifest e pacote ZIP. O usuário não precisa procurar manualmente em pastas: a interface lista os pacotes e mostra a narrativa.

Essa tela é o equivalente a uma bancada de evidências. Ela existe porque resultado científico e resultado de engenharia precisam de rastro. Uma execução sem relatório é difícil de defender. Um relatório sem manifest é difícil de reproduzir. Um gráfico sem CSV é difícil de auditar. Um CSV sem contexto é difícil de explicar.

Quando o usuário abre um pacote em Research Reports, ele deve procurar quatro coisas. Primeiro, o resumo narrativo: o que foi executado e por quê. Segundo, os arquivos tabulares: métricas, comparações, ablations e cenários. Terceiro, as figuras: gráficos, matrizes, canais e saliency. Quarto, o manifest: parâmetros, seeds, device, modelos e caminhos de artefatos.

Dica! Um bom relatório deve responder sem abrir o código: o que rodou, com quais parâmetros, quais arquivos gerou e quais evidências sustentam o resultado.

Essa organização também ajuda na rotina de vídeo ou banca. Em vez de retrainar, o apresentador mostra a campanha salva, abre as tabelas, mostra os gráficos e depois volta ao Study Lab para explicar um par específico. Assim a demonstração fica dinâmica sem sacrificar reprodutibilidade.

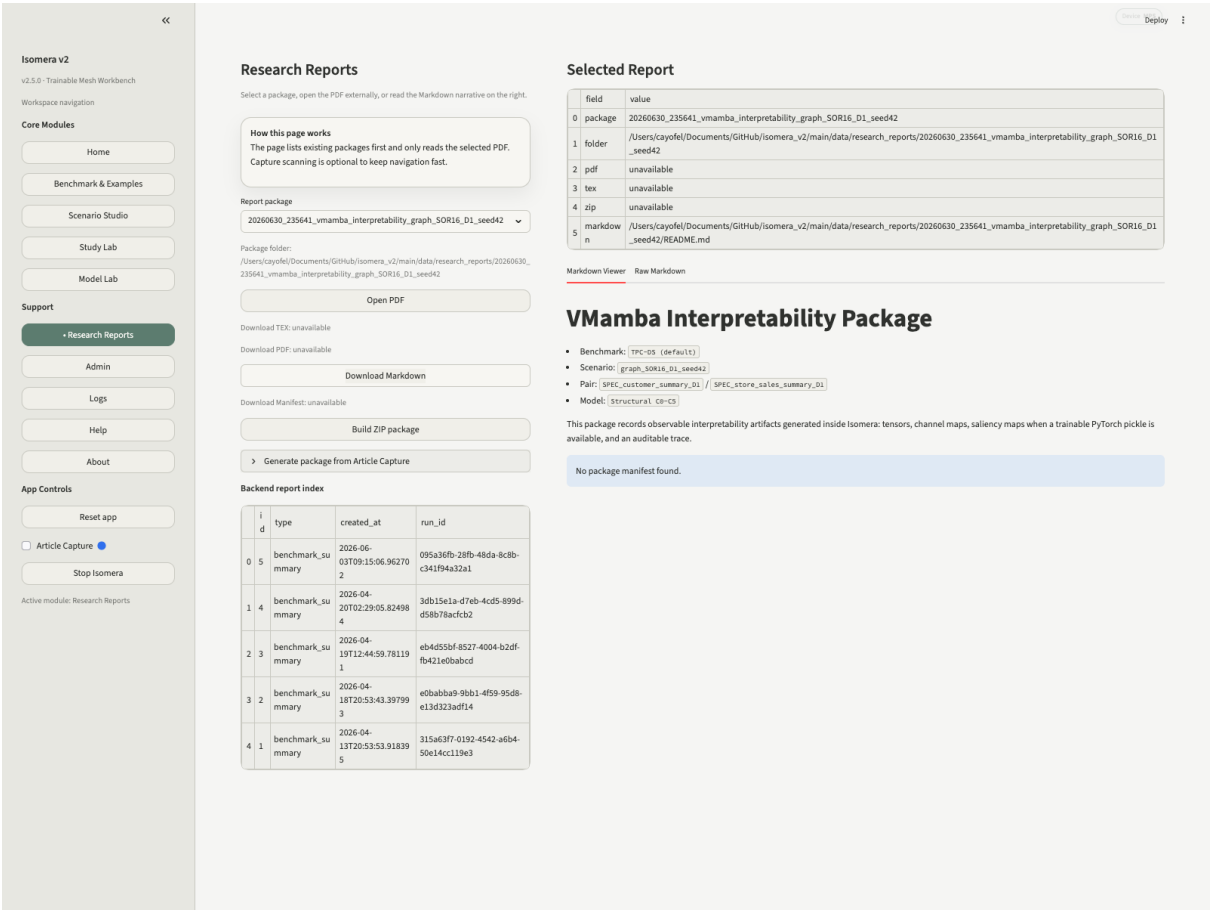


Figure 9.1: Research Reports. Leitura de pacotes gerados por execuções e estudos.

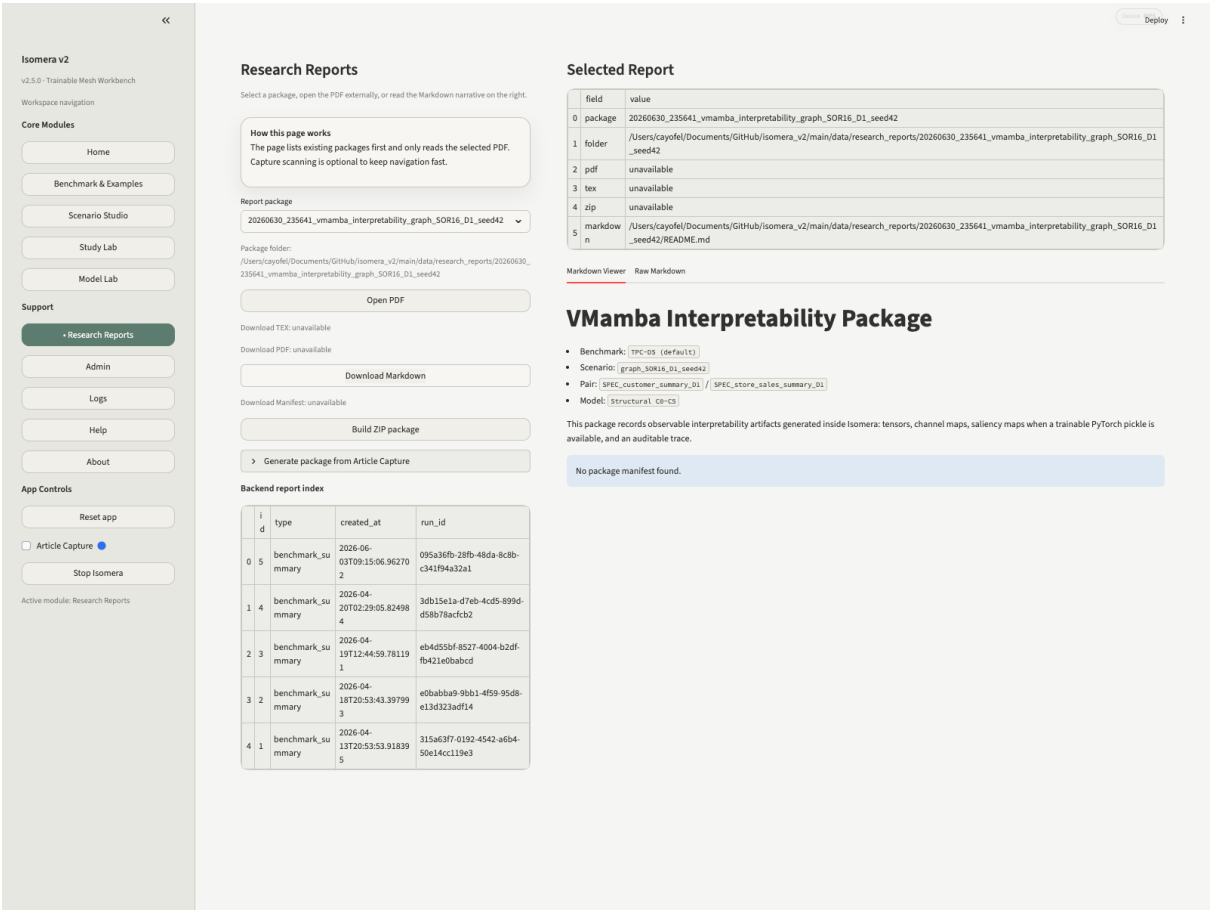


Figure 9.2: Research Reports com scroll. Use esta área para abrir detalhes, anexos e relatórios longos.

Chapter 10

Admin

Onde clicar: Menu lateral → Admin.

Admin mostra o estado técnico da ferramenta. Essa rota é útil para depuração, demonstração de maturidade e entendimento de onde ficam backend, warehouse e configurações.

O Admin não é a parte mais chamativa da ferramenta, mas é uma das mais importantes para operação. Ele responde a perguntas que normalmente só apareceriam no terminal: qual backend está ativo, quantos cenários estão registrados, se o warehouse está acessível, se há integração graph-native prevista e quais URLs/configurações estão em uso.

Em ambientes de pesquisa, Admin ajuda a detectar diferença entre máquinas. Em ambientes corporativos, ajuda a separar store de produto, warehouse de cenários e configurações. Esse isolamento reduz risco de confundir dados de benchmark com dados operacionais.

Dica! Use Admin como verificação de ambiente. Ele ajuda a separar erro de dados, erro de configuração e erro de execução.

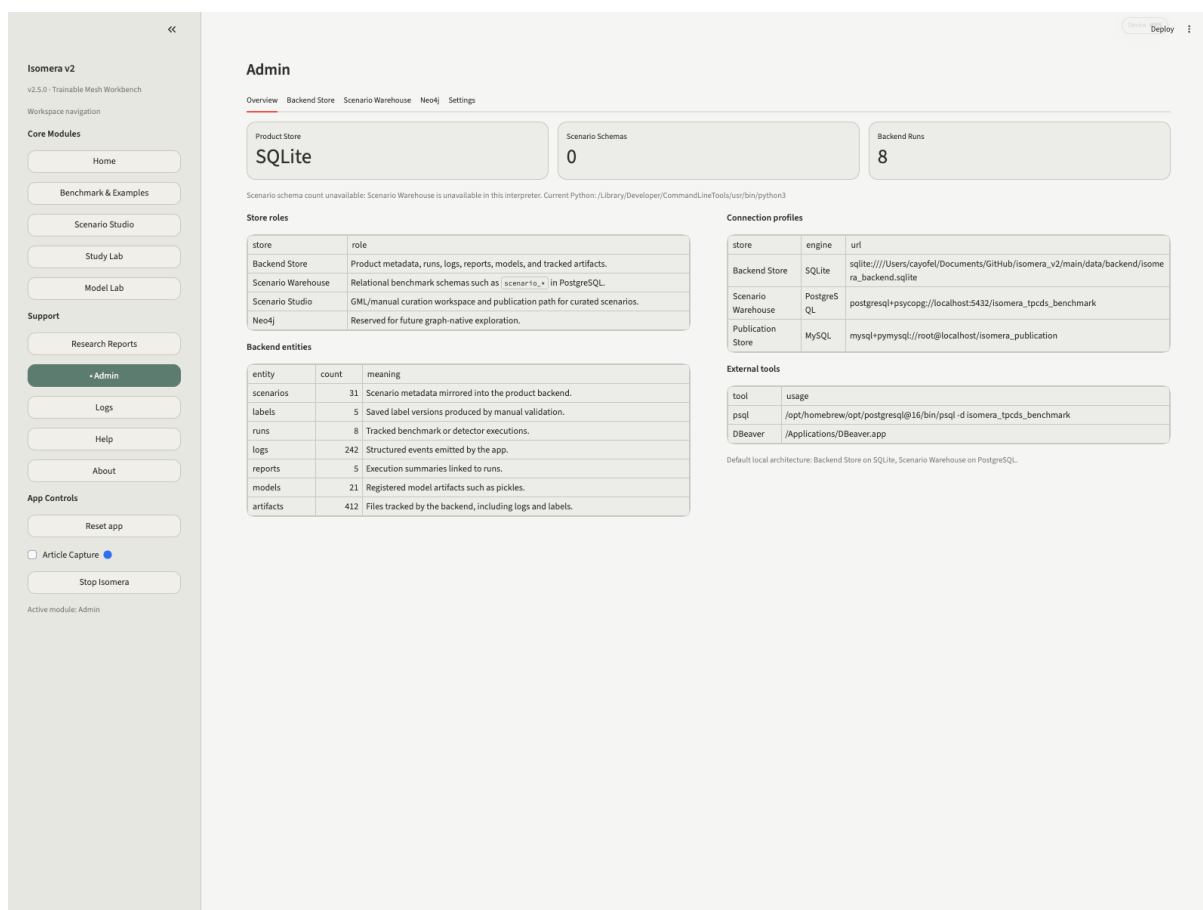


Figure 10.1: Admin Overview. Visão geral de stores e entidades.

10.1 Backend Store

Onde clicar: Admin → Backend Store.

O Backend Store guarda metadados de produto: cenários, labels, runs, logs, reports, modelos e artefatos.

Pense no Backend Store como o caderno de bordo do aplicativo. Ele não é necessariamente onde vivem todos os grafos ou todos os dados brutos. Ele guarda o que o produto precisa para se lembrar do que aconteceu: quais runs existiram, quais relatórios foram gerados, quais modelos foram registrados e quais artefatos foram rastreados.

Na prática: Se algo parece ter rodado, mas não aparece em relatório ou benchmark, verifique o Backend Store. Ele ajuda a diferenciar problema de execução, problema de registro e problema de visualização.

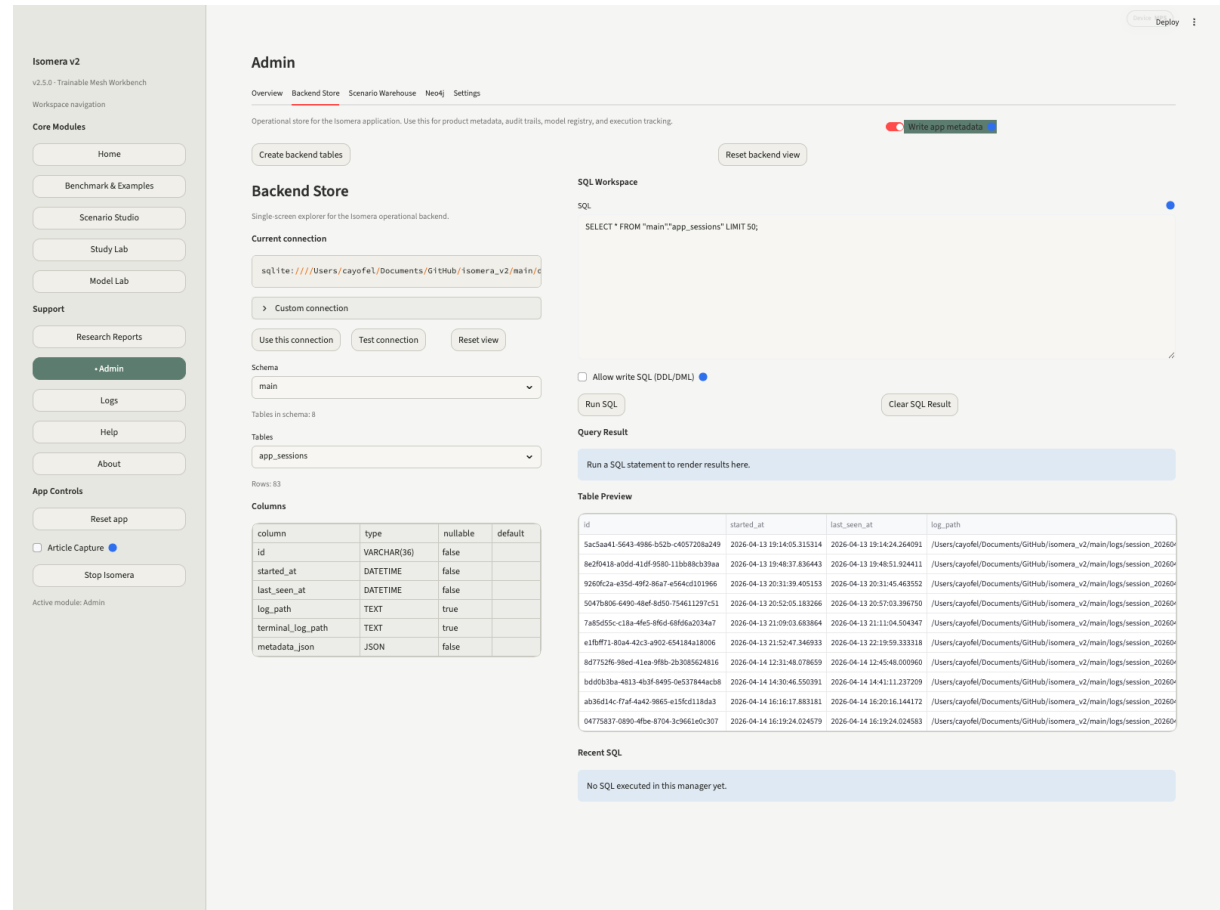


Figure 10.2: Backend Store. Tabelas e registros operacionais.

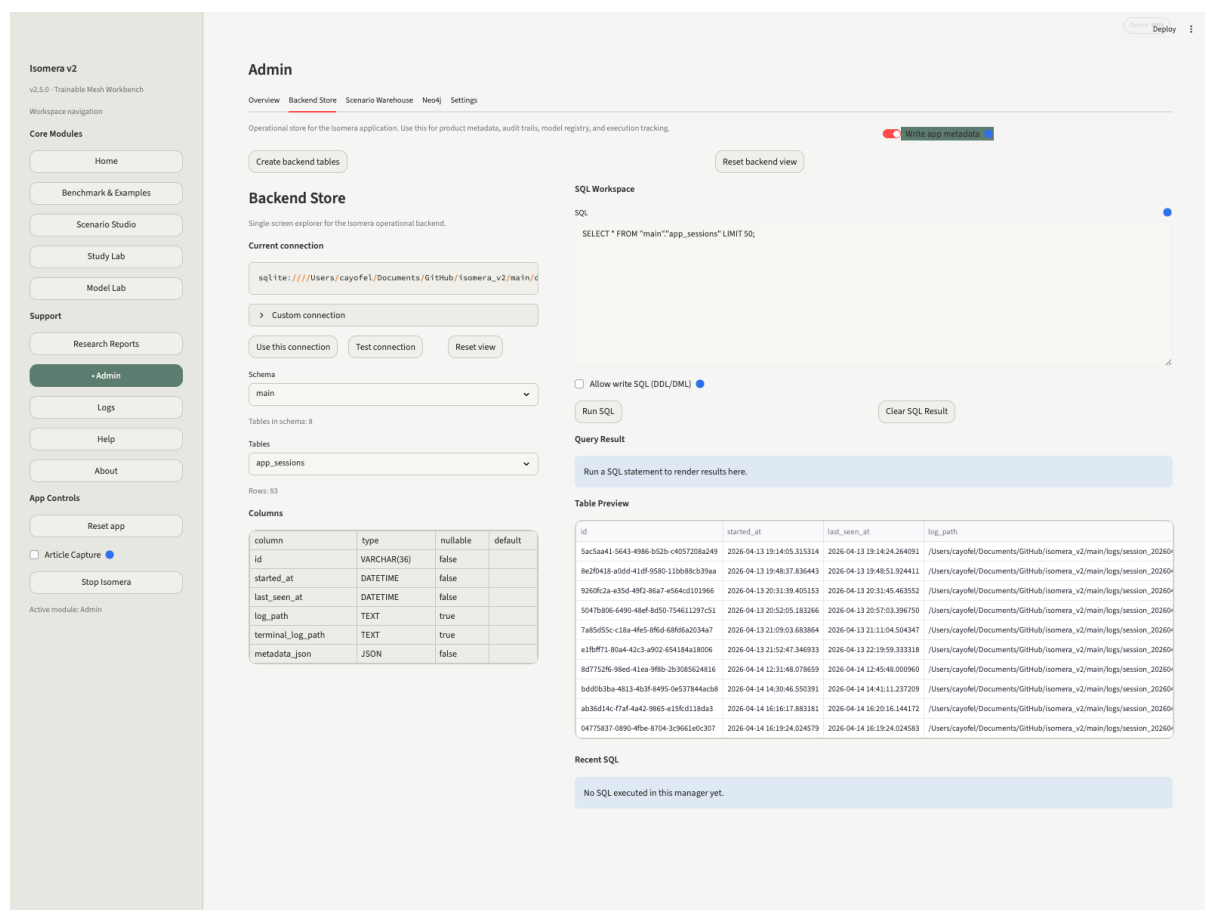


Figure 10.3: Backend Store com scroll. A captura valida entidades, contadores e tabelas abaixo da área inicial.

10.2 Scenario Warehouse

Onde clicar: Admin → Scenario Warehouse.

O Scenario Warehouse mostra schemas relacionais de cenários. Essa rota conecta a ferramenta a bancos e materializações SQL.

O Scenario Warehouse representa a ponte com o mundo relacional. A linhagem pode nascer em um banco, em schemas, em tabelas e em contratos. Ao mostrar esses schemas, o Isomera deixa claro que o grafo não precisa ser desenhado manualmente. Ele pode ser materializado a partir de estruturas reais.

Essa tela também ajuda a explicar como o benchmark pode ser construído a partir de dados persistidos. Cada cenário pode ser representado de forma isolada, com suas tabelas, metadados e relações. Depois o Scenario Studio transforma isso em grafo e o Benchmark usa esse grafo para avaliar detectores.

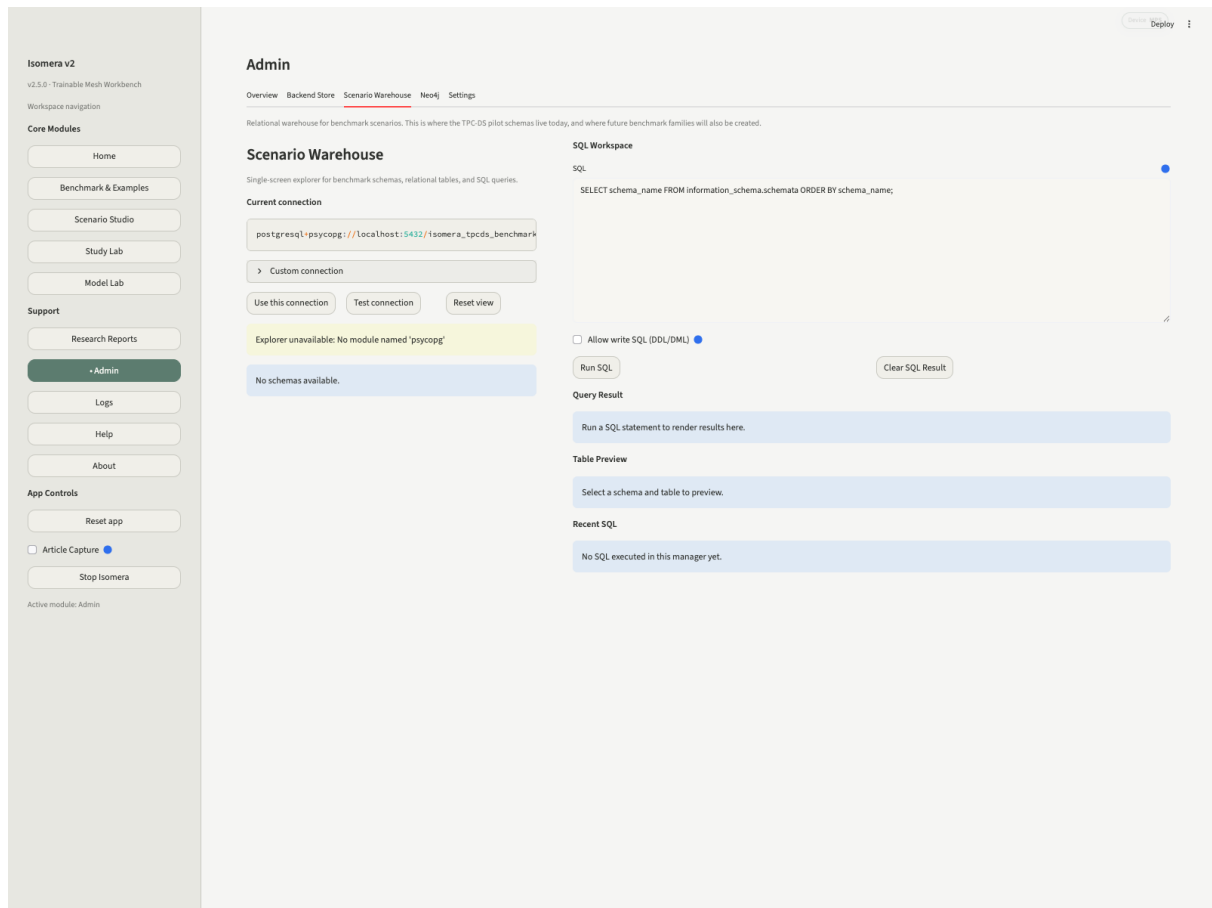


Figure 10.4: Scenario Warehouse. Estado do warehouse de cenários.

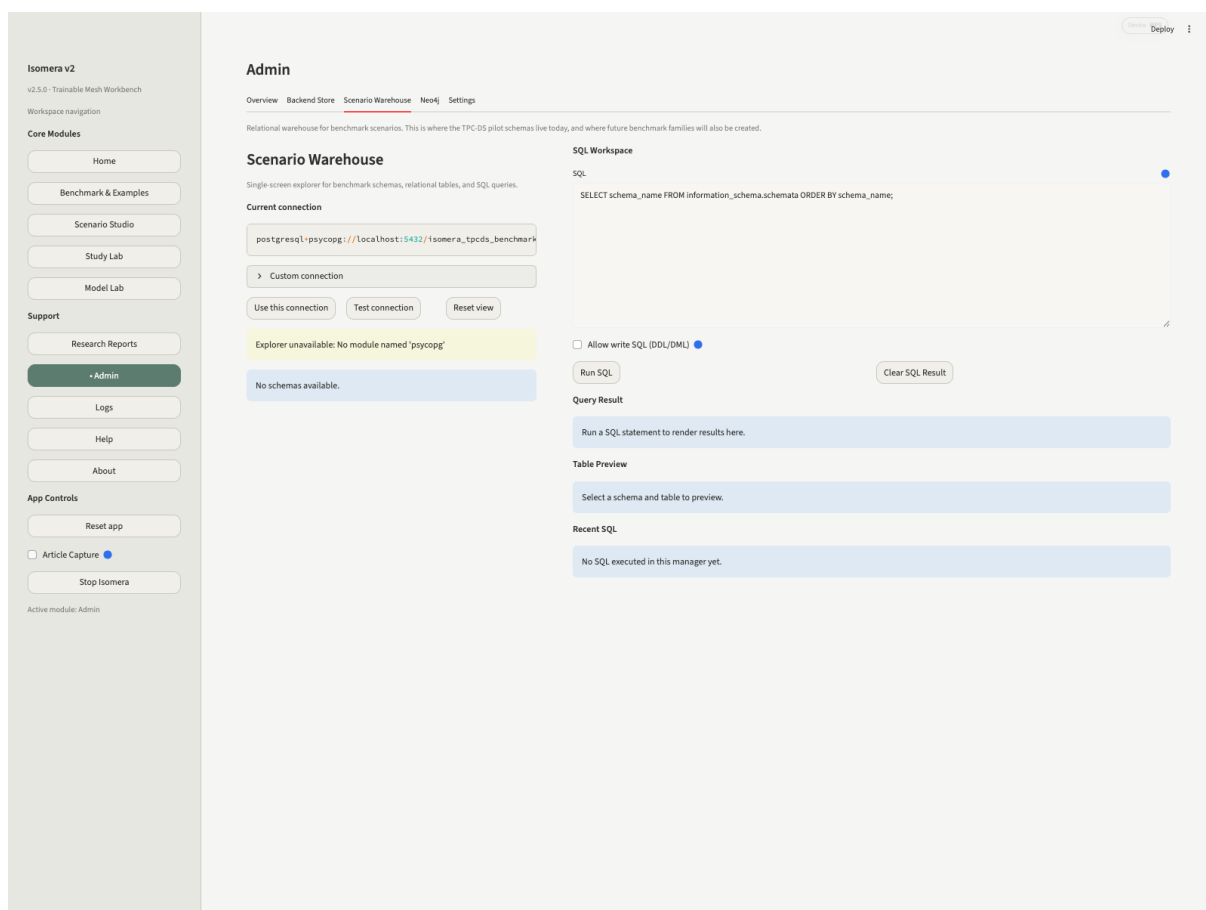


Figure 10.5: Scenario Warehouse com scroll. A tela apoia auditoria de schemas e materialização relacional.

10.3 Neo4j e Settings

Onde clicar: Admin → Neo4j ou Settings.

Neo4j aparece como espaço reservado para exploração graph-native. Settings centraliza conexões e perfis de backend.

Neo4j aparece como uma direção natural para evolução. Como o domínio é grafo, uma base graph-native pode facilitar consultas exploratórias, vizinhanças, caminhos e visualizações. Mesmo que a execução atual não dependa dela, a tela indica que a arquitetura foi pensada para crescer.

Settings é a parte onde a ferramenta deixa de ser apenas local e passa a aceitar perfis. URLs, conexões e stores precisam ser configuráveis porque o mesmo Isomera pode rodar em máquina de pesquisa, ambiente de demonstração ou servidor de laboratório.

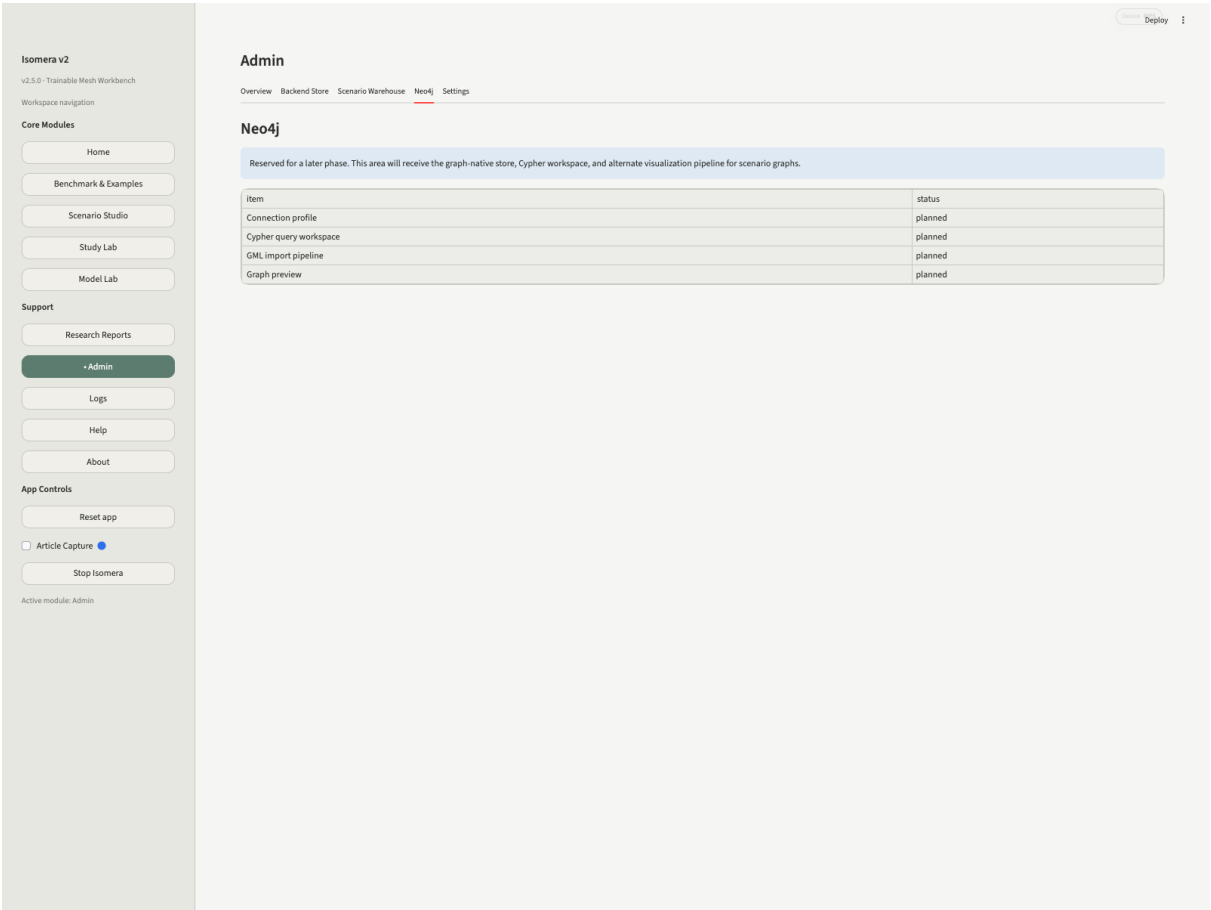


Figure 10.6: Neo4j. Espaço de integração graph-native.

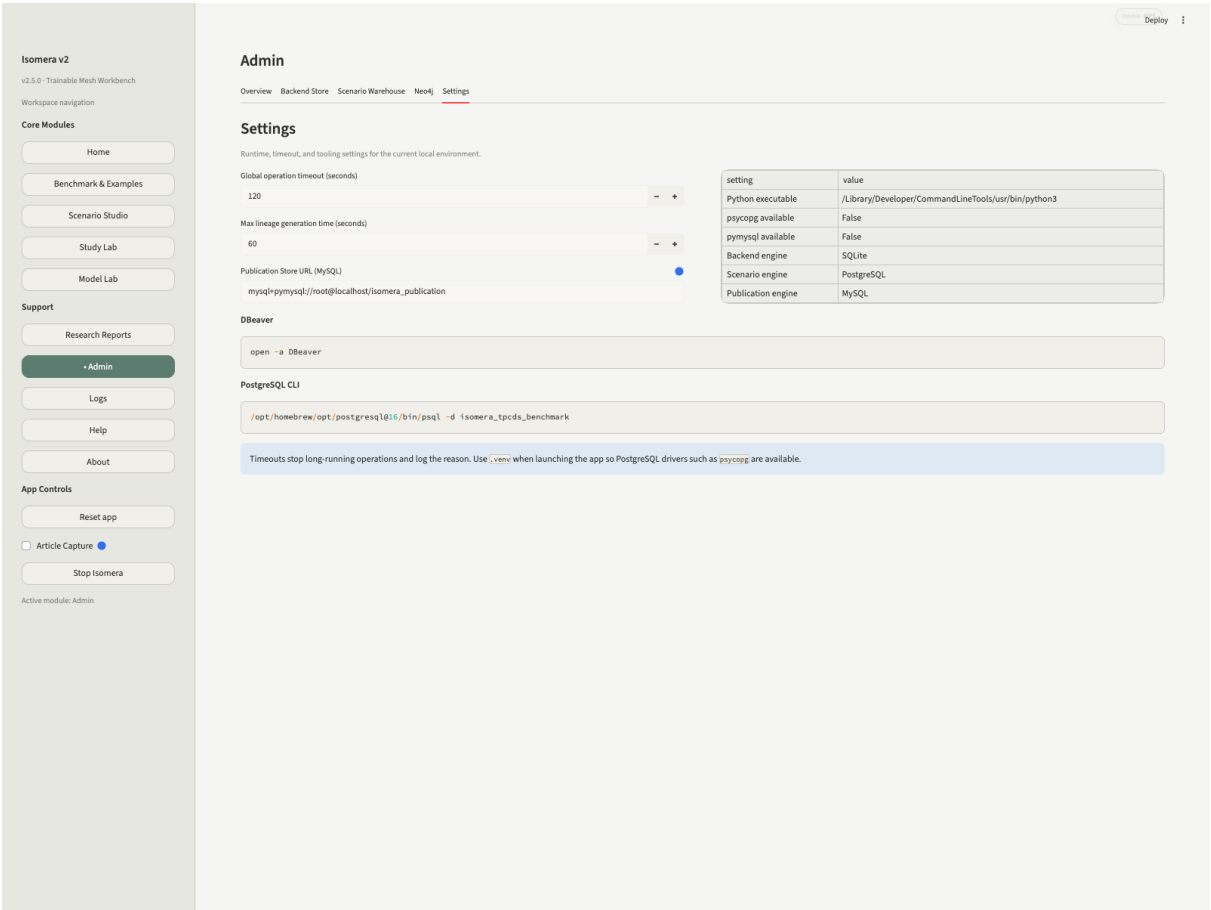


Figure 10.7: Settings. Configurações de conexão e runtime.

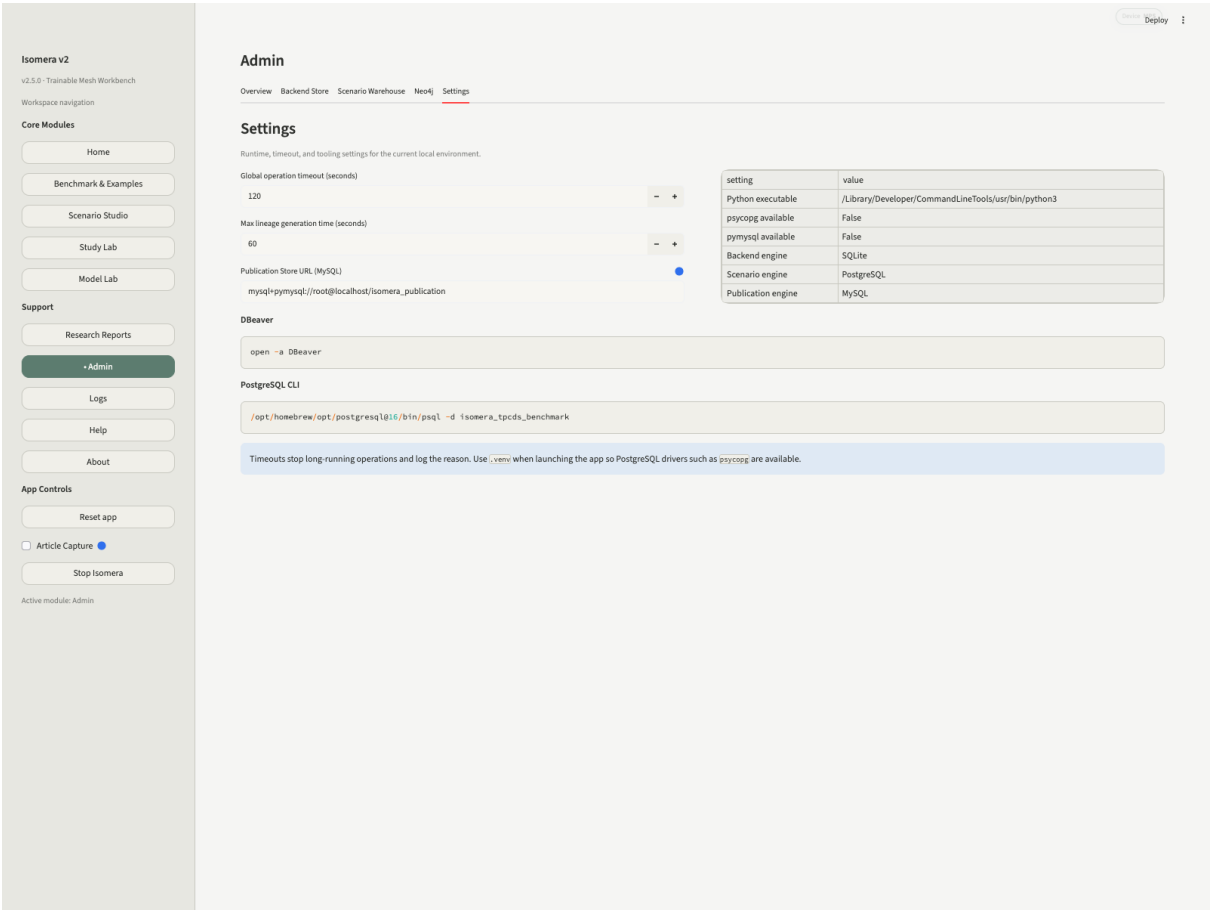


Figure 10.8: Settings com scroll. A captura mostra configurações e perfis adicionais de runtime.

Chapter 11

Logs

Onde clicar: Menu lateral → Logs.

Logs ajudam a explicar o que aconteceu sem abrir o terminal. Eles são relevantes para depuração e também para auditoria de execução.

Logs são a memória de baixo nível da ferramenta. Se um botão foi clicado, se uma rota foi aberta, se uma execução falhou ou se um arquivo foi lido, a sessão deve registrar o suficiente para investigação. Isso é importante porque ferramentas interativas podem esconder erro visualmente: um botão pode parecer não fazer nada quando, na verdade, houve falha de dependência, caminho ou permissão.

Dica! Comece pelo Session Log para entender a ação do usuário. Só depois vá ao Terminal Logs para investigar erro bruto ou dependência ausente.

O Session Log tem foco em eventos estruturados do app. O Terminal Logs mostra saídas mais próximas do processo. Na prática, o usuário começa pelo Session Log para entender a ação e desce para Terminal Logs quando precisa ver erro bruto.

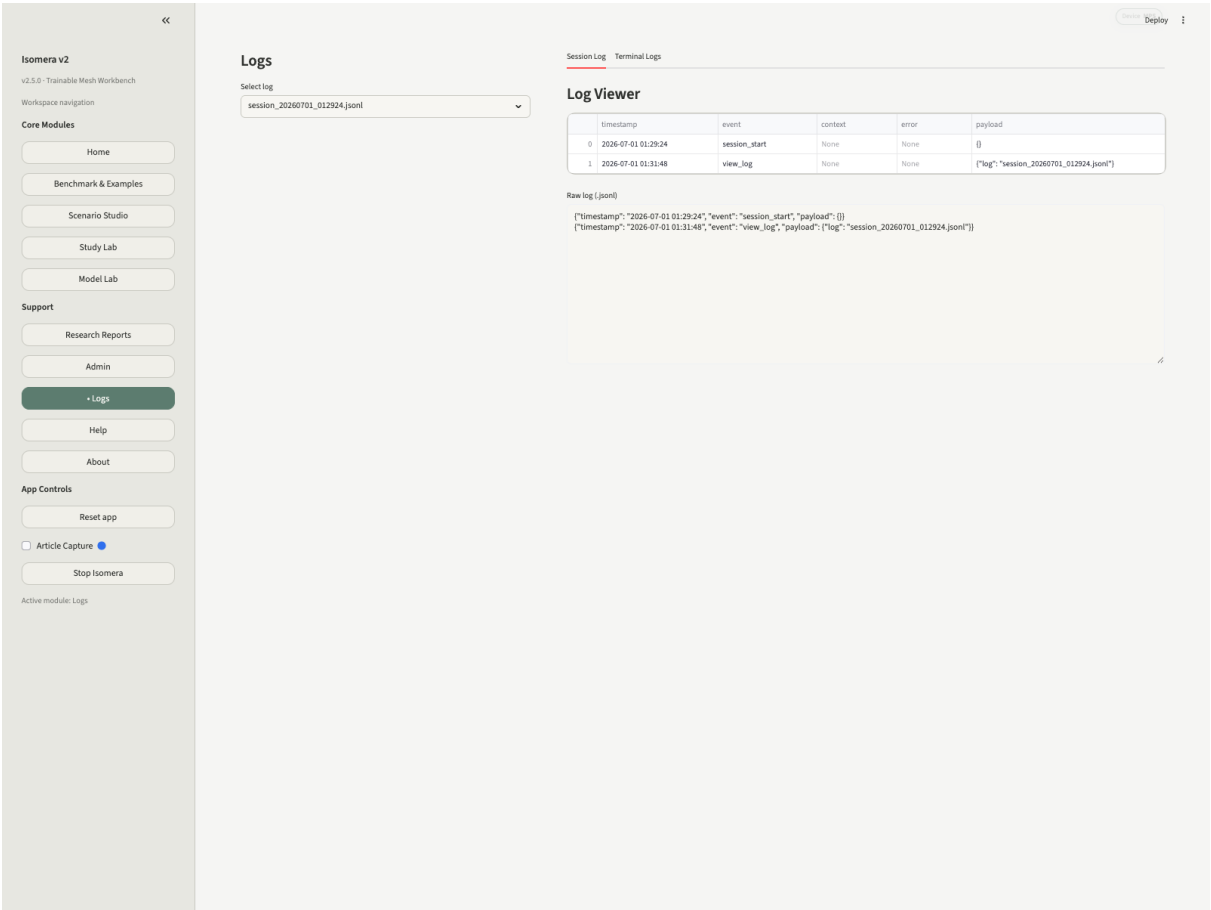


Figure 11.1: Session Log. Eventos estruturados da sessão.

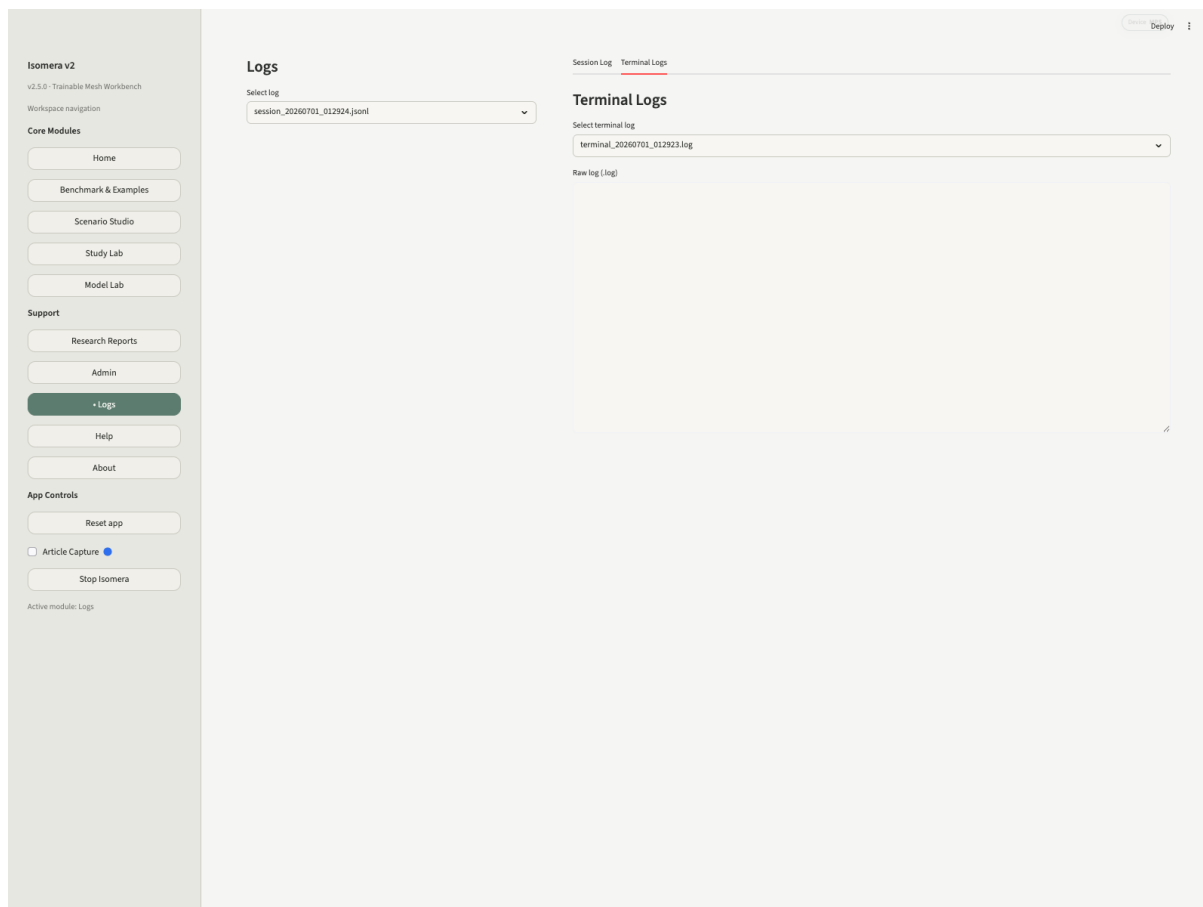


Figure 11.2: Terminal Logs. Logs de terminal quando disponíveis.

Chapter 12

Help e About

12.1 Help

Onde clicar: Menu lateral → Help.

Help concentra apresentação e documentação técnica. A apresentação ajuda em demonstrações; Tech Docs abre os arquivos Markdown do hub técnico.

Help é a área de apoio ao usuário. Ela não deve ser vista como enfeite. Em uma ferramenta de pesquisa aplicada, a documentação precisa estar acessível dentro do produto, porque o usuário alterna constantemente entre conceito e execução. Uma aba explica o roteiro de apresentação; a outra abre a documentação técnica.

Quando uma pessoa nova entra no projeto, Help é o primeiro lugar para aprender vocabulário. Quando uma pessoa técnica precisa revisar uma decisão de arquitetura, Tech Docs é o caminho. Quando uma banca ou professor pede para mostrar o que foi feito, a apresentação embutida ajuda a manter o fluxo.

Dica! Em apresentações, use Help para guiar a narrativa e About para fixar a versão demonstrada. Isso evita confusão entre builds e campanhas.

◀
Deploy

Isomera v2

v2.5.0 - Trainable Mesh Workbench

Workspace navigation

Core Modules

- Home
- Benchmark & Examples
- Scenario Studio
- Study Lab
- Model Lab

Support

- Research Reports
- Admin
- Logs
- + Help
- About

App Controls

- Reset app
- ☐ Article Capture ●
- Stop Isomera

Active module: Help

VMamba-Mesh Presentation
Tech Docs

VMamba-Mesh Presentation

Roteiro visual para uma apresentação dinâmica de 10 minutos, com pontos de troca para a ferramenta.

Mensagem central: o problema não é apenas desenhar grafos de linhagem. O problema é detectar redundância estrutural em Data Mesh de forma reprodutível, comparável e explicável.

Estratégia da apresentação:

- 2 minutos: problema, contexto e evolução dos artigos.
- 3 minutos: por que VMamba-Mesh e o que muda.
- 3 minutos: resultados do Article IV.
- 2 minutos: demonstração no Isomera.

[Download HTML presentation](#)

Live demo switches

Open Deep Learning Workbench

Open Article Reproducibility

Open Research Reports

Content Articles I-III Why VMamba-Mesh Article IV Method Benchmarks Results Live Demo Script Technical Q&A

Problema

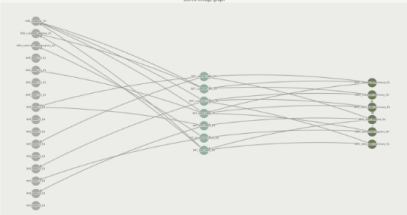
Em arquiteturas Data Mesh, domínios diferentes podem recriar tabelas semanticamente equivalentes. Isso aumenta custo, inconsistência e retrabalho.

Contexto

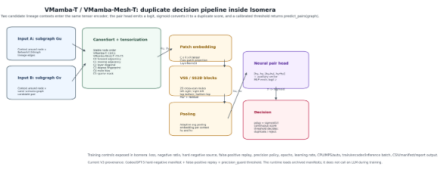
O Isomera representa SOR, SOT e SPEC como grafos de linhagem. A pergunta operacional vira: quais pares de nós/tabelas são redundantes?

O que precisa funcionar na apresentação

Mostrar que os resultados não são só números soltos: eles podem ser reproduzidos dentro da ferramenta.



Input/output contract: lineage graph to duplicate-pair evidence.



Isomera pipeline: graph materialization, model execution, reproducible reports.

Figure 12.1: Help: apresentação VMamba-Mesh.

Figure 12.2: Help: Tech Docs.

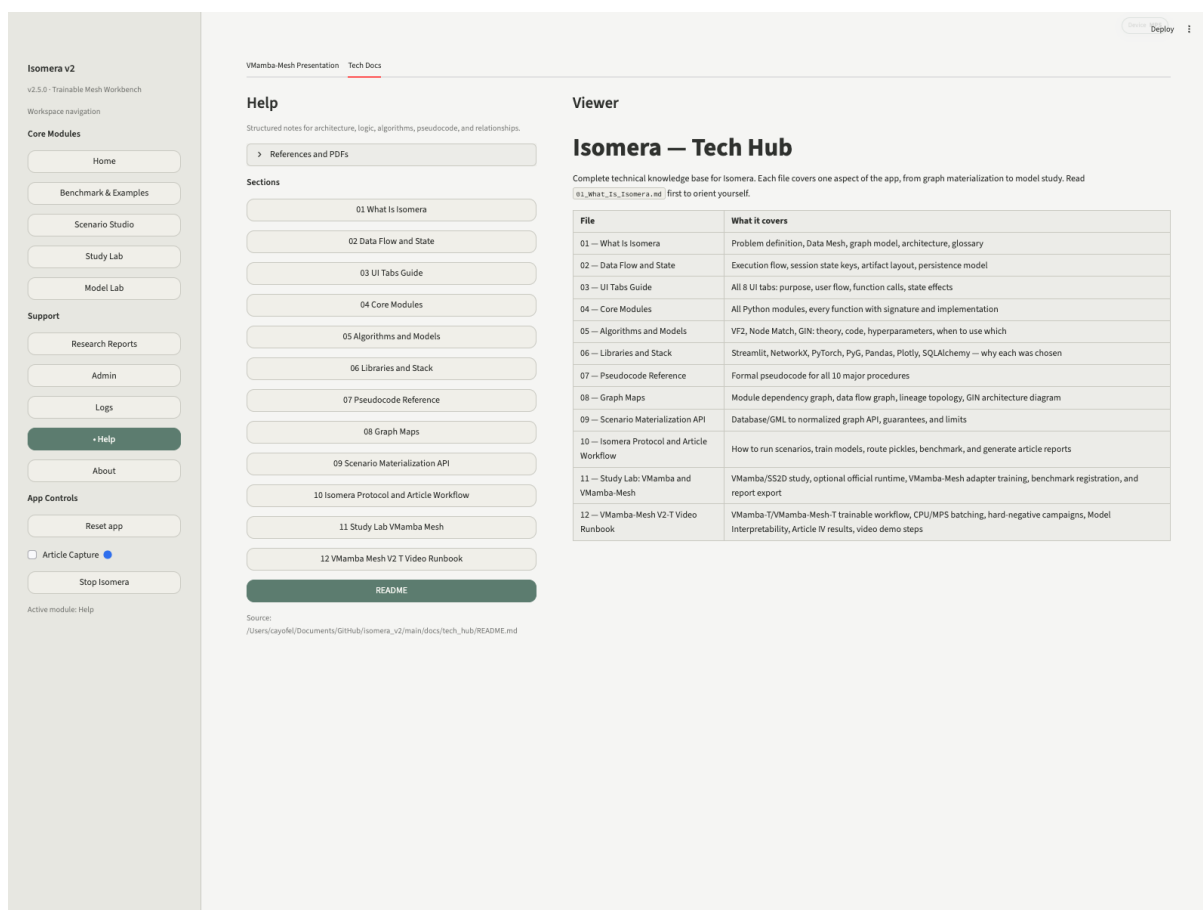


Figure 12.3: Tech Docs com scroll. A rota mostra documentação Markdown longa diretamente dentro do Isomera.

12.2 About

Onde clicar: Menu lateral → About.

About documenta versão, codinome, autor, orientador, contexto do projeto e notas de funcionalidades.

About fecha o ciclo de produto. Ele informa versão, autoria, contexto e notas. Essa tela é útil em gravações, apresentações e entregas públicas porque identifica claramente o que está sendo executado. Em projetos de pesquisa, isso evita confusão entre versões, especialmente quando o mesmo estudo evolui por meses.

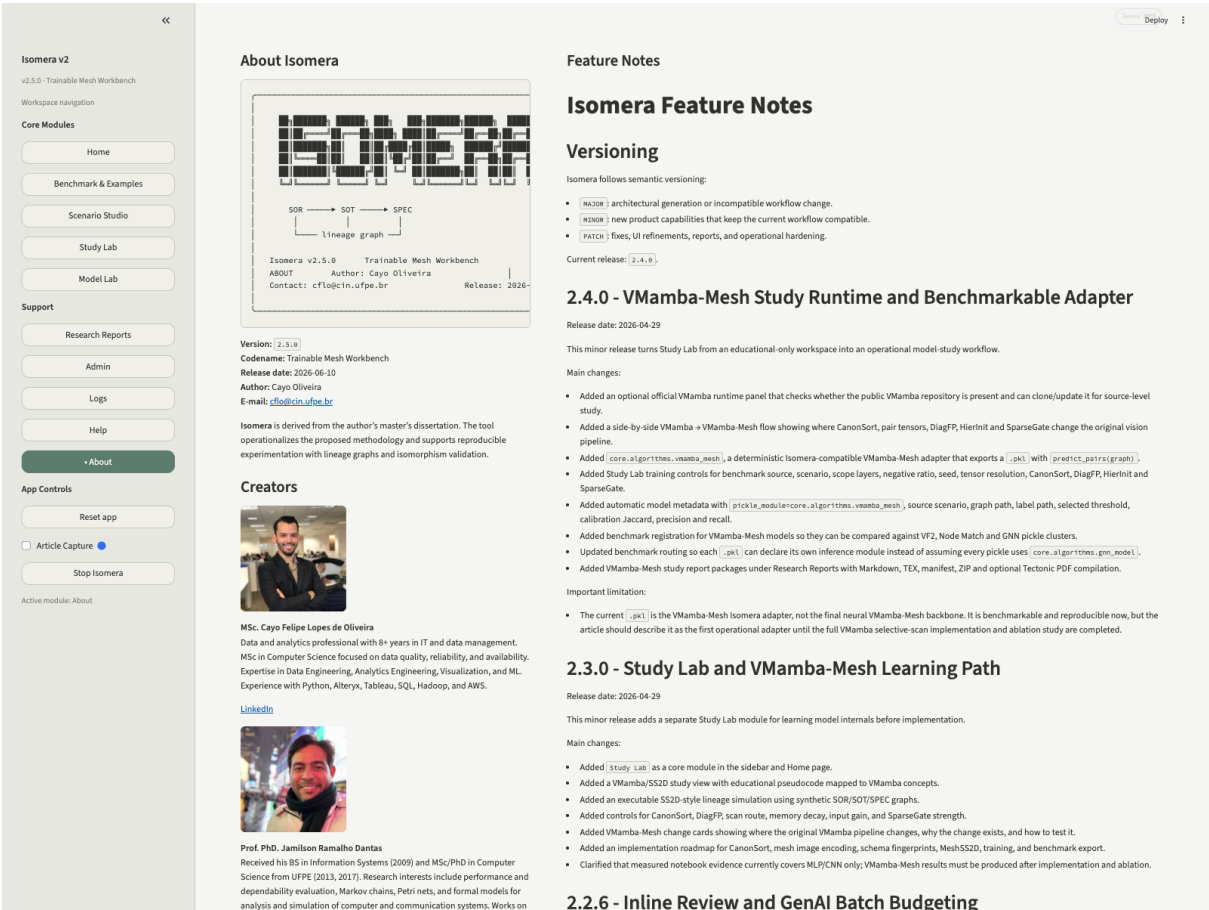


Figure 12.4: About. Informações de versão, autoria e contexto.

The screenshot displays the Isomera v2 web application. On the left is a sidebar with navigation links: Home, Benchmark & Examples, Scenario Studio, Study Lab, Model Lab, Research Reports, Admin, Logs, Help, and About (highlighted). The main content area is divided into two columns. The left column, titled 'About Isomera', features a large 'ISOMERA' logo, a diagram showing the workflow from SOR to SOT to SPEC, and text about the version (v2.5.0), author (Cayo Oliveira), and release date (2026-06-10). Below this is a section for 'Creators' with photos and biographies of Cayo Felipe Lopes de Oliveira and Prof. PhD. Jamilson Ramalho Dantas. The right column, titled 'Feature Notes', contains 'Isomera Feature Notes' and 'Versioning' information, detailing semantic versioning and listing changes for the 2.4.0 release. It also includes a section for '2.3.0 - Study Lab and VMamba-Mesh Learning Path' and '2.2.6 - Inline Review and GenAI Batch Budgeting'.

About Isomera

Version: `2.5.0`
 Codename: Trainable Mesh Workbench
 Release date: 2026-06-10
 Author: Cayo Oliveira
 E-mail: cflo@cin.ufpe.br
 Release: 2926

Isomera v2.5.0 Trainable Mesh Workbench
 ABOUT Author: Cayo Oliveira
 Contact: cflo@cin.ufpe.br Release: 2926

Isomera is derived from the author's master's dissertation. The tool operationalizes the proposed methodology and supports reproducible experimentation with lineage graphs and isomorphism validation.

Creators

MSC. Cayo Felipe Lopes de Oliveira
 Data and analytics professional with 8+ years in IT and data management. MSC in Computer Science focused on data quality, reliability, and availability. Expertise in Data Engineering, Analytics Engineering, Visualization, and ML. Experience with Python, Alteryx, Tableau, SQL, Hadoop, and AWS.
[LinkedIn](#)

Prof. PhD. Jamilson Ramalho Dantas
 Received his BS in Information Systems (2009) and MSC/PhD in Computer Science from UFPE (2013, 2017). Research interests include performance and dependability evaluation, Markov chains, Petri nets, and formal models for analysis and simulation of computer and communication systems. Works on

Feature Notes

Isomera Feature Notes

Versioning

Isomera follows semantic versioning:

- MAJOR**: architectural generation or incompatible workflow change.
- MINOR**: new product capabilities that keep the current workflow compatible.
- PATCH**: fixes, UI refinements, reports, and operational hardening.

Current release: `2.4.0`.

2.4.0 - VMamba-Mesh Study Runtime and Benchmarkable Adapter

Release date: 2026-04-29

This minor release turns Study Lab from an educational-only workspace into an operational model-study workflow.

Main changes:

- Added an optional official VMamba runtime panel that checks whether the public VMamba repository is present and can clone/update it for source-level study.
- Added a side-by-side VMamba → VMamba-Mesh flow showing where CanonSort, pair tensors, DiagFP, HierInit and SparseGate change the original vision pipeline.
- Added `core.algorithms.vmbamba_mesh`, a deterministic isomera-compatible VMamba-Mesh adapter that exports a `.pkl` with `(predict, pairs(graph))`.
- Added Study Lab training controls for benchmark source, scenario, scope layers, negative ratio, seed, tensor resolution, CanonSort, DiagFP, HierInit and SparseGate.
- Added automatic model metadata with `pickle_module:core.algorithms.vmbamba_mesh`, source scenario, graph path, label path, selected threshold, calibration Jaccard, precision and recall.
- Added benchmark registration for VMamba-Mesh models so they can be compared against VF2, Node Match and GNN pickle clusters.
- Updated benchmark routing so each `.pkl` can declare its own inference module instead of assuming every pickle uses `core.algorithms.gnn_model`.
- Added VMamba-Mesh study report packages under Research Reports with Markdown, TEX, manifest, ZIP and optional Tectonic PDF compilation.

Important limitation:

- The current `.pkl` is the VMamba-Mesh Isomera adapter, not the final neural VMamba-Mesh backbone. It is benchmarkable and reproducible now, but the article should describe it as the first operational adapter until the full VMamba selective-scan implementation and ablation study are completed.

2.3.0 - Study Lab and VMamba-Mesh Learning Path

Release date: 2026-04-29

This minor release adds a separate Study Lab module for learning model internals before implementation.

Main changes:

- Added `Study Lab` as a core module in the sidebar and Home page.
- Added a VMamba/SS2D study view with educational pseudocode mapped to VMamba concepts.
- Added an executable SS2D-style lineage simulation using synthetic SOR/SOT/SPEC graphs.
- Added controls for CanonSort, DiagFP, scan route, memory decay, input gain, and SparseGate strength.
- Added VMamba-Mesh change cards showing where the original VMamba pipeline changes, why the change exists, and how to test it.
- Added an implementation roadmap for CanonSort, mesh image encoding, schema fingerprints, MeshSS2D, training, and benchmark export.
- Clarified that measured notebook evidence currently covers MLP/CNN only; VMamba-Mesh results must be produced after implementation and ablation.

2.2.6 - Inline Review and GenAI Batch Budgeting

Figure 12.5: About com scroll. A captura preserva notas de funcionalidades e informações de projeto.

Chapter 13

Métricas e resultados

O Isomera usa métricas de qualidade e eficiência. Jaccard mede a qualidade dos pares duplicados identificados:

$$J = \frac{TP}{TP + FP + FN}$$

Aqui, TP são pares que o modelo marcou como duplicados e realmente eram duplicados. FP são pares marcados como duplicados mas que não eram. FN são pares que o modelo deixou de marcar como duplicados, embora fossem duplicados.

SF-Jaccard combina acerto e tempo. A leitura prática é: quantos pares duplicados corretos por segundo o detector consegue entregar, ponderando a qualidade da identificação.

Dica! Não escolha modelo por uma métrica isolada. Jaccard mostra qualidade do conjunto; tempo mostra viabilidade; SF-Jaccard ajuda a comparar eficiência.

O Jaccard é uma métrica de conjunto. Ele compara o conjunto de pares que o modelo marcou como duplicado com o conjunto de pares que realmente eram duplicados. Se o modelo marca muitos pares errados, o FP cresce e o Jaccard cai. Se deixa de marcar pares verdadeiros, o FN cresce e o Jaccard também cai. Por isso ele é uma métrica adequada quando a pergunta principal é qualidade da identificação.

O tempo mediano de execução mede custo operacional. Ele não diz se o modelo está certo, mas diz se o modelo é viável. Em uma empresa, um detector excelente mas lento demais pode ser ruim para uso recorrente. Um detector rápido mas impreciso pode ser útil como filtro inicial. O Isomera mantém essas dimensões separadas para evitar conclusões simplistas.

O SF-Jaccard junta qualidade e velocidade. Ele não substitui o Jaccard, mas ajuda a enxergar eficiência: quantos pares corretamente identificados por segundo o método entrega, levando em conta a qualidade. Um modelo pode ter Jaccard um pouco menor e SF-Jaccard maior se for muito rápido. Outro pode ter Jaccard melhor e SF-Jaccard menor se custar mais. A escolha depende da aplicação.

Intervalos de confiança representam incerteza estatística sobre a métrica estimada. Se dois modelos têm médias diferentes, mas intervalos amplamente sobrepostos, a evidência de superioridade deve ser tratada com cautela. No Isomera, ICs são úteis para distinguir ganho consistente de variação causada por cenário, seed, amostragem de pares ou ruído experimental.

Fique atento! Não interprete uma diferença pequena de Jaccard como vitória automática. Verifique intervalo de confiança, número de cenários, número de runs, seed, variação entre cenários e custo operacional.

Como ler os resultados. Para pesquisa, olhe primeiro Jaccard e intervalos de confiança. Para produção, olhe também ET e SF-Jaccard. Para explicação de modelo, volte ao Study Lab e abra um par específico.

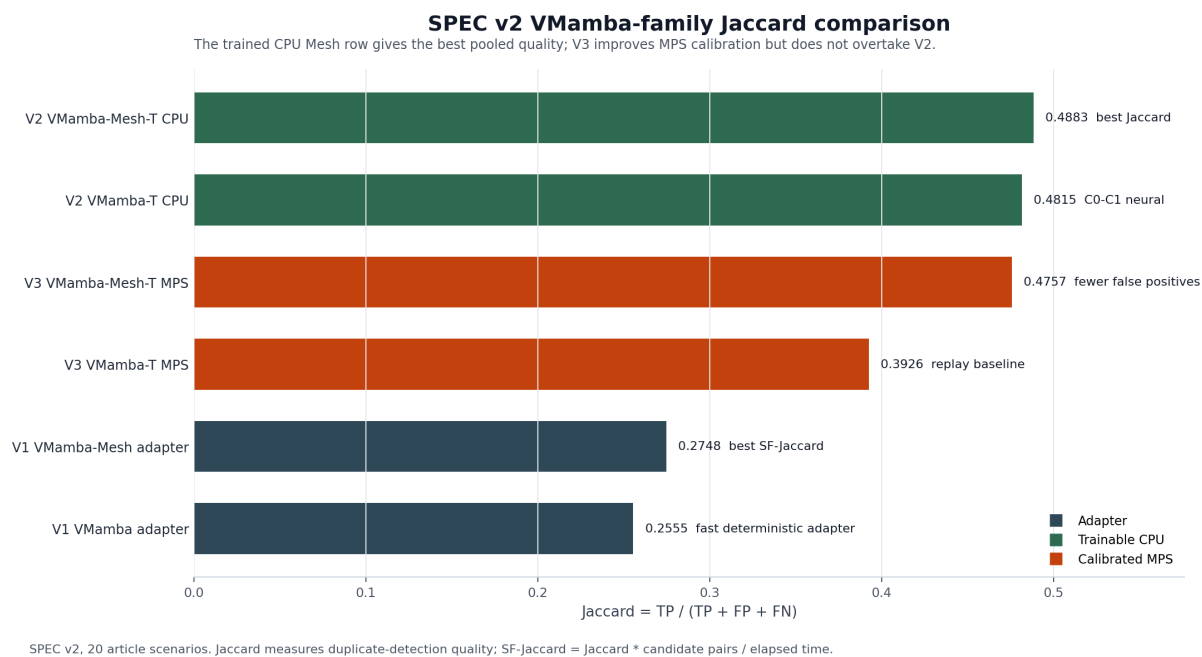


Figure 13.1: Comparação de Jaccard no SPEC v2.

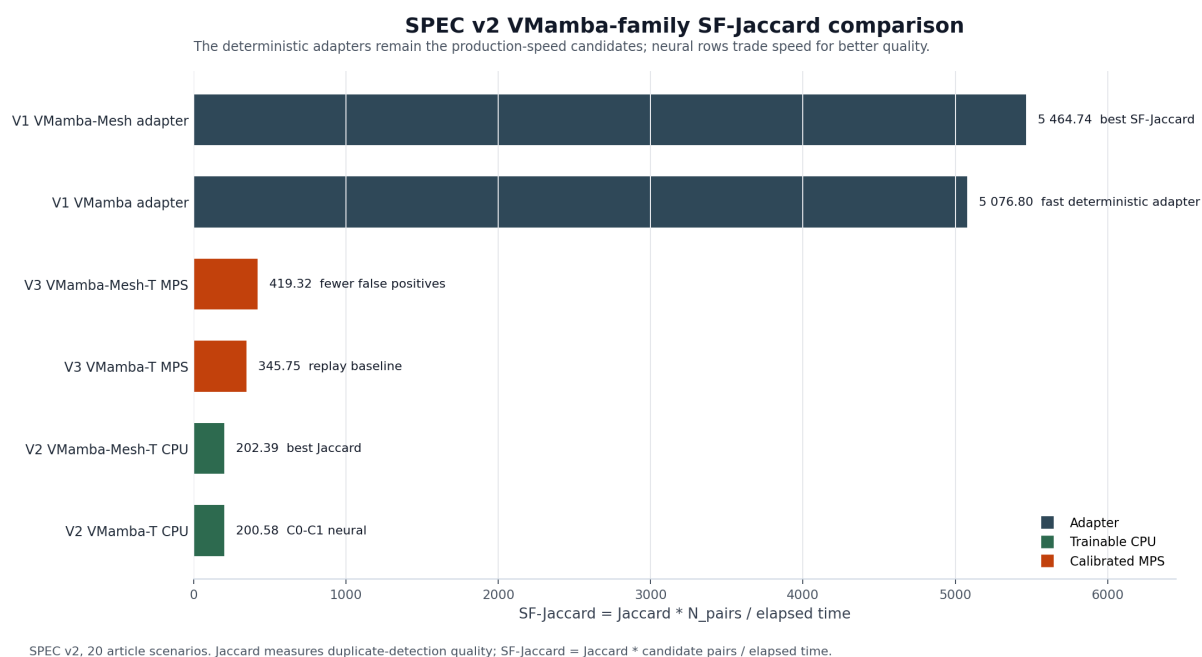


Figure 13.2: Comparação de SF-Jaccard no SPEC v2.

Chapter 14

Famílias de modelos no Isomera

O Isomera foi desenhado para comparar modelos diferentes sob o mesmo contrato. Essa decisão é mais importante do que parece. Se cada modelo tivesse uma entrada, uma saída e uma métrica diferente, a ferramenta viraria um conjunto de experimentos isolados. Ao exigir que todos os detectores retornem pares por meio de um contrato comum, o Isomera permite comparar abordagens determinísticas, modelos de grafo e modelos neurais de tensor no mesmo benchmark.

Dica! Leia as famílias como uma escada de comparação: baselines simples, modelos de grafo, tensorização e versões treináveis. Cada degrau responde uma pergunta diferente.

14.1 VF2

VF2 é um algoritmo clássico de isomorfismo de grafos. Ele tenta verificar correspondência estrutural entre grafos ou subgrafos. No contexto do Isomera, VF2 funciona como baseline auditável. Quando ele marca um par, a decisão está ligada à estrutura e às restrições de matching. Isso é útil para casos em que a duplicidade se parece muito com equivalência estrutural.

O limite do VF2 aparece quando a duplicidade é parcial, semântica ou ruidosa. Duas tabelas podem ser redundantes mesmo que uma transformação intermediária tenha mudado de nome, que uma aresta esteja ausente ou que uma fonte adicional tenha sido incorporada. Nesses casos, matching exato pode ser rígido demais. Ainda assim, VF2 é indispensável porque dá uma referência: se até o baseline determinístico encontra o par, provavelmente o caso é estruturalmente forte.

14.2 Node Match

Node Match é uma família de comparação mais direta. Ela usa propriedades dos nós, rótulos, camadas e vizinhança para encontrar candidatos. É menos expressiva que uma rede neural, mas mais simples de explicar. Em governança, simplicidade importa. Um método simples pode ser usado como triagem, como controle ou como ferramenta de depuração.

Quando Node Match falha, a falha também ensina. Se dois nós parecem duplicados para o especialista, mas não compartilham sinais suficientes para Node Match, talvez a duplicidade esteja em contexto mais amplo. Se Node Match encontra muitos falsos positivos,

talvez os rótulos ou camadas estejam genéricos demais. O Isomera usa esse tipo de baseline para não depender apenas de modelos treináveis.

14.3 GNN e GIN

GNNs aprendem representações de grafos por propagação de mensagens. Cada nó agrega informação dos vizinhos, atualiza embeddings e produz uma representação que pode ser usada para classificar pares. GIN é uma variante forte para distinguir estruturas, por isso faz sentido como baseline neural de grafo.

No Isomera, o GNN/GIN é importante porque trabalha diretamente sobre grafo, enquanto VMamba-Mesh transforma grafo em tensor. Essa diferença cria uma comparação científica interessante. O GNN pergunta: o padrão de duplicidade está melhor representado por mensagens em vizinhança? O VMamba-Mesh pergunta: o padrão de duplicidade está melhor representado por campos matriciais e canais estruturais?

O treino de GNN também exige cuidado com negativos. Como pares duplicados são raros, o modelo pode aprender a dizer “não duplicado” para tudo e ainda parecer bom em acurácia simples. Por isso métricas como Jaccard e SF-Jaccard são mais informativas para a tarefa.

14.4 Vanilla VMamba

O Vanilla VMamba no Isomera representa a ideia de usar uma leitura inspirada em VMamba sem todos os canais de malha. Ele serve como ponto de comparação. A pergunta é: se eu transformar a estrutura em uma imagem ou campo mais simples, quanto consigo detectar? Esse baseline ajuda a medir a contribuição dos canais do VMamba-Mesh.

O nome “vanilla” não significa inútil. Pelo contrário: em alguns cenários ele pode ser rápido e eficiente. O importante é entender o que ele vê. Ele não recebe o mesmo conjunto completo C0–C5 do VMamba-Mesh-T. Portanto, se ele se sai bem em eficiência, isso mostra que há sinais fortes na representação mais simples. Se perde qualidade em certos casos, isso sugere que direção, camada, grau, rota e máscara ajudam.

14.5 VMamba-Mesh adapter

O VMamba-Mesh adapter é determinístico. Ele não é a rede neural treinável. Ele calcula features estruturais a partir do tensor e gera um score ponderado. Esse caminho foi importante porque permitiu validar a engenharia da representação antes de treinar a versão neural. Em pesquisa aplicada, essa etapa é saudável: primeiro se prova que a representação faz sentido; depois se coloca aprendizado treinável em cima.

O adapter também é útil operacionalmente porque tende a ser simples, rápido e explicável. O usuário consegue entender que o score vem de similaridades estruturais, canais e pesos. Ele não substitui o modelo neural, mas ajuda a mostrar a contribuição da tensorização.

14.6 VMamba-T

VMamba-T é a versão treinável que usa um subconjunto mais simples de canais, normalmente C0 e C1. Ela serve para separar dois efeitos. O primeiro efeito é ter uma rede neural treinável. O segundo efeito é ter canais VMamba-Mesh completos. Se VMamba-T melhora em relação a baselines simples, há ganho do caminho neural. Se VMamba-Mesh-T melhora em relação a VMamba-T, há evidência de ganho dos canais adicionais.

O fluxo de decisão é neural. O tensor entra, passa por patch embedding, blocos VSS/SS2D, pooling e head de par. A saída é um logit. A sigmoid transforma o logit em score contínuo. O threshold transforma score em decisão binária.

14.7 VMamba-Mesh-T

VMamba-Mesh-T é a versão treinável completa. Ela usa C0–C5 como entrada e aplica o backbone neural sobre a representação enriquecida. Essa é a versão que mais se aproxima da proposta final de combinar governança de dados, grafos de linhagem e redes neurais visuais.

A diferença central entre VMamba-T e VMamba-Mesh-T está no que a rede recebe. VMamba-T enxerga direção básica. VMamba-Mesh-T enxerga direção, reverso, camada, grau, rota e máscara. Isso dá ao modelo mais sinais para distinguir duplicidade real de semelhança superficial.

Resumo da família. VF2 e Node Match são controles determinísticos. GNN/GIN aprende sobre grafo. Vanilla VMamba testa leitura matricial simplificada. VMamba-Mesh adapter valida features estruturais. VMamba-T testa rede neural treinável com canais básicos. VMamba-Mesh-T testa rede neural treinável com canais completos.

Chapter 15

Treinamento, hiperparâmetros e ablation

Treinar um modelo no Isomera não significa apenas apertar um botão. Significa definir um protocolo. O protocolo precisa dizer quais cenários entram, como pares positivos e negativos são formados, qual loss será usada, qual otimizador atualiza pesos, quantas épocas serão executadas, qual device será utilizado e como o threshold será escolhido.

Dica! Um treino sem seed, hiperparâmetros e manifest não é um experimento reprodutível. Registre o protocolo antes de comparar resultados.

15.1 Dataset supervisionado

A base supervisionada nasce dos pares rotulados. Pares positivos são aqueles que o ground truth considera duplicados. Pares negativos são aqueles que não devem ser classificados como duplicados. Como o número de negativos possíveis costuma ser muito maior, o Isomera usa políticas de negative ratio e hard negatives para formar um conjunto de treino controlado.

Um negative ratio de 4, por exemplo, significa usar quatro negativos para cada positivo. Isso aumenta a exposição do modelo a exemplos não duplicados sem deixar a classe negativa dominar completamente. Hard negatives são negativos parecidos com positivos. Eles são úteis porque forçam o modelo a aprender detalhes, não apenas diferenças óbvias.

15.2 Loss e otimizador

Weighted BCE é adequada quando há desbalanceamento. Ela mantém a forma da binary cross entropy, mas ajusta pesos para reduzir o efeito de classe majoritária. Focal loss é útil quando o modelo precisa concentrar aprendizado em exemplos difíceis. BCE simples é referência, mas pode ser insuficiente quando positivos são raros.

AdamW é usado porque combina adaptação de taxa de aprendizado com weight decay desacoplado. Em modelos neurais pequenos e médios, costuma ser uma escolha estável. O learning rate controla o tamanho dos passos. Weight decay ajuda a regularizar. Epochs controlam quantas passagens o modelo faz sobre o dataset.

Para um par com rótulo $y \in \{0, 1\}$ e probabilidade estimada $\hat{y} = \sigma(z)$, a binary cross entropy é:

$$\mathcal{L}_{BCE} = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})].$$

Na Weighted BCE, os termos positivos e negativos recebem pesos diferentes, o que é útil quando pares duplicados são raros. Em cenários desbalanceados, acurácia pode ser enganosa: um modelo que prediz sempre “não duplicado” pode obter alta acurácia e ainda assim ser inútil para encontrar duplicidades.

Fique atento! Loss baixa não garante bom detector de duplicidade. Em dados desbalanceados, o modelo pode reduzir a perda aprendendo bem a classe majoritária. Por isso, loss, Jaccard, falsos positivos, falsos negativos e threshold precisam ser lidos em conjunto.

15.3 Device: CPU, MPS e auto

CPU é o caminho mais previsível. Em cenários pequenos, pode ser mais rápido porque não paga overhead de enviar operações para GPU. MPS usa Apple Metal no Mac quando PyTorch o expõe. MPS pode ajudar quando há batches maiores, modelos mais largos ou campanhas longas. O modo auto tenta resolver o melhor dispositivo disponível.

O ponto importante é que device não muda a definição do modelo. Ele muda onde as operações rodam. Se CPU e MPS produzem números próximos, a diferença principal deve aparecer em tempo. Se há diferença relevante de qualidade, é preciso investigar determinismo, seed, precisão numérica e caminho de execução.

15.4 Batching

Batching existe para reduzir overhead. Em vez de calcular um par por vez, o Isomera pode processar embeddings ou pares em blocos. Isso é especialmente importante para MPS. Sem batching suficiente, o custo de despacho pode superar o ganho de paralelismo. Com batching maior, o hardware passa a ser melhor aproveitado.

No manual, o usuário deve lembrar de duas variáveis: encoder batch e inference batch. Encoder batch controla quantos contextos são codificados juntos. Inference batch controla quantos pares são avaliados em bloco. Ajustar esses valores é parte da engenharia de performance.

15.5 Ablation

Ablation é o estudo de remover ou alterar componentes para entender contribuição. No Isomera, ablations podem envolver canais, loss, negative ratio, hard negatives, device, batch size, epochs, threshold e replay de falsos positivos. O objetivo não é apenas procurar um número maior; é entender por que uma mudança melhora uma métrica e piora outra.

Um bom ablation precisa registrar antes e depois. Se uma configuração aumenta SF-Jaccard mas reduz Jaccard, ela pode ser boa para triagem rápida e ruim para decisão conservadora. Se melhora tempo em MPS mas mantém qualidade, pode ser boa para

campanhas longas. Se melhora qualidade mas o intervalo de confiança ainda cruza zero, a evidência deve ser apresentada com cuidado.

Um estudo de ablation deve alterar uma variável por vez sempre que possível. Se canais, loss, threshold e negative ratio mudam simultaneamente, o resultado não identifica qual componente causou a diferença observada. A comparação mais forte é do tipo controle versus variante: manter dataset, seed, split, modelo-base e métrica constantes, alterando apenas o componente investigado.

Na prática: Ao usar a tela de treino, anote: benchmark, cenários, modelo, canais, loss, otimizador, learning rate, weight decay, epochs, negative ratio, hard negatives, threshold, device, encoder batch, inference batch e seed. Esses campos são a diferença entre experimento e lembrança solta.

Chapter 16

Playbooks de uso

Dica! Use os playbooks como roteiro de operação, não como teoria. Eles são úteis para gravação, aula, banca e onboarding de novos usuários.

16.1 Playbook 1: primeira abertura

Abra o Isomera e permaneça na Home. Antes de clicar em qualquer execução, observe o menu lateral. A ferramenta está organizada em módulos porque cada módulo representa um papel. Benchmark mede. Scenario Studio cria. Study Lab explica. Model Lab registra. Research Reports arquiva. Admin configura. Logs auditam. Help ensina. About identifica.

Depois, abra Benchmark & Examples e vá para Concepts. Essa é a melhor segunda tela para quem está começando. Ela explica cenários, nomes, camadas e cobertura de modelos. Não rode nada ainda. Entenda primeiro o que será medido.

Em seguida, abra Study Lab. Passe pelas abas sem executar treino. Veja Deep Learning Workbench, Original VMamba, SS2D Demo e VMamba-Mesh Changes. O objetivo é entender o vocabulário. Só depois disso faz sentido abrir Train Model Adapter ou Model Interpretability.

16.2 Playbook 2: executar um benchmark pronto

Entre em Benchmark & Examples. Escolha o benchmark. Abra Run Benchmark. Verifique se a família de modelo está disponível. Se houver aviso de dependência, não ignore. Dependências como torch ou torch-geometric alteram quais modelos conseguem rodar.

Escolha uma política de execução adequada. Para teste rápido, use cenário pequeno ou execução limitada. Para resultado de campanha, use configuração registrada. Depois de rodar, leia métricas e salve relatório. Não copie apenas o número final; confira também tempo, pares encontrados, status e manifest.

Depois da execução, vá para Research Reports. Abra o pacote gerado. Confirme se existem CSV, JSON, Markdown e figuras. Se algo não apareceu, volte aos logs. Esse caminho é o ciclo básico de execução auditável.

16.3 Playbook 3: explicar VMamba-Mesh

Comece mostrando o grafo e a matriz. Explique que o grafo mostra a história da tabela e a matriz transforma essa história em campo computacional. Depois mostre os canais C0–C5. Cada canal adiciona uma visão: direção, reverso, camada, grau, rota e máscara.

Abra Study Lab. Em Original VMamba, explique que a inspiração vem de modelos visuais que leem mapas 2D. Em Run SS2D Demo, mostre como rotas percorrem a matriz. Em VMamba-Mesh Changes, explique CanonSort, DiagFP, MeshSS2D, HierInit e SparseGate. Por fim, vá para Deep Learning Workbench e compare dois modelos no mesmo par.

16.4 Playbook 4: treinar modelo

Abra Study Lab e entre em Train Model Adapter. Escolha benchmark, cenário e família. Se escolher VMamba-T, lembre que a entrada é mais simples. Se escolher VMamba-Mesh-T, a entrada usa C0–C5. Configure loss, otimizador, learning rate, epochs, negative ratio e device.

Antes de rodar, leia o aviso de tempo. Se estiver em Mac, teste CPU e MPS com batches adequados. Para uma gravação curta, não treine do zero; mostre a tela, explique os parâmetros e depois abra Model Reports com a campanha salva.

Depois do treino, confira se o pickle foi salvo e se o manifest registra hiperparâmetros. Em seguida, abra Model Lab para verificar se o artefato aparece. Por fim, rode Benchmark para medir o modelo.

16.5 Playbook 5: analisar interpretabilidade

Abra Study Lab e entre em Model Interpretability. Escolha benchmark, cenário, par e modelo. O ideal é usar um par conhecido, como SOR16-D1, porque o grafo e os canais já aparecem em outros materiais. Primeiro leia o par. Depois observe o grafo. Em seguida veja a matriz. Só então interprete saliency.

Ao olhar a saliency, não diga que ela prova causalidade. Diga que ela mostra sensibilidade local do score. Se a região forte aparece em canais coerentes, isso sugere que o modelo está usando evidência estrutural. Se aparece em regiões vazias, isso pode indicar problema de treino ou representação.

16.6 Playbook 6: criar cenário novo

Abra Scenario Studio. Escolha a fonte: Scenario Warehouse, GML Catalog ou Manual Builder. Carregue o grafo e confira visualmente. Se a estrutura não faz sentido, pare. Não avance para labels ou treino com um grafo ruim.

Depois configure filtros de pares. Comece conservador: mesma camada, domínio relevante e assinaturas estruturais. Revise pares candidatos. Publique o cenário apenas quando o ground truth estiver claro. Depois disso, o cenário poderá aparecer no Benchmark e no treino.

16.7 Playbook 7: usar em uma empresa

Comece com uma prova de conceito em um domínio. Extraia linhagem, gere grafo, rotule poucos pares e rode baselines. Mostre achados para especialistas de negócio. Ajuste regras de candidatura. Só depois treine modelos neurais.

Em produção, integre com catálogo e processos de governança. O Isomera pode apontar duplicidades, mas a decisão de consolidar tabelas precisa passar por owners, impacto downstream, contratos de dados e comunicação com consumidores.

Chapter 17

Glossário operacional

Dica! Consulte o glossário quando um termo aparecer em várias telas. Ele padroniza o vocabulário entre produto, relatório e artigo.

Termo	Significado no Isomera
Benchmark	Conjunto de cenários, grafos, pares reais e regras de execução usado para comparar detectores.
Cenário	Um grafo específico de linhagem, geralmente identificado por quantidade de SOR, domínios e seed.
Ground truth	Lista de pares considerados duplicados de acordo com validação ou contrato do estudo.
Detector	Algoritmo ou modelo que recebe grafo e retorna pares candidatos a duplicidade.
Pickle	Artefato serializado de modelo, usado para carregar pesos, parâmetros ou objeto executor.
Manifest	Arquivo de rastreabilidade com parâmetros, seeds, artefatos, métricas e contexto de execução.
CanonSort	Ordenação canônica dos nós antes de construir matriz/tensor.
DiagFP	Marcação diagonal com sinais de camada e identidade estrutural.
MeshSS2D	Adaptação da leitura SS2D para rotas coerentes com linhagem.
SparseGate	Mecanismo para reduzir influência de células vazias em matriz esparsa.
Hard negative	Par negativo difícil, estruturalmente parecido com positivo, usado para treinar melhor o classificador.
Logit	Saída bruta do head neural antes da sigmoid.
Sigmoid	Função que transforma logit em score contínuo entre 0 e 1.
Threshold	Corte usado para transformar score em decisão duplicado/não duplicado.
Saliency	Mapa de sensibilidade local do score em relação ao tensor de entrada.

SF-Jaccard

Métrica que combina qualidade de Jaccard com tempo para estimar eficiência de pares corretos por segundo.

Chapter 18

Catálogo visual tela a tela

Este capítulo é um manual de campo. Ele repete as telas principais em formato de ficha para que o leitor consiga estudar a ferramenta como estudaria um produto corporativo: uma página por rota, com intenção, controles, teoria e checklist. A repetição é proposital. Nos capítulos anteriores, as telas aparecem dentro da narrativa. Aqui, elas aparecem como referência rápida para operação e treinamento.

Dica! Este catálogo é para consulta rápida. Use as fichas quando precisar lembrar onde clicar, o que observar e como validar uma tela específica.

18.1 Home

Onde clicar: Abrir o Isomera e permanecer na tela inicial.

Quando usar. Use esta tela para explicar o produto inteiro antes de entrar em detalhes. Ela é o ponto de partida para usuário novo, demonstração, aula ou revisão de versão.

O que observar na tela. Observe o menu lateral, a separação entre módulos centrais e suporte, o controle de captura e o botão de parada. A Home mostra que a ferramenta não é apenas benchmark: ela cobre criação, estudo, modelos, relatórios, administração e documentação.

Teoria ou algoritmo por trás. A Home organiza uma arquitetura de produto em torno do ciclo de vida do experimento: criar cenário, executar detector, estudar modelo, salvar relatório e auditar execução.

Checklist de validação. Verificar se o menu lateral está visível, se a versão/codiname aparecem, se os módulos principais abrem e se o app está responsivo.

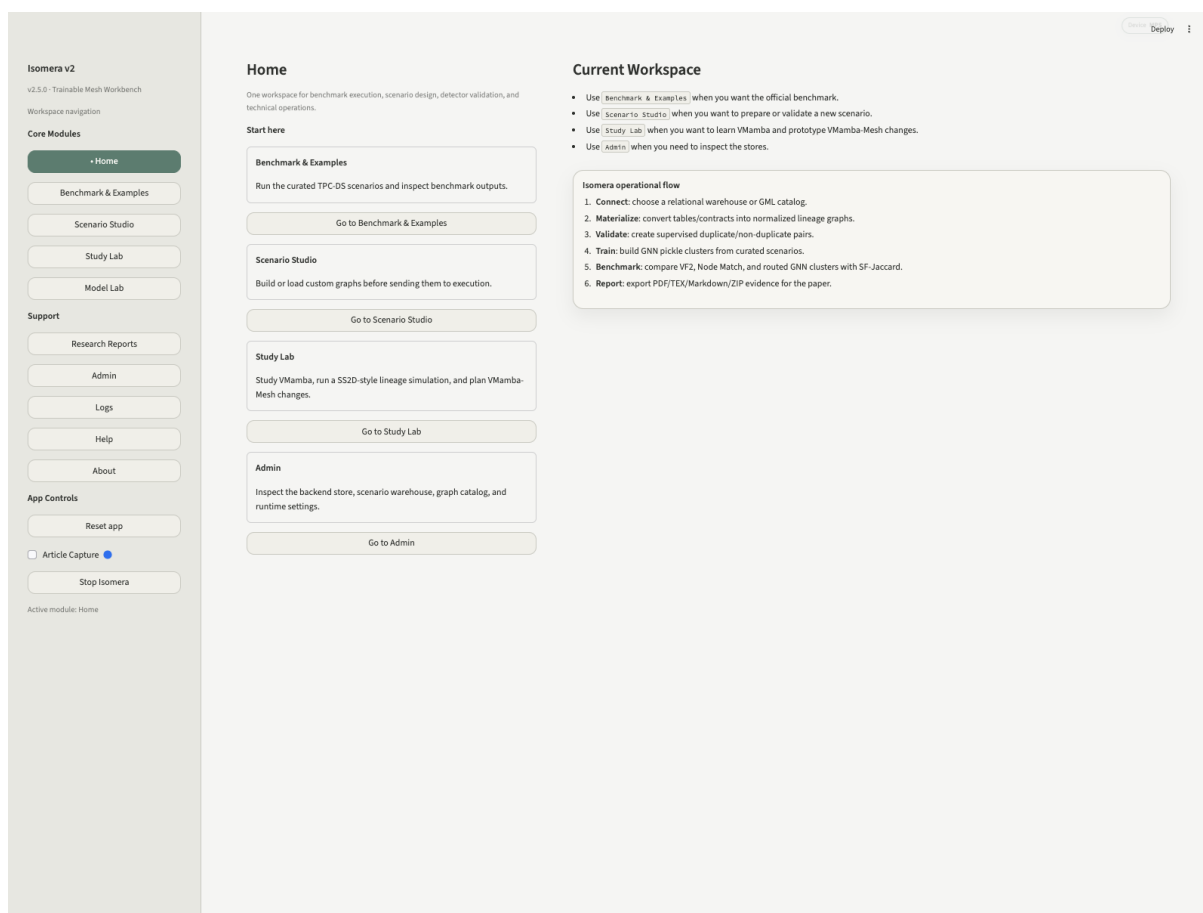


Figure 18.1: Ficha visual: Home

18.2 Benchmark: reprodução de campanha

Onde clicar: Benchmark & Examples → Article Reproducibility.

Quando usar. Use quando quiser repetir uma campanha salva ou mostrar que o resultado não depende de memória informal.

O que observar na tela. Observe seletor de artigo/campanha, benchmark, cenário, modelo, seed e modo de execução. A tela deve deixar claro o que será reproduzido antes de rodar.

Teoria ou algoritmo por trás. A reprodução é baseada em manifestos. O manifesto transforma uma execução em objeto auditável: entradas, parâmetros, artefatos, métricas esperadas e tolerâncias.

Checklist de validação. Conferir campanha selecionada, modo de execução, cenário, modelo e se o trace aparece depois da execução.

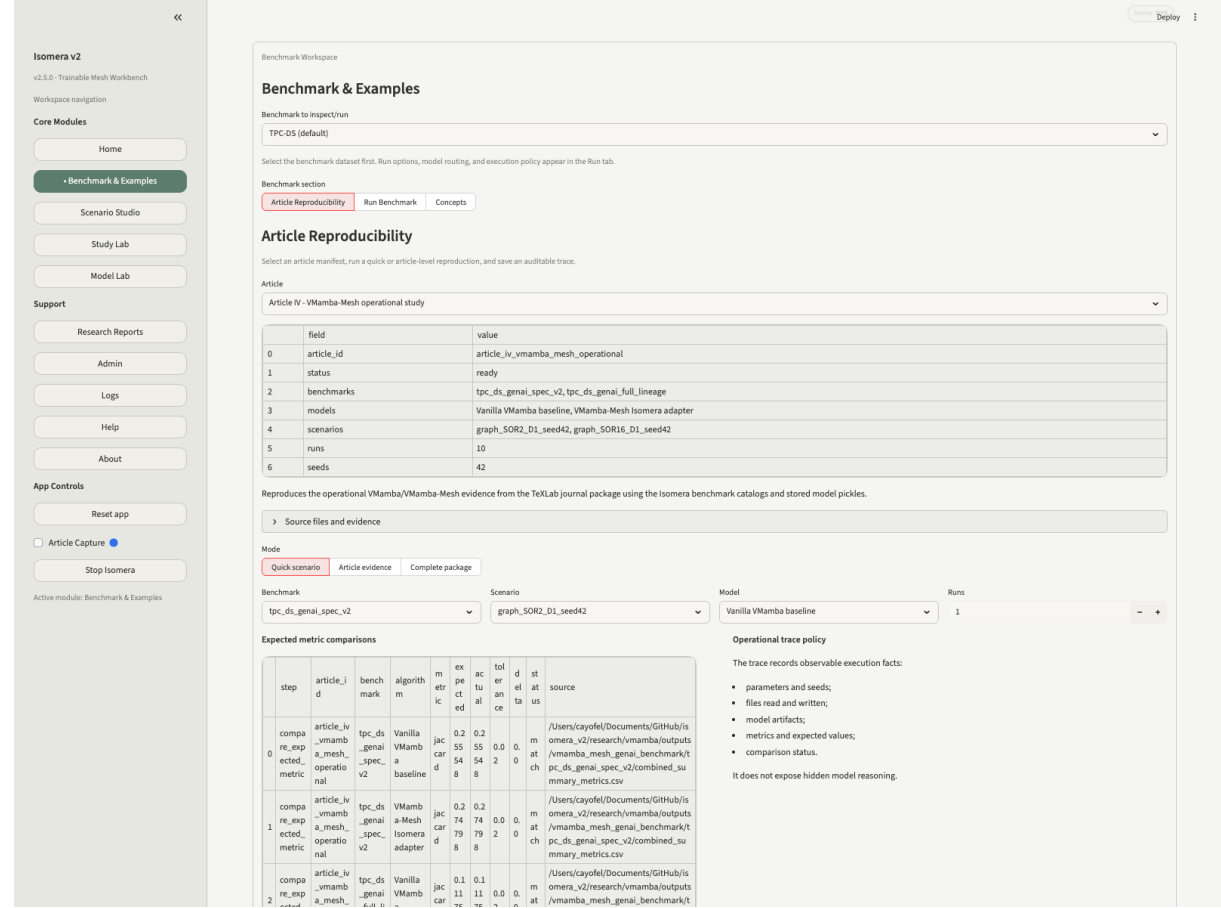


Figure 18.2: Ficha visual: Benchmark: reprodução de campanha

18.3 Benchmark: execução direta

Onde clicar: Benchmark & Examples → Run Benchmark.

Quando usar. Use quando quiser rodar um detector agora, sem necessariamente seguir um pacote de reprodução completo.

O que observar na tela. Observe benchmark ativo, modelos disponíveis, política de execução, roteamento e área de resultados.

Teoria ou algoritmo por trás. A execução chama o contrato de cada detector, recebe pares previstos e compara com ground truth. Daí nascem TP, FP, FN, Jaccard, tempo e SF-Jaccard.

Checklist de validação. Verificar se o benchmark correto está selecionado, se o modelo tem dependências instaladas e se o relatório foi salvo após a execução.

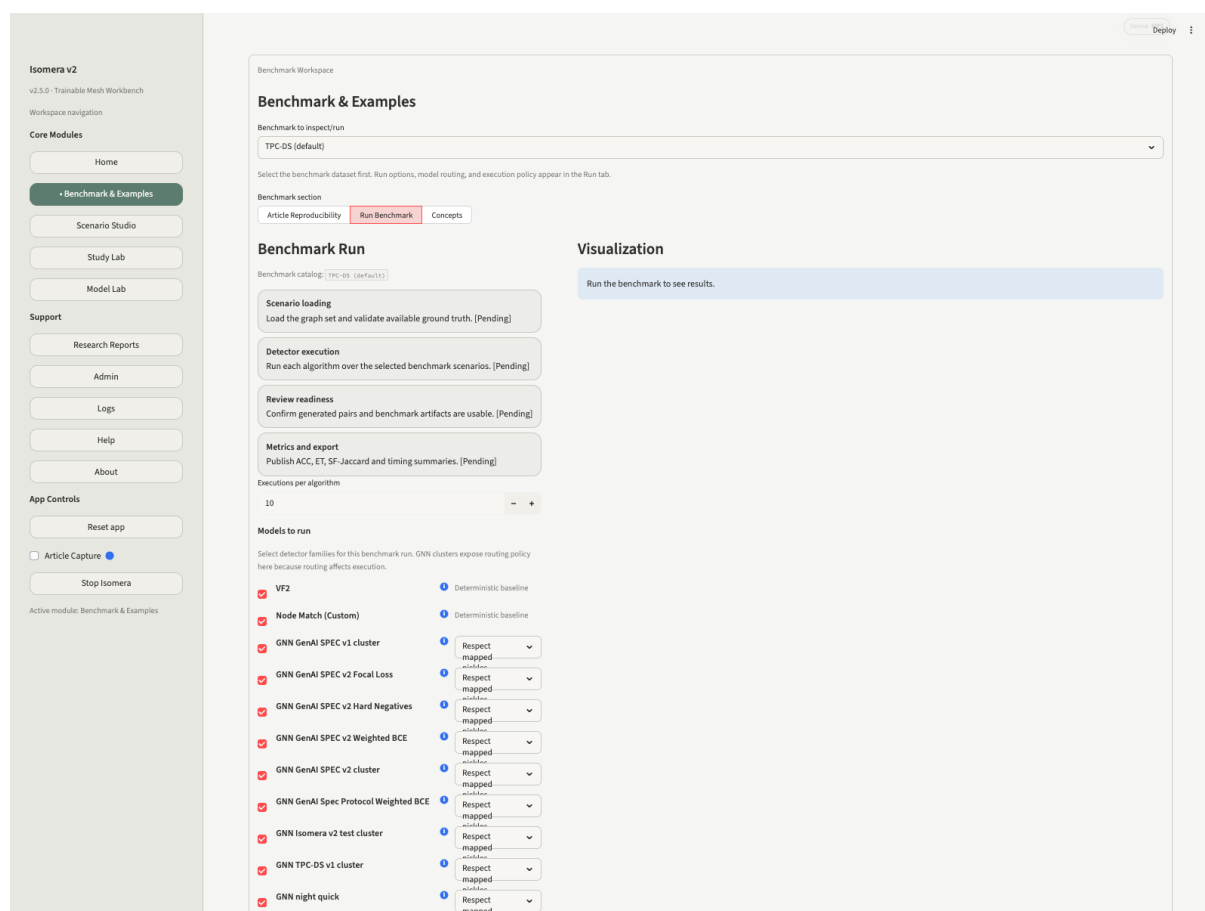


Figure 18.3: Ficha visual: Benchmark: execução direta

18.4 Benchmark: conceitos

Onde clicar: Benchmark & Examples → Concepts.

Quando usar. Use antes de executar um benchmark novo ou quando precisar explicar cenário, pares e cobertura.

O que observar na tela. Observe nomes de cenários, matriz SOR x Domínios, pares reais e cobertura de roteamento por família de modelo.

Teoria ou algoritmo por trás. A tela conecta estrutura experimental com interpretação. O nome do cenário codifica fontes, domínios e seed; a matriz ajuda a entender o espaço de avaliação.

Checklist de validação. Confirmar cenários, ground truth, cobertura de modelos e se os pares reais podem ser inspecionados.

The screenshot displays the 'Benchmark & Examples' section of the Isomera v2 interface. The left sidebar contains navigation links for Home, Benchmark & Examples (active), Scenario Studio, Study Lab, Model Lab, Support, Research Reports, Admin, Logs, Help, and About. Below these are App Controls including a Reset app button, an Article Capture toggle, and a Stop Isomera button. The main content area is titled 'Benchmark Concepts' and includes sections for Benchmark scope, Scenario naming, Research goal, and Model routing coverage. The Model routing coverage section contains a table with columns: model_family, coverage, status, route_policy, candidate_pickles, and best_of_active. The table lists various benchmarks and their routing status.

	model_family	coverage	status	route_policy	candidate_pickles	best_of_active
0	GNN GenAI SPEC v1 cluster	20/20	complete	explicit_map	0	False
1	GNN GenAI SPEC v2 Focal Loss	20/20	complete	explicit_map	0	False
2	GNN GenAI SPEC v2 Hard Negatives	20/20	complete	explicit_map	0	False
3	GNN GenAI SPEC v2 Weighted BCE	20/20	complete	explicit_map	0	False
4	GNN GenAI SPEC v2 cluster	20/20	complete	explicit_map	0	False
5	GNN GenAI Spec Protocol Weighted BCE	20/20	complete	explicit_map	0	False
6	GNN Isomera v2 test cluster	2/20	partial	explicit_map	0	False
7	GNN TPC-DS v1 cluster	20/20	complete	explicit_map	0	False
8	GNN night quick	1/20	partial	explicit_map	0	False
9	VMamba Mesh Isomera adapter	20/20	complete	explicit_map	0	False
10	VMamba Mesh Isomera adapter	1/20	partial	explicit_map	0	False
11		20/20	complete	explicit		

Figure 18.4: Ficha visual: Benchmark: conceitos

18.5 Scenario Studio: visão geral

Onde clicar: Scenario Studio.

Quando usar. Use para criar, carregar ou revisar cenários antes de qualquer benchmark.

O que observar na tela. Observe a jornada vertical, o método de entrada e as etapas de fonte, validação e treino.

Teoria ou algoritmo por trás. O Scenario Studio materializa grafos e prepara ground truth. Ele é a etapa em que dados brutos viram benchmark treinável.

Checklist de validação. Confirmar fonte do cenário, carregamento do grafo, filtros de pares e destino de publicação.

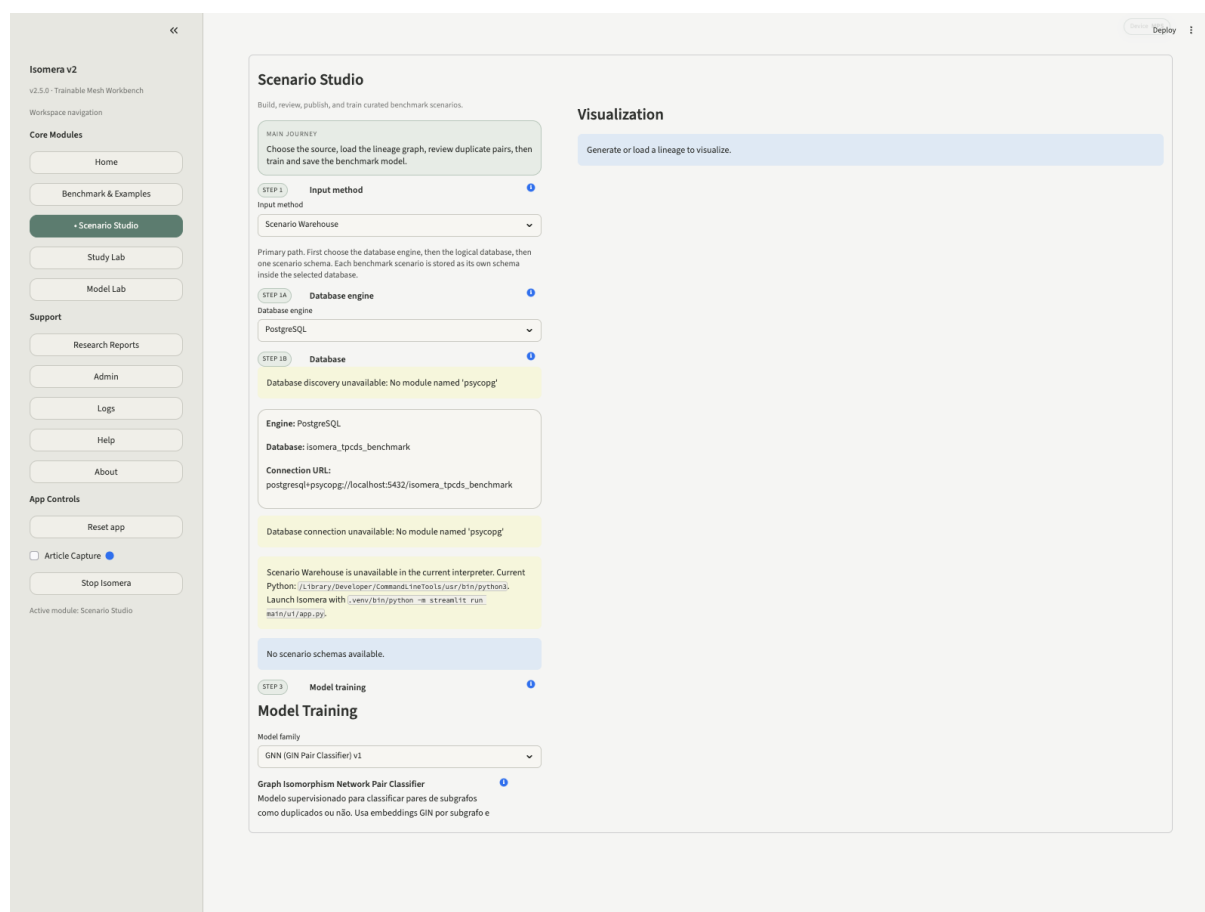


Figure 18.5: Ficha visual: Scenario Studio: visão geral

18.6 Scenario Studio: fonte relacional

Onde clicar: Scenario Studio → escolher Scenario Warehouse.

Quando usar. Use quando o cenário vem de banco relacional, schemas ou materialização persistida.

O que observar na tela. Observe engine, database, schema, status de conexão e detalhes da estrutura.

Teoria ou algoritmo por trás. A fonte relacional permite transformar arquiteturas de dados reais em grafos. Cada tabela vira nó; dependências viram arestas.

Checklist de validação. Verificar conexão, schema correto, quantidade de tabelas e se o carregamento do cenário conclui sem erro.

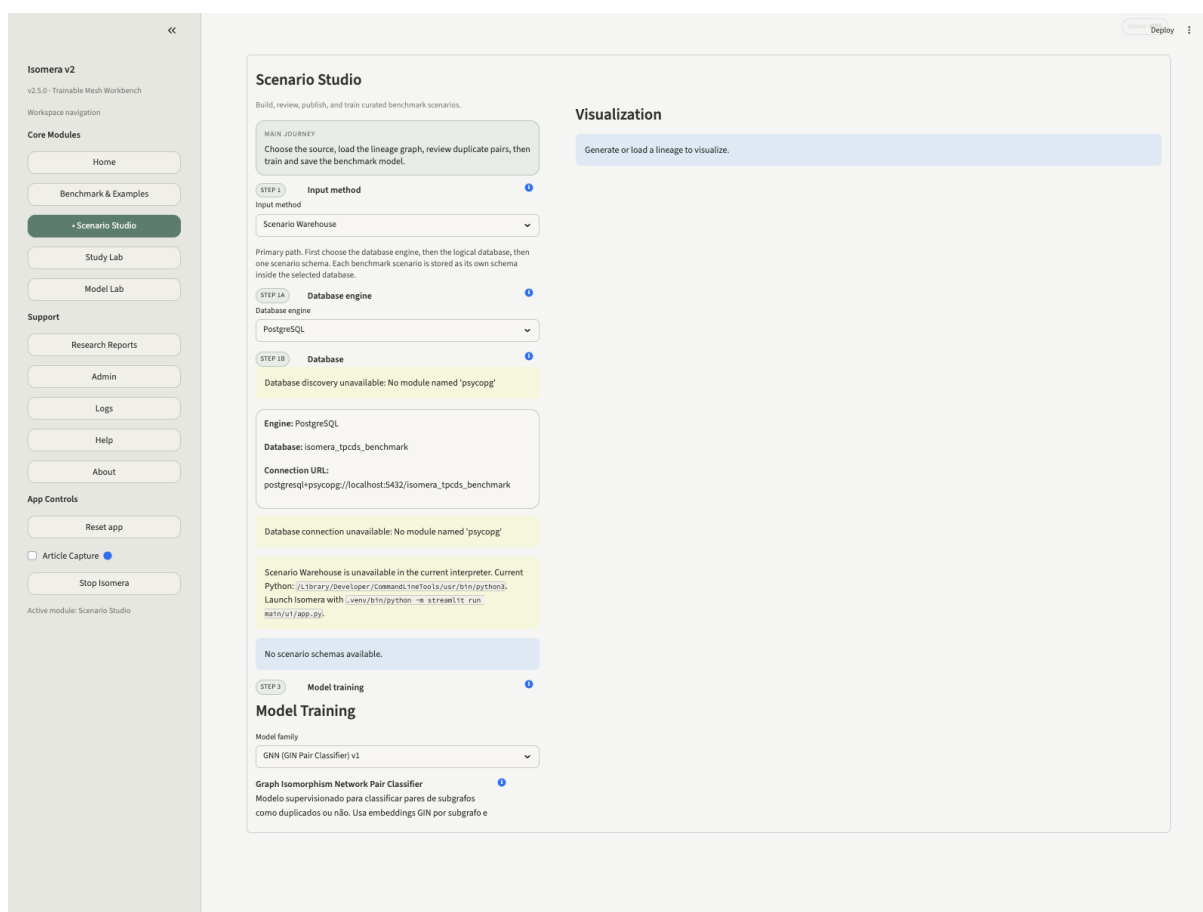


Figure 18.6: Ficha visual: Scenario Studio: fonte relacional

18.7 Scenario Studio: validação de pares

Onde clicar: Scenario Studio → descer até filtros e validação.

Quando usar. Use depois de carregar um grafo, antes de publicar o benchmark.

O que observar na tela. Observe filtros por camada, domínio, grau, assinatura de pais/filhos e fila de revisão.

Teoria ou algoritmo por trás. A validação cria ground truth. Bons labels dependem de filtros claros e revisão humana ou assistida por LLM.

Checklist de validação. Conferir quantidade de pares antes/depois dos filtros, pares revisados e destino do benchmark.

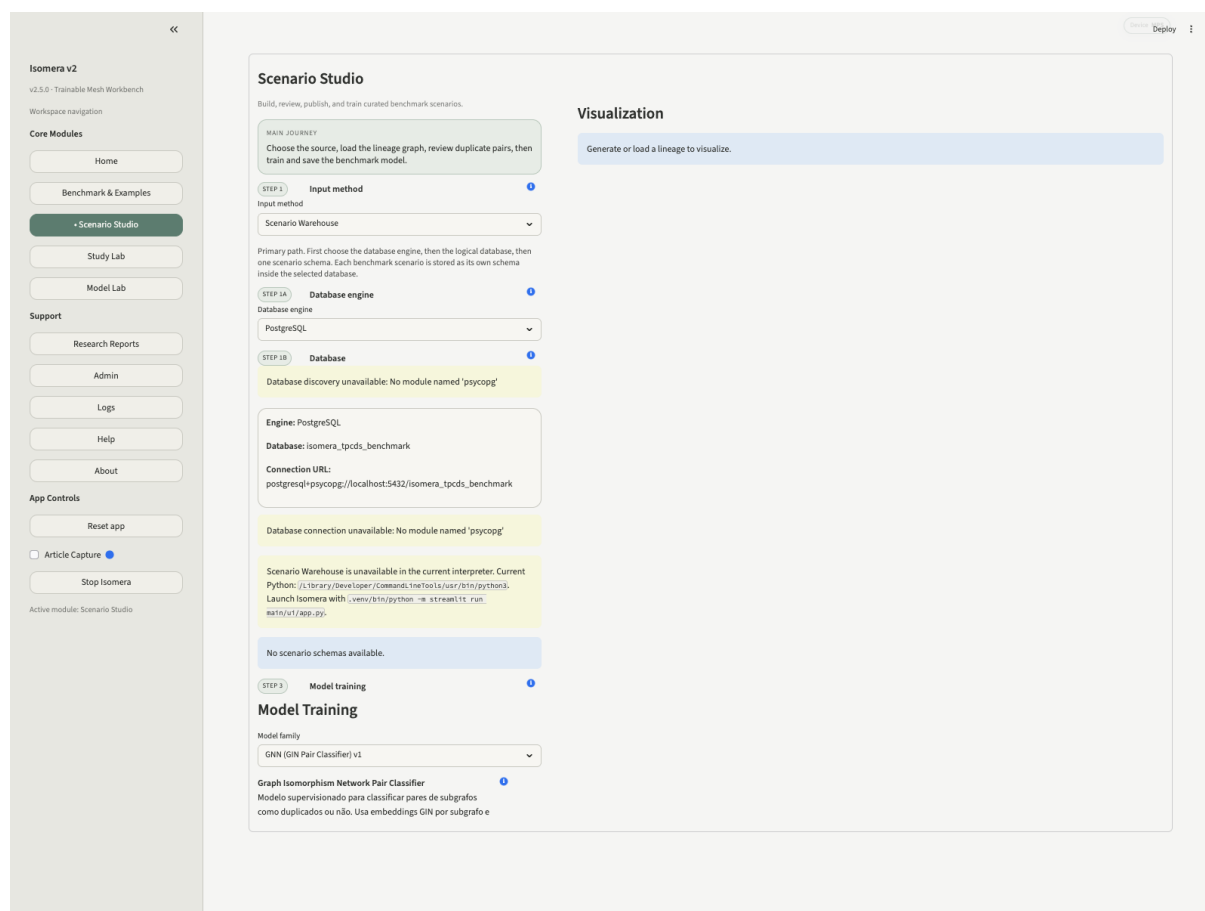


Figure 18.7: Ficha visual: Scenario Studio: validação de pares

18.8 Scenario Studio: treinamento

Onde clicar: Scenario Studio → seção Model Training.

Quando usar. Use para treinar modelos vinculados aos cenários publicados.

O que observar na tela. Observe família de modelo, protocolo, parâmetros, loss, otimizador, balanceamento, device e destino do pickle.

Teoria ou algoritmo por trás. O treinamento transforma pares rotulados em artefato reutilizável. O manifest precisa registrar hiperparâmetros e contexto.

Checklist de validação. Confirmar cenários de treino, seed, device, batch, loss, otimizador, nome do artefato e registro final.

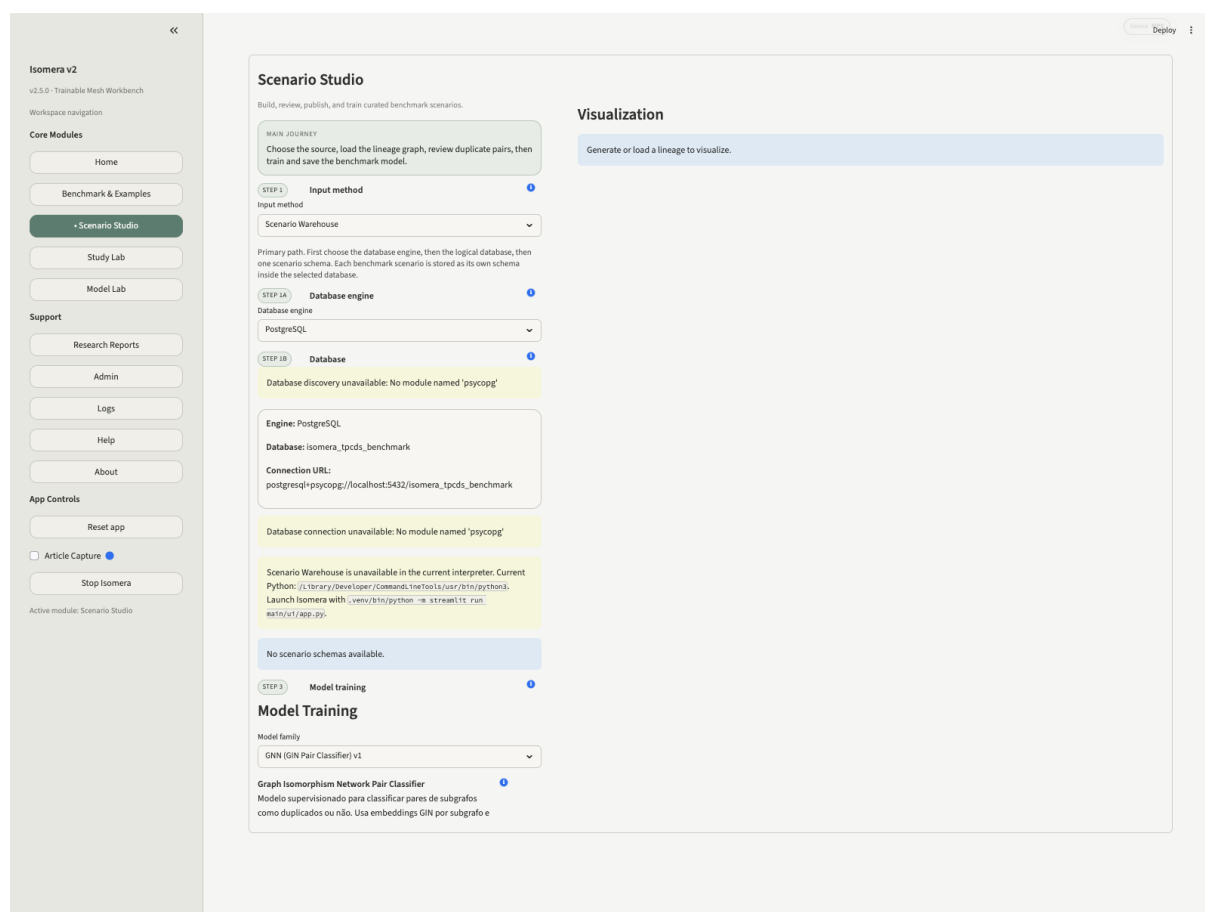


Figure 18.8: Ficha visual: Scenario Studio: treinamento

18.9 Study Lab: Deep Learning Workbench

Onde clicar: Study Lab → Deep Learning Workbench.

Quando usar. Use para comparar modelos no mesmo benchmark, cenário e par.

O que observar na tela. Observe seletores de benchmark, cenário, par, modelo A, modelo B, features e outputs lado a lado.

Teoria ou algoritmo por trás. A bancada usa o mesmo contrato do benchmark, mas em escala local. Ela é a ponte entre métrica agregada e caso específico.

Checklist de validação. Selecionar par conhecido, comparar modelos e verificar se score/decisão mudam de forma coerente.

Study Lab

Goal
Study deep learning model families, inspect lineage tensors, compare benchmark-compatible outputs, and reproduce article evidence with an auditable trace.

Study focus
Deep Learning Workbench

Model family
Vanilla VMamba baseline

Use this module to study model families, inspect VMamba concepts, train adapters, and register benchmarkable artifacts.

References

- VMamba repository: [kzeronko/VMamba](#)
- VMamba paper: [arXiv:2403.12046](#)
- Mamba paper: [arXiv:2312.00752](#)

> Why this is a simulation first

Study path

1. Compare model families in the Deep Learning Workbench.
2. Read the original VMamba concepts.
3. Run the SS2D-style lineage demo.
4. Toggle adapter changes.
5. Train a model adapter on an Isomera benchmark and save a `.pk1`.
6. Open Model Interpretability after a run to inspect CO-ES, saliency and trace artifacts.
7. Reproduce article evidence through Benchmark & Examples.

Deep Learning Workbench

Compare model families over the same benchmark scenario and keep the result compatible with the Isomera benchmark contract.

Model family
Vanilla VMamba baseline

Workbench contract
Every runnable model must expose `(predict, pairs(graph))`. The app records operational trace data: parameters, files, model artifact, metrics and comparison status.

Benchmark
TPC-DS (default)

Scenario
graph_SOR16_D1_seed42

Runs
1

Duplicate pair
SOT_catalog_sales_D1 <-> SOT_time_sales_D1

Model A
Vanilla VMamba baseline

Model B
VMamba-Mesh Isomera adapter

Scenario graph

Selected pair features

feature	value
canonsort_route_alignment	1
child_signature_jaccard	1
context_edge_similarity	0.75
context_node_similarity	0.8
diag_fingerprint_similarity	0.725
domain_hist_jaccard	1
exact_child_jaccard	0.3333
exact_parent_jaccard	0.5
hierarchical_context_match	1
layer_hist_jaccard	1

Compare selected models

Figure 18.9: Ficha visual: Study Lab: Deep Learning Workbench

18.10 Study Lab: Original VMamba

Onde clicar: Study Lab → Original VMamba.

Quando usar. Use para explicar a inspiração da arquitetura antes de falar das mudanças Mesh.

O que observar na tela. Observe pseudocódigo, partes da arquitetura e tabela com patch embedding, VSS block, SS2D e head.

Teoria ou algoritmo por trás. VMamba lê mapas 2D com blocos visuais de espaço de estados. Isomera adapta a ideia para matrizes de linhagem.

Checklist de validação. Confirmar que a explicação diferencia inspiração arquitetural de dependência do runtime oficial.

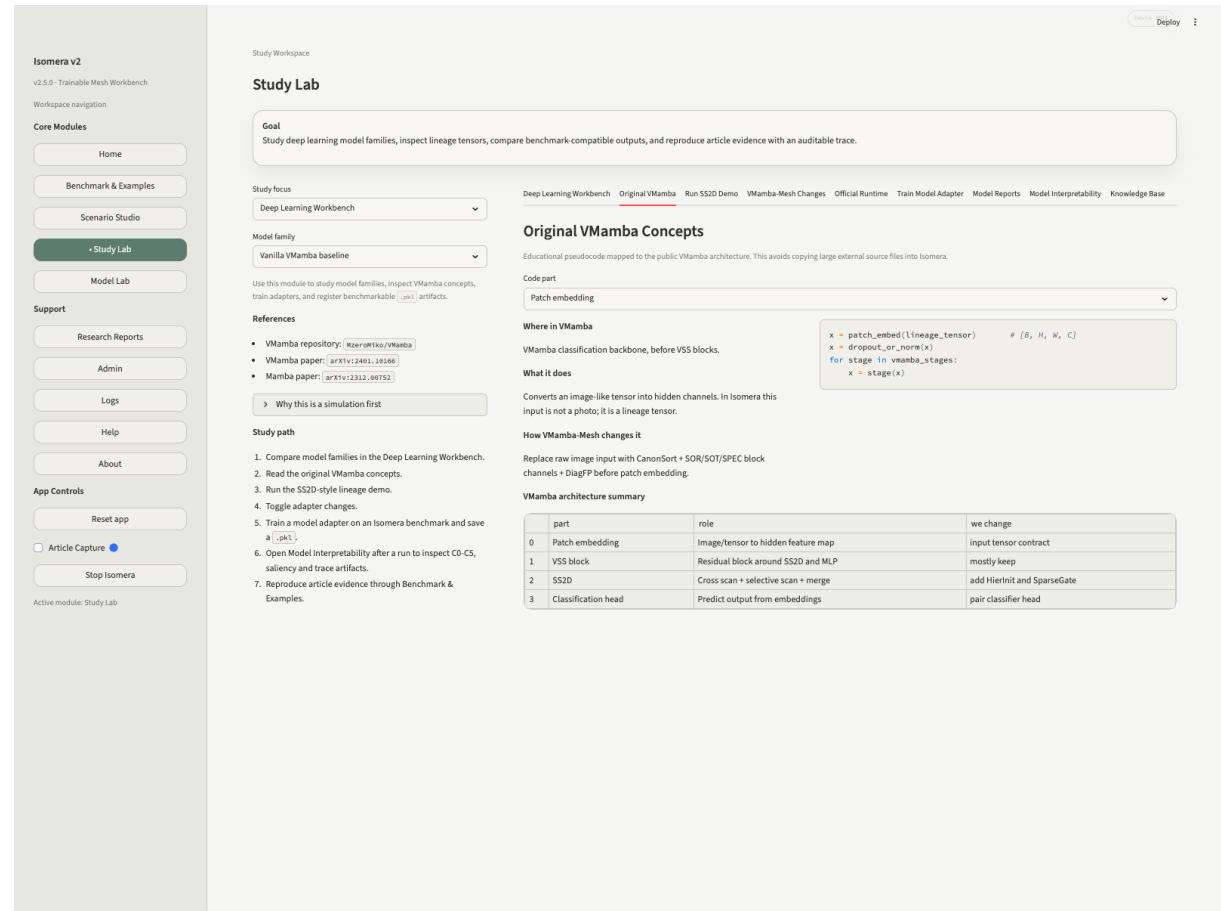


Figure 18.10: Ficha visual: Study Lab: Original VMamba

18.11 Study Lab: SS2D Demo

Onde clicar: Study Lab → Run SS2D Demo.

Quando usar. Use para mostrar como uma matriz pode ser percorrida por rotas de scan.

O que observar na tela. Observe controles de domínio, SOR por domínio, resolução, rota, decaimento, ganho e SparseGate.

Teoria ou algoritmo por trás. SS2D propaga contexto por rotas 2D. No Isomera, essas rotas são aplicadas sobre matriz/tensor de linhagem.

Checklist de validação. Alterar rota ou SparseGate e observar mudança em métricas e heatmaps.

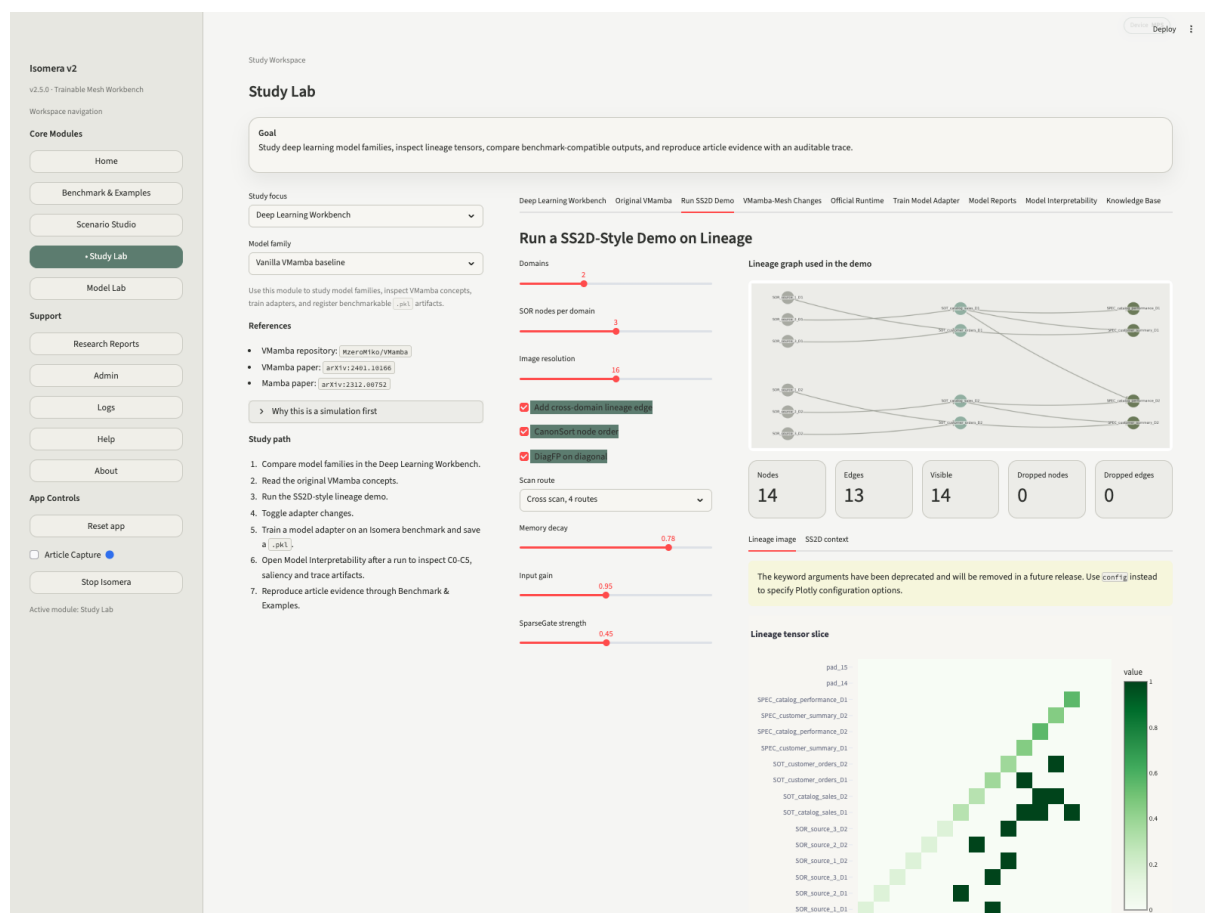


Figure 18.11: Ficha visual: Study Lab: SS2D Demo

18.12 Study Lab: mudanças VMamba-Mesh

Onde clicar: Study Lab → VMamba-Mesh Changes.

Quando usar. Use para explicar exatamente o que foi adaptado em relação ao VMamba original.

O que observar na tela. Observe seletor de mudança, toggles de CanonSort, DiagFP, HierInit, SparseGate e comparação de métricas conceituais.

Teoria ou algoritmo por trás. As mudanças preservam semântica de linhagem: ordem canônica, diagonal informativa, rota hierárquica e tratamento de esparsidade.

Checklist de validação. Ativar/desativar mudanças e observar se contraste entre células ativas e vazias faz sentido.

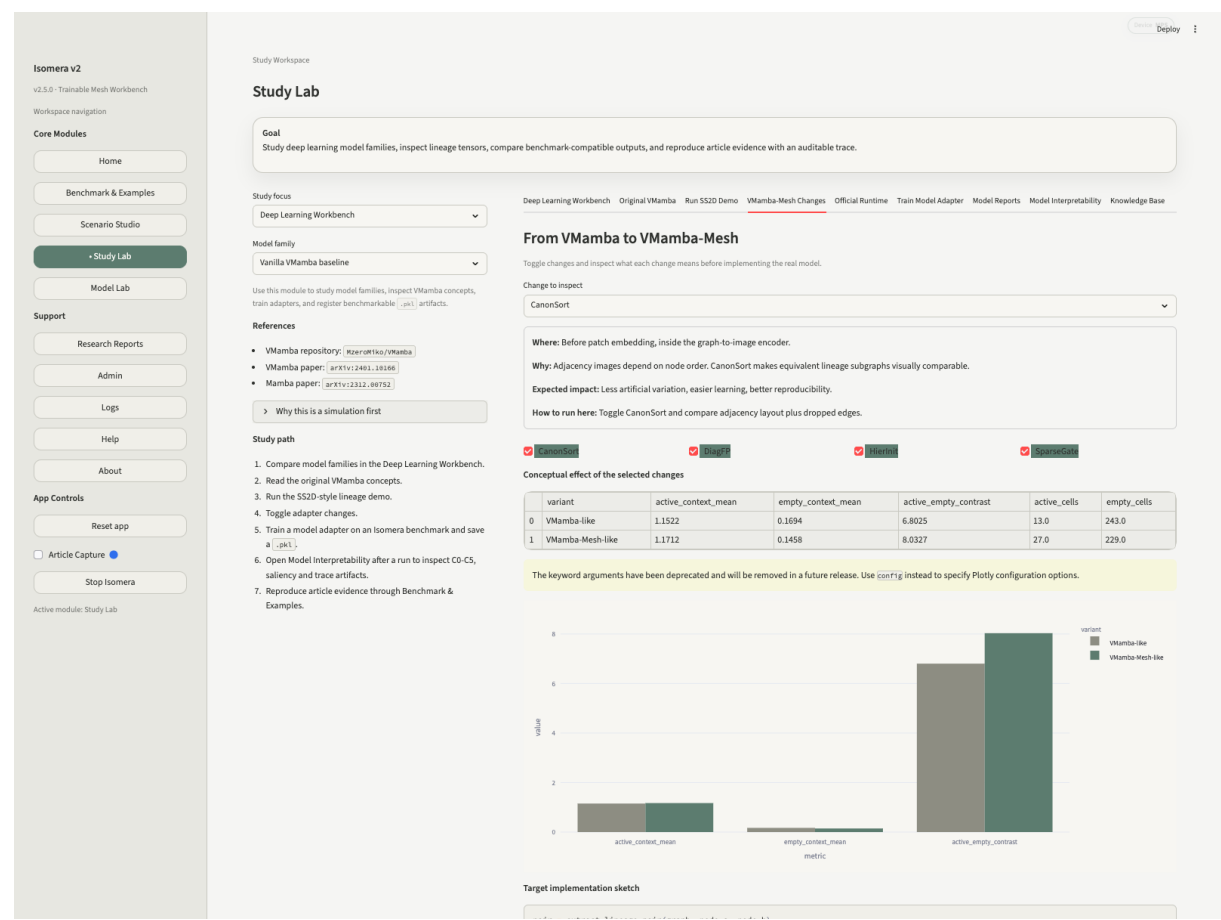


Figure 18.12: Ficha visual: Study Lab: mudanças VMamba-Mesh

18.13 Study Lab: runtime oficial

Onde clicar: Study Lab → Official Runtime.

Quando usar. Use para mostrar onde fica a referência externa do VMamba e como ela se separa do produto Isomera.

O que observar na tela. Observe status do runtime, comandos de instalação e referências.

Teoria ou algoritmo por trás. O runtime oficial orienta estudo da arquitetura original; o Isomera mantém caminho próprio para benchmark reproduzível.

Checklist de validação. Verificar status, referências e explicar que CPU/MPS no Mac não é o mesmo stack CUDA/Triton oficial.

The screenshot displays the Isomera v2 Study Lab interface. On the left is a sidebar with navigation options: Home, Benchmark & Examples, Scenario Studio, Study Lab (selected), Model Lab, Support, Research Reports, Admin, Logs, Help, About, App Controls, and Article Capture. The main content area is titled 'Study Lab' and includes a 'Goal' section, 'Study focus' (Deep Learning Workbench), 'Model family' (Vanilla VMamba baseline), 'References' (VMamba repository, VMamba paper, Mamba paper), and 'Study path' (a 7-step guide). The 'Official VMamba Runtime' section provides operational details for VMamba to VMamba-Mesh, including a table of requirements (1 CanonSort, 2 Pair tensor, 3 DiagFP, 4 Hieffit, 5 SparseGate) and a 'Runtime status' table. The 'Manual install command' section shows a terminal command to clone the repository and install the runtime. A 'References used by the Study Lab' section lists the VMamba repository, paper, and Mamba paper.

Official VMamba Runtime

Optional: clone/check the official VMamba repository for source-level study. The benchmarkable Isomera adapter below does not modify the official code.

VMamba -> VMamba-Mesh operational delta

Official VMamba	VMamba-Mesh
Image -> patch embedding -> VSS blocks -> SS2D cross scan -> classifier.	Lineage graph -> canonical pair tensor -> lineage-aware scan -> duplicate pair head.
Assumption: dense visual texture and spatial locality.	Assumption: sparse SOR -> SOT -> SPEC causality and schema fingerprints.

1 CanonSort	2 Pair tensor	3 DiagFP	4 Hieffit	5 SparseGate
Stable node order.	A, B, diff, masks.	Schema identity.	Upstream memory.	Ignore empty cells.

Runtime status

runtime_root	repository_present	git_present	pip_present	torch_available	torch_version	vmamba_reference_file_present
/Users/cayofel/Documents/GitHub/isomera_v2/main/external/VMamba	False	True	False	True	2.8.0	False

Manual install command

```
mkdir -p /Users/cayofel/Documents/GitHub/isomera_v2/main/external
git clone --depth 1 https://github.com/Mzeorotko/VMamba.git /Users/cayofel/Documents/GitHub/isomera_v2/main/external/VMamba
./Library/Developer/CommandLineTools/usr/bin/python3 -m pip install -r /Users/cayofel/Documents/GitHub/isomera_v2/main/external/VMamba/r
```

Install/Update official VMamba runtime

For the article path, use the official runtime to understand VMamba internals. Use the Train Adapter tab to create a comparable .pt over Isomera scenarios.

References used by the Study Lab

- VMamba repository: <https://github.com/Mzeorotko/VMamba.git>
- VMamba paper: <https://arxiv.org/abs/2401.10166>
- Mamba paper: <https://arxiv.org/abs/2312.00752>

Figure 18.13: Ficha visual: Study Lab: runtime oficial

18.14 Study Lab: treino VMamba

Onde clicar: Study Lab → Train Model Adapter.

Quando usar. Use para treinar ou explicar como treinar VMamba-T e VMamba-Mesh-T.

O que observar na tela. Observe benchmark, cenário, família, positive pairs, tensor resolution, preset, patch size, loss, optimizer, negative ratio, hard negatives e device.

Teoria ou algoritmo por trás. O treino gera um modelo de par: tensor entra, backbone produz embeddings, head gera logit, sigmoid gera score e threshold decide.

Checklist de validação. Conferir hiperparâmetros, aviso de tempo, destino de saída e relatório salvo.

The screenshot shows the 'Study Lab' interface in Isomera v2. The left sidebar contains navigation links for Home, Benchmark & Examples, Scenario Studio, Study Lab (active), Model Lab, Support, Research Reports, Admin, Logs, Help, About, App Controls, and Stop Isomera. The main content area is titled 'Study Lab' and includes a goal, study focus, model family, references, study path, and configuration options for the 'Train Model Adapter'.

Study Lab Goal: Study deep learning model families, inspect lineage tensors, compare benchmark-compatible outputs, and reproduce article evidence with an auditable trace.

Study focus: Deep Learning Workbench

Model family: Vanilla VMamba baseline

References:

- VMamba repository: [hzeronko/VMamba](#)
- VMamba paper: [arXiv:2403.18166](#)
- Mamba paper: [arXiv:2312.00752](#)

Study path:

1. Compare model families in the Deep Learning Workbench.
2. Read the original VMamba concepts.
3. Run the SS2D-style lineage demo.
4. Toggle adapter changes.
5. Train a model adapter on an Isomera benchmark and save a `.pkl`.
6. Open Model Interpretability after a run to inspect CO-C5, saliency and trace artifacts.
7. Reproduce article evidence through Benchmark & Examples.

Train Model Adapter Configuration:

- Benchmark source:** TPC-DS (default)
- Scenario:** graph_SOR16_D1_seed42
- Model implementation:** VMamba-Mesh adapter (deterministic)
- Graph exists:** yes
- Positive pairs:** 17
- Labels file:** yes
- Model version:** vmamba_me...
- Seed:** 42
- Tensor resolution:** 32
- Negative ratio:** 4
- Model artifact name:** VMambaMesh_adapter_graph_SOR16_D1_seed42

Checklist:

- ☒ Register model in Benchmark & Examples
- ☒ Generate study report PDF if Tectonic is available

Buttons: Train and save VMamba-Mesh pickle

Notification: The saved pickle is comparable with VF2, Node Match and GNN clusters because it returns predicted duplicate pairs over the same graph.

Figure 18.14: Ficha visual: Study Lab: treino VMamba

18.15 Study Lab: relatórios de modelo

Onde clicar: Study Lab → Model Reports.

Quando usar. Use para abrir campanhas já executadas sem retreinar durante uma apresentação.

O que observar na tela. Observe seletor de relatório, tabelas, gráficos e manifests disponíveis.

Teoria ou algoritmo por trás. Relatórios separam execução pesada de comunicação. Eles preservam contexto e evidência para revisão posterior.

Checklist de validação. Confirmar que o relatório certo está aberto e que CSV, figuras e manifest correspondem à campanha esperada.

The screenshot displays the Isomera v2 Study Lab interface. On the left is a sidebar with navigation options: Home, Benchmark & Examples, Scenario Studio, Study Lab (selected), Model Lab, Support, Research Reports, Admin, Logs, Help, About, App Controls, Reset app, Article Capture, Stop Isomera, and Active module: Study Lab. The main area is titled 'Study Lab' and contains a 'Goal' section with the text: 'Study deep learning model families, inspect lineage tensors, compare benchmark-compatible outputs, and reproduce article evidence with an auditable trace.' Below this is a 'Study focus' dropdown set to 'Deep Learning Workbench' and a 'Model family' dropdown set to 'Vanilla VMamba baseline'. A 'References' section lists three items: 'VMamba repository', 'VMamba paper', and 'Mamba paper'. A 'Study path' section lists seven steps. The 'Model Reports' section is active, showing a 'Run trainable VMamba ablation campaign' button. Below this is a 'Trainable VMamba ablation reports' section with a dropdown for '20260604_152340_43472_95825000_vmamba_trainable_ablation'. A 'Neural processing contract' section follows, explaining the auditable path and providing a 'Best configuration by SF-Jaccard' JSON object.

```

{
  "benchmark": "tpc_ds_genai_spec_v2",
  "scenario": "graph_SOR8_DS_seed42",
  "algorithm": "VMamba-Mesh-T (v3-fpreplay-mps-spec)",
  "family": "VMamba-Mesh-T (v3-fpreplay-mps-spec)",
  "family_base": "VMamba-Mesh-T",
  "variant": "vmamba_mesh_t",
  "experiment_tag": "v3-fpreplay-mps-spec",
  "channel_contract": [
    [{"channel": "C0", "name": "Forward adjacency", "role": "edges in canonical lineage direction"}, {"channel": "C1", "name": "Reverse adjacency", "role": "reverse edges for bidirectional scan context"}, {"channel": "C2", "name": "Layer diagonal", "role": "SOR/SOT/SPEC layer identity on the diagonal"}, {"channel": "C3", "name": "Degree fingerprint", "role": "local structural degree signature on the diagonal"}, {"channel": "C4", "name": "Lineage route bias", "role": "route prior for SOR -> SOT -> SPEC traversal"}, {"channel": "C5", "name": "Sparse mask", "role": "occupied cells so sparse zeros are not treated as missing evidence"}],
  "preset": "article_cpu",
  "resolution": 16,
  "patch_size": 2,
  "depths": "12-2-9-2",
  "dims": "16-32-64-128",
  "hidden_dim": 128,
  "embedding_dim": 128,
  "epochs": 10,
  "batch_size": 16,
  "inference_batch_size": 4096,
  "encoder_batch_size": 64
}

```

Figure 18.15: Ficha visual: Study Lab: relatórios de modelo

18.16 Study Lab: interpretabilidade

Onde clicar: Study Lab → Model Interpretability.

Quando usar. Use para explicar uma decisão de par com grafo, matriz, canais e saliency.

O que observar na tela. Observe seletores de benchmark, cenário, par, modelo e artefato.

Teoria ou algoritmo por trás. A interpretabilidade usa sensibilidade local do score em relação ao tensor de entrada. Ela mostra onde a decisão foi mais sensível.

Checklist de validação. Escolher par conhecido, gerar figuras e verificar se saliency, matriz e canais aparecem juntos.

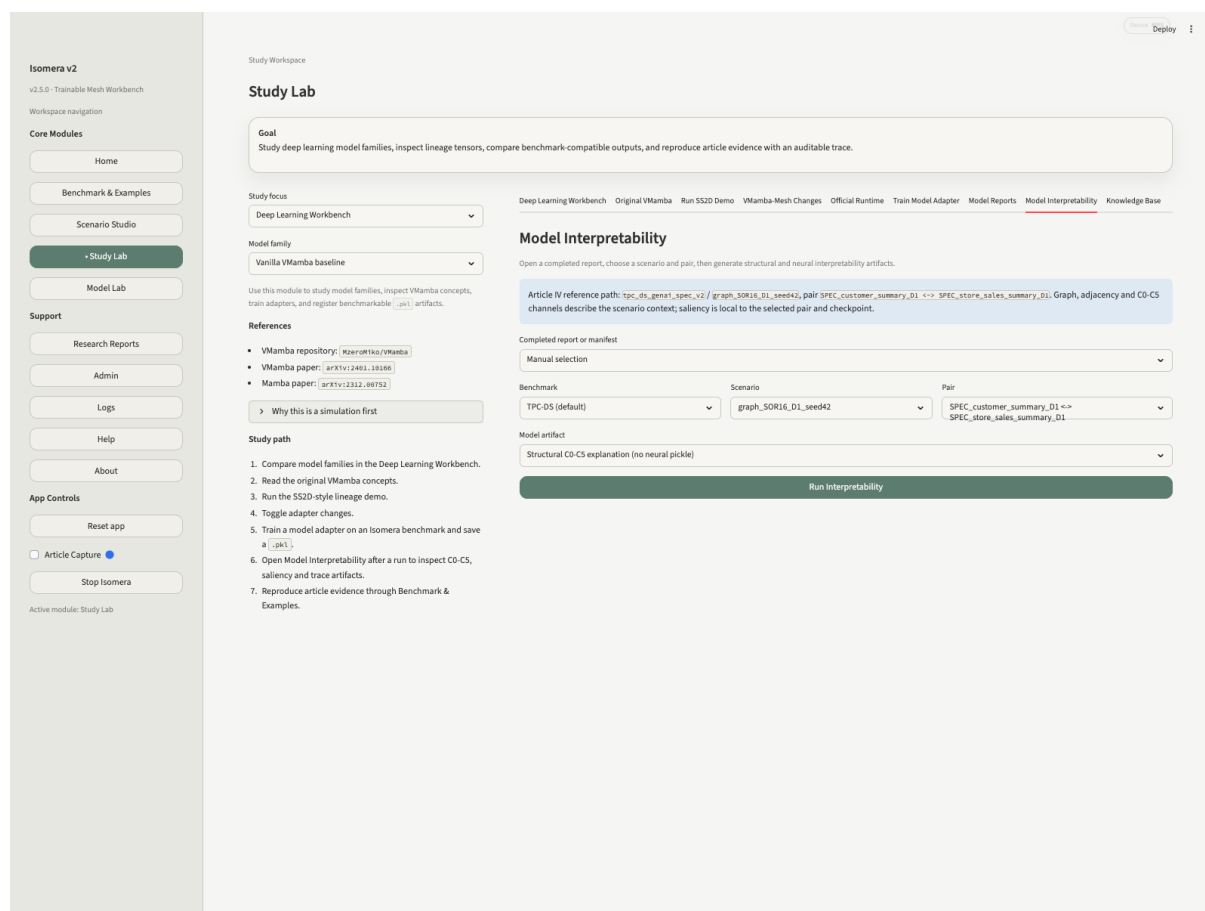


Figure 18.16: Ficha visual: Study Lab: interpretabilidade

18.17 Study Lab: Knowledge Base

Onde clicar: Study Lab → Knowledge Base.

Quando usar. Use para estudar teoria dentro do próprio aplicativo.

O que observar na tela. Observe tópicos de fundação, encoding, reprodutibilidade, interpretabilidade e runbook.

Teoria ou algoritmo por trás. A Knowledge Base conecta documentação local com telas de execução, reduzindo dependência de PDFs externos.

Checklist de validação. Abrir cada KB e confirmar que os caminhos apontam para arquivos existentes.

Isomera v2
v2.5.0 - Trainable Mesh Workbench

Workspace navigation

Core Modules

- Home
- Benchmark & Examples
- Scenario Studio
- Study Lab**
- Model Lab

Support

- Research Reports
- Admin
- Logs
- Help
- About

App Controls

- Reset app
- ☐ Article Capture ☒
- Stop Isomera

Active module: Study Lab

Study Lab

Goal
Study deep learning model families, inspect lineage tensors, compare benchmark-compatible outputs, and reproduce article evidence with an auditable trace.

Study focus
Deep Learning Workbench

Model family
Vanilla VMamba baseline

Use this module to study model families, inspect VMamba concepts, train adapters, and register benchmarkable artifacts.

References

- VMamba repository: [github.com/haotian-liu/VMamba](#)
- VMamba paper: [arXiv:2403.12016](#)
- Mamba paper: [arXiv:2312.00752](#)

> Why this is a simulation first

Study path

1. Compare model families in the Deep Learning Workbench.
2. Read the original VMamba concepts.
3. Run the SS2D-style lineage demo.
4. Toggle adapter changes.
5. Train a model adapter on an Isomera benchmark and save artifacts.
6. Open Model Interpretability after a run to inspect CO-C5, saliency and trace artifacts.
7. Reproduce article evidence through Benchmark & Examples.

Knowledge Base
Curated local KBs used by the Deep Learning Workbench and article reproducibility flow.

topic	path	available	summary
VMamba Foundation	github/knowledge_bases/vmamba_foundation.md	☒	Mamba, VMamba, SS2D, VSSM, cross scan and forward types.
VMamba-Mesh Encoding	github/knowledge_bases/vmamba_mesh_encoding.md	☒	CanonSort, tensorization, DiagFFT, MeshSS2D, HierInit and SparseGate.
Reproducibility	github/knowledge_bases/vmamba_mesh_reproducibility.md	☒	Benchmarks, article evidence, metrics and expected values.
Interpretability	github/knowledge_bases/vmamba_mesh_interpretability.md	☒	ERF, SS2D routes, tensor channels and sensitivity views.
VMamba-T Video Runbook	github/knowledge_bases/vmamba_trainable_video_runbook.md	☒	Validated trainable reports, hyperparameters, video steps and pending GPU campaign.

Open KB
VMamba Foundation

VMamba Foundation

1. Definicoes

Mamba é uma família de modelos de espaço de estados seletivos. A ideia central é manter uma memória recorrente eficiente em sequências longas, usando parâmetros dependentes da entrada.

VMamba leva essa ideia para visao computacional. Como uma imagem é bidimensional, o modulo SS2D converte mapas 2D em rotas 1D, aplica selective scan e depois reconstrói a representacao espacial.

2. Conceitos Principais

- `VSSM`: Visual State-Space Model, o backbone visual do VMamba.
- `VSS block`: bloco residual que envolve normalizacao, SS2D e MLP.
- `SS2D`: rota 2D para 1D, selective scan e merge.
- `cross_scan`: cria rotas por linhas e colunas, nos sentidos direto e reverso.
- `cross_merge`: recombina as rotas em uma representacao 2D.
- `forward_type`: variantes internas do VMamba, como `v04`, `v05`, `v05id` e `v052d`.

3. Evidencia Local

Os notebooks locais validaram que o código oficial pode ser estudado sem alterar o clone upstream:

- [research/vmamba/notebooks/vmamba_local_sourceSmoke.py](#)
- [research/vmamba/notebooks/vmamba_official_source_walkthrough_isomera_case.py](#)

Resultados registrados:

- `cross_scan` recebeu entrada `[1, 1, 4, 4]` e produziu `[1, 4, 1, 16]`.

Figure 18.17: Ficha visual: Study Lab: Knowledge Base

18.18 Model Lab

Onde clicar: Model Lab.

Quando usar. Use para inspecionar pickles, famílias e roteamento por benchmark.

O que observar na tela. Observe lista de artefatos, tamanhos, status, cobertura e tabela de roteamento.

Teoria ou algoritmo por trás. Model Lab é o registro de modelos. Ele liga artefatos a benchmarks e evita rodar modelo fora do escopo.

Checklist de validação. Validar pickle, conferir família, verificar cobertura e confirmar política de roteamento.

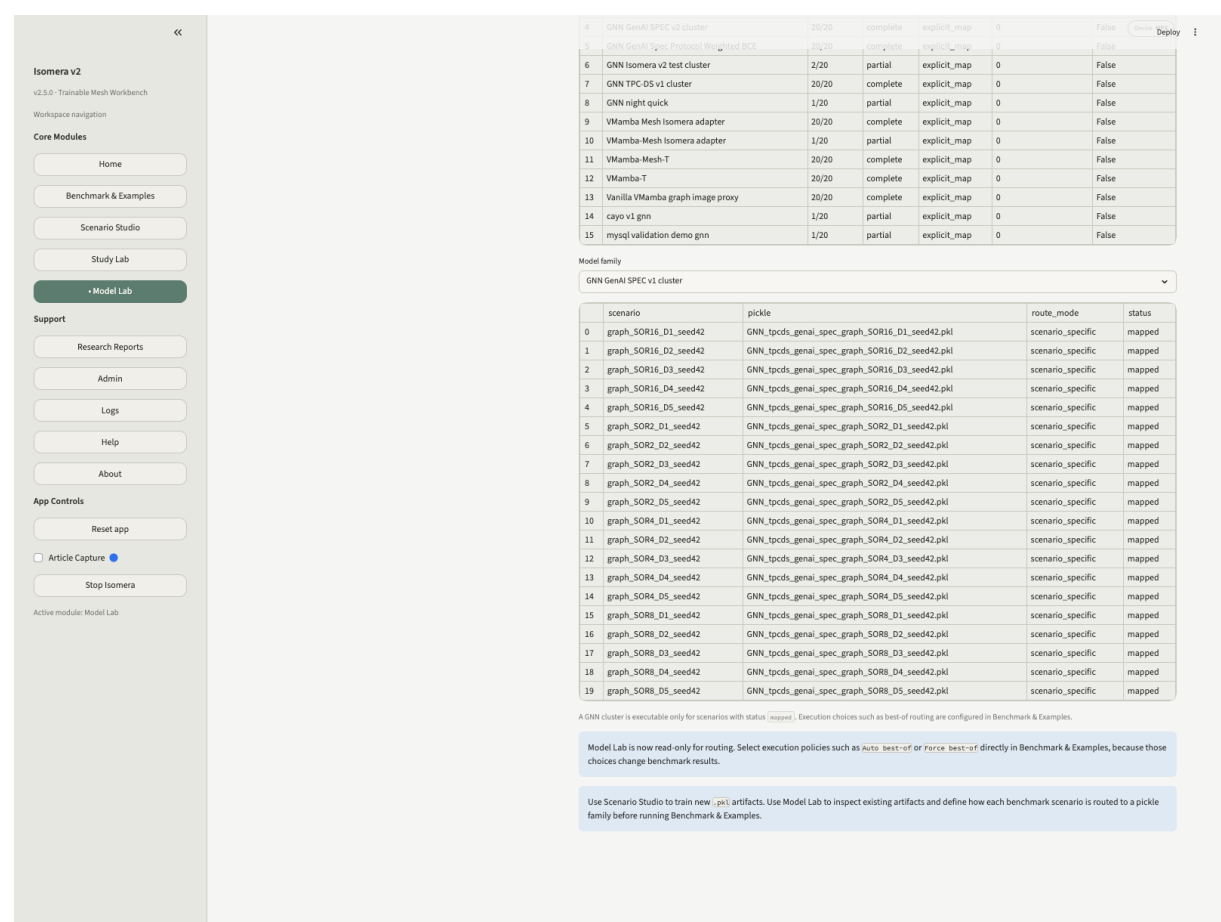


Figure 18.18: Ficha visual: Model Lab

18.19 Research Reports

Onde clicar: Research Reports.

- Quando usar.** Use para abrir resultados, pacotes, anexos e manifests de execuções.
- O que observar na tela.** Observe seletor de pacote, viewer Markdown, arquivos brutos e downloads.
- Teoria ou algoritmo por trás.** Research Reports preserva a evidência. Um resultado deve ser acompanhado por narrativa, CSV, JSON, figuras e manifest.
- Checklist de validação.** Abrir pacote correto, conferir arquivos e baixar ZIP quando necessário.

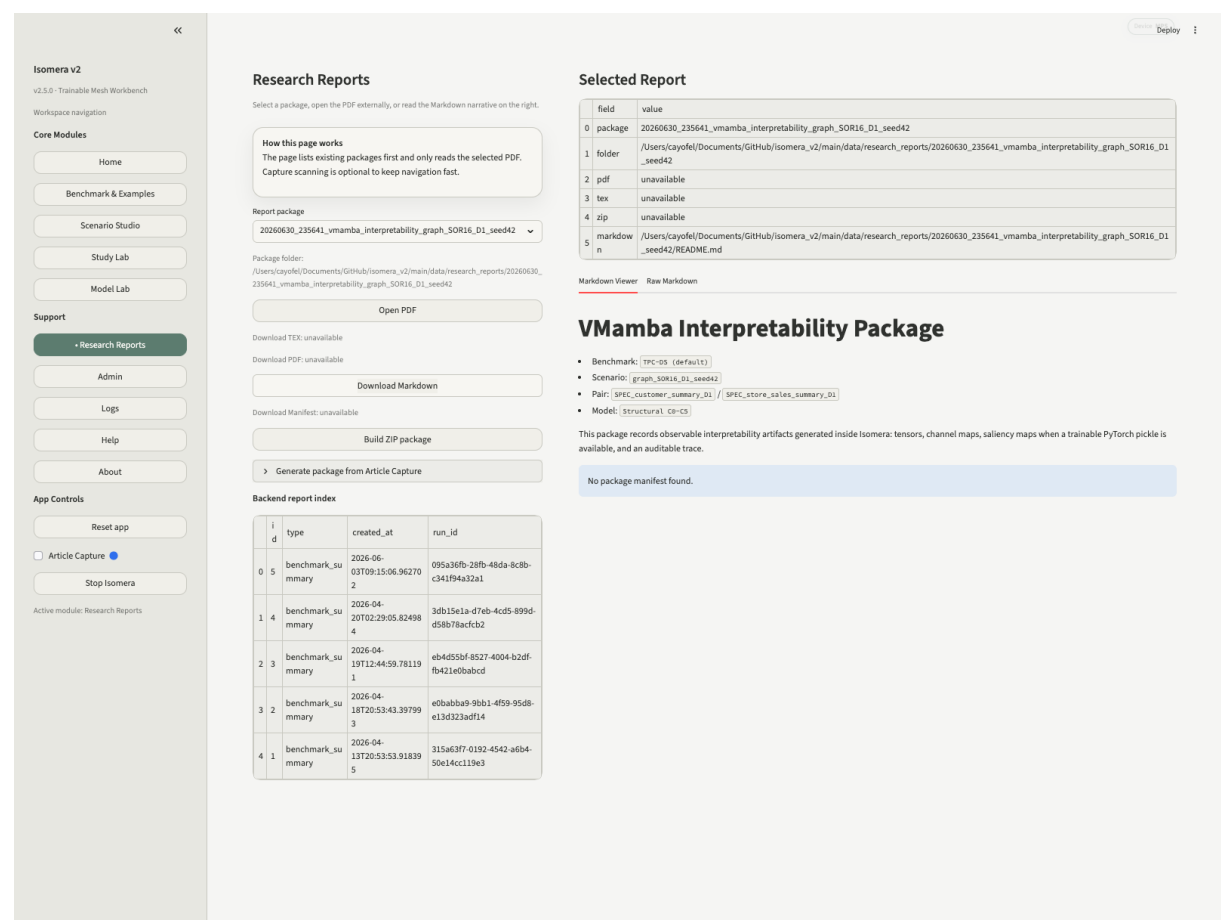


Figure 18.19: Ficha visual: Research Reports

18.20 Admin: Overview

Onde clicar: Admin → Overview.

Quando usar. Use para verificar o estado geral técnico do Isomera.

O que observar na tela. Observe contadores de stores, cenários, runs e entidades do backend.

Teoria ou algoritmo por trás. Admin separa store de produto, warehouse de cenários e configurações. Isso facilita depuração.

Checklist de validação. Conferir se stores estão acessíveis e se os contadores fazem sentido.

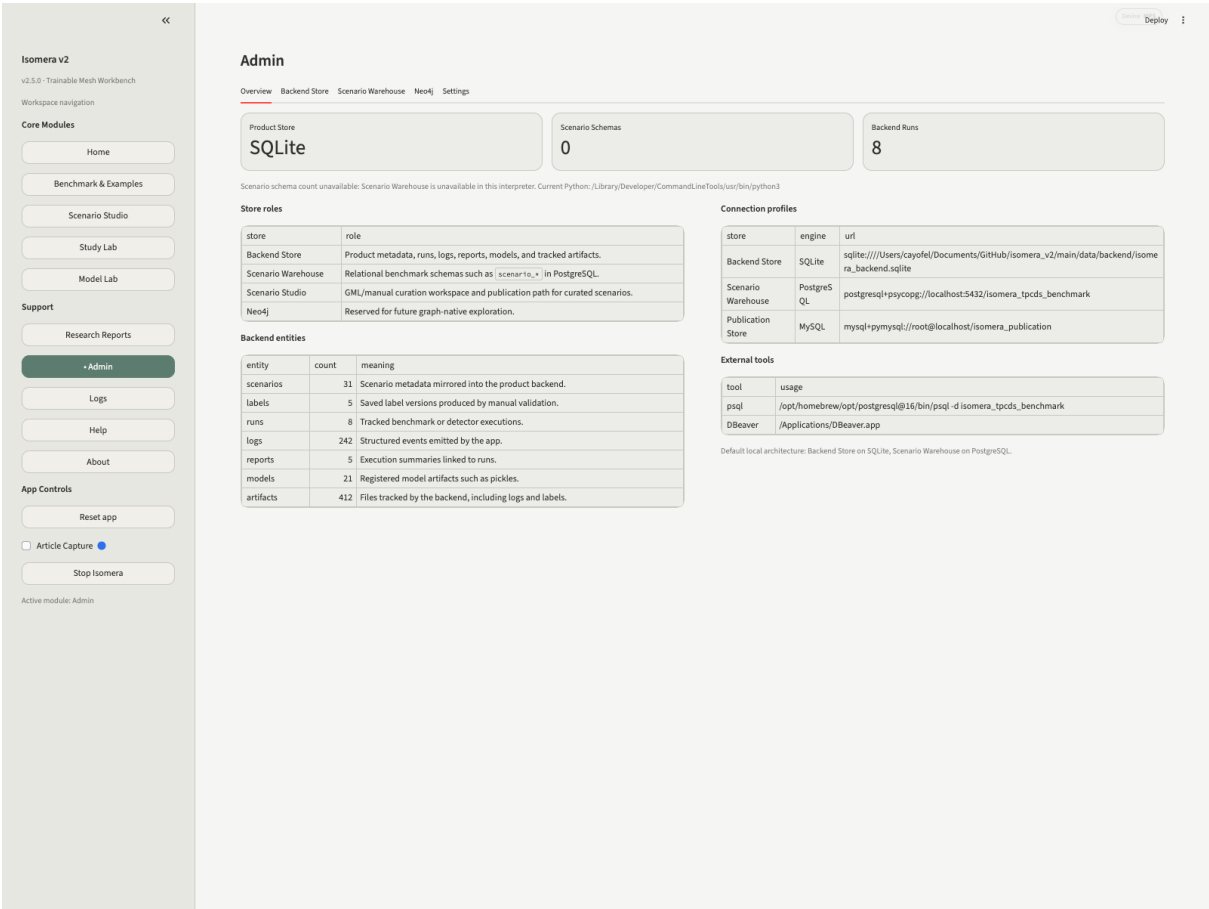


Figure 18.20: Ficha visual: Admin: Overview

18.21 Admin: Backend Store

Onde clicar: Admin → Backend Store.

Quando usar. Use para auditar metadados de produto.

O que observar na tela. Observe tabelas de cenários, labels, runs, logs, reports, modelos e artefatos.

Teoria ou algoritmo por trás. O backend guarda rastreabilidade da ferramenta. Ele não substitui o warehouse; ele registra a operação do produto.

Checklist de validação. Verificar runs recentes, modelos registrados e relatórios vinculados.

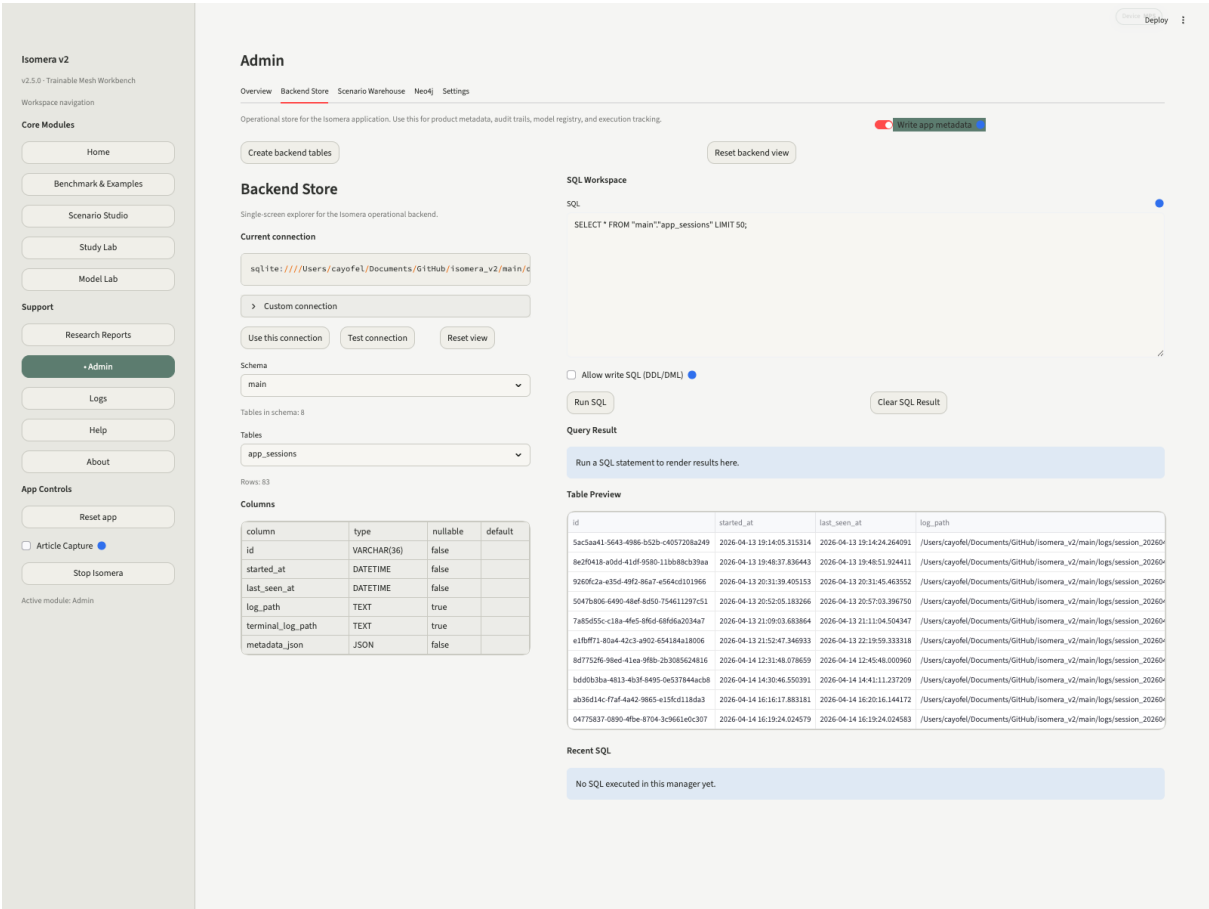


Figure 18.21: Ficha visual: Admin: Backend Store

18.22 Admin: Scenario Warehouse

Onde clicar: Admin → Scenario Warehouse.

Quando usar. Use para verificar schemas e materializações de cenário.

O que observar na tela. Observe status do banco, schemas disponíveis e relação com Scenario Studio.

Teoria ou algoritmo por trás. O warehouse é a ponte relacional: tabelas e schemas viram grafos de benchmark.

Checklist de validação. Confirmar conexão, schemas esperados e consistência com cenários listados.

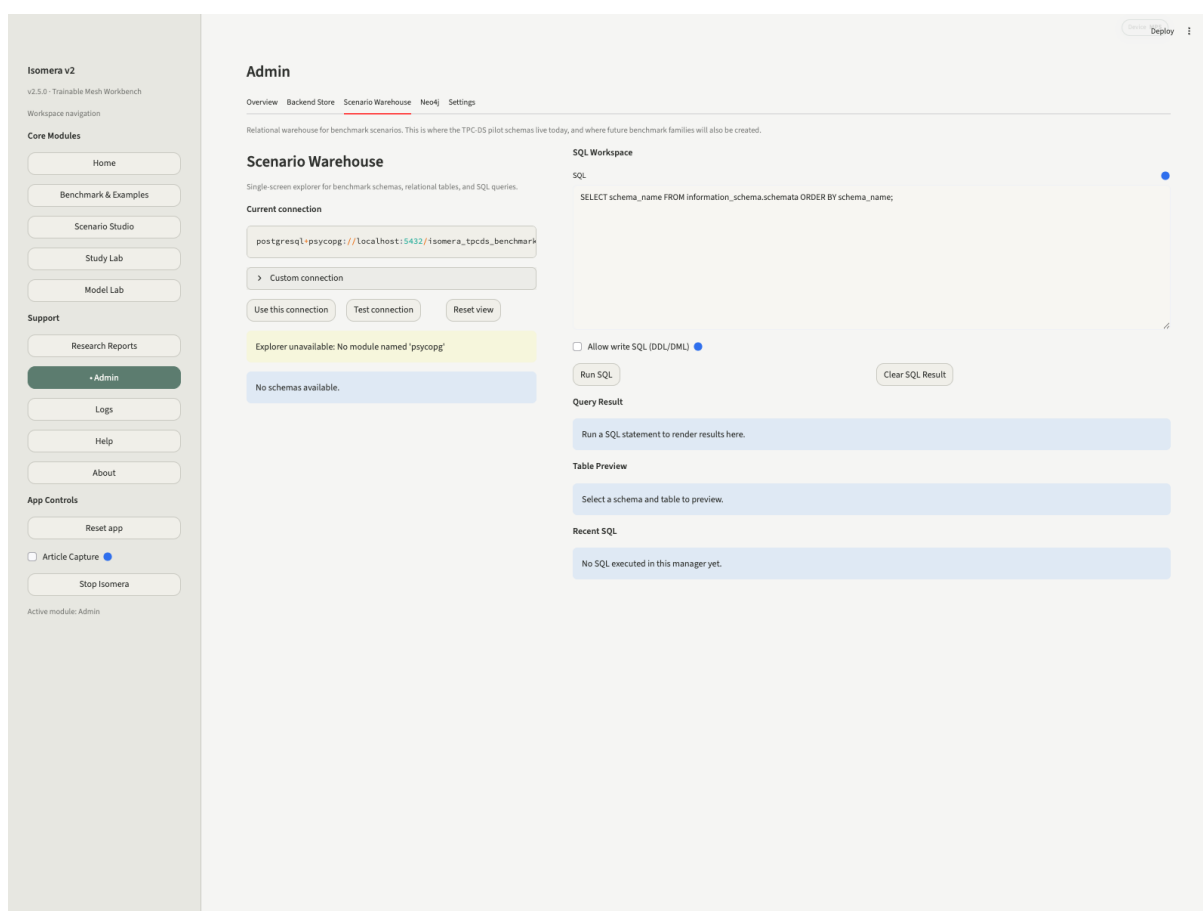


Figure 18.22: Ficha visual: Admin: Scenario Warehouse

18.23 Admin: Neo4j

Onde clicar: Admin → Neo4j.

Quando usar. Use para entender a direção graph-native da ferramenta.

O que observar na tela. Observe status da integração e mensagens de disponibilidade.

Teoria ou algoritmo por trás. Neo4j é uma evolução natural para consultas de caminhos, vizinhanças e exploração graph-native.

Checklist de validação. Verificar se a tela deixa claro se a integração está ativa ou reservada.

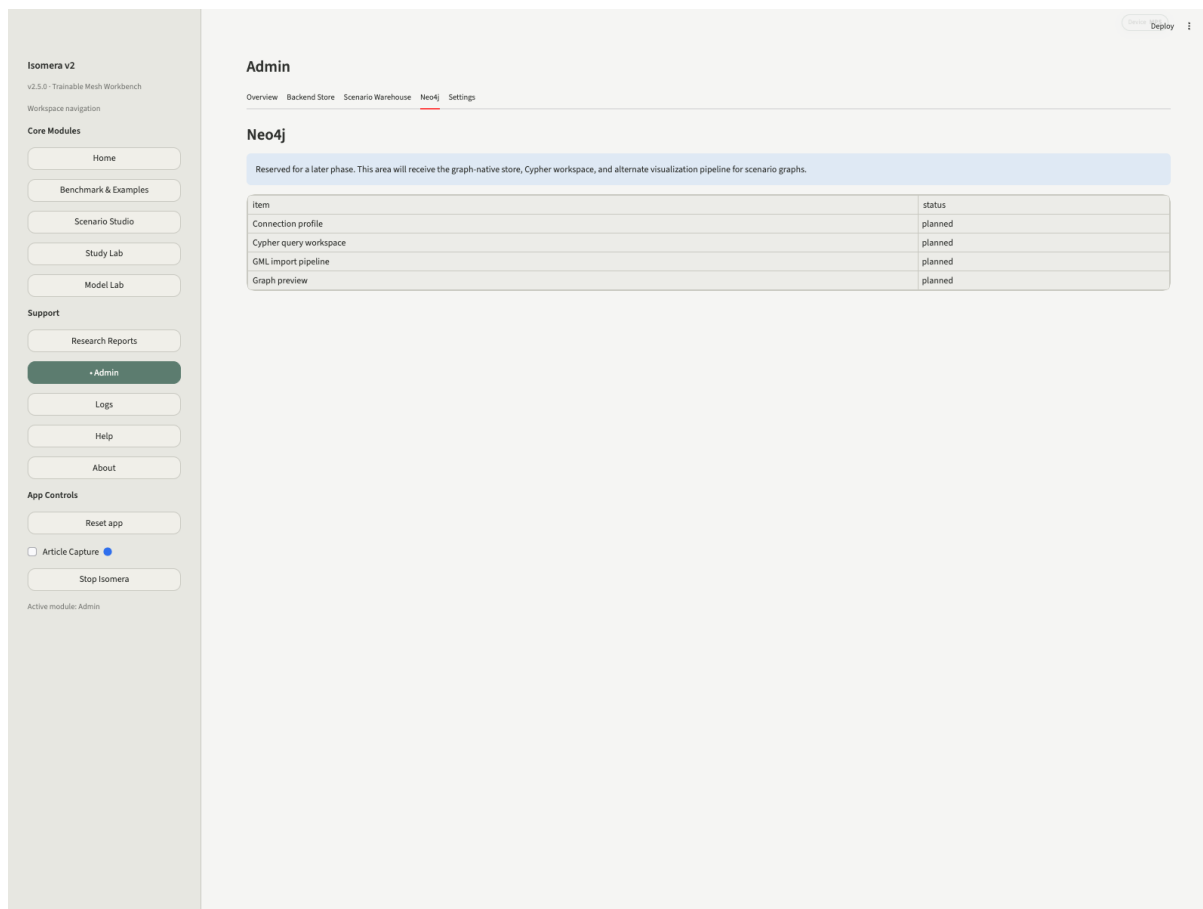


Figure 18.23: Ficha visual: Admin: Neo4j

18.24 Admin: Settings

Onde clicar: Admin → Settings.

Quando usar. Use para revisar conexões, stores e perfis de runtime.

O que observar na tela. Observe URLs, configurações e campos editáveis.

Teoria ou algoritmo por trás. Settings torna o Isomera portátil entre ambiente local, laboratório e possível instalação corporativa.

Checklist de validação. Conferir variáveis sensíveis, conexões e se alterações exigem reinício.

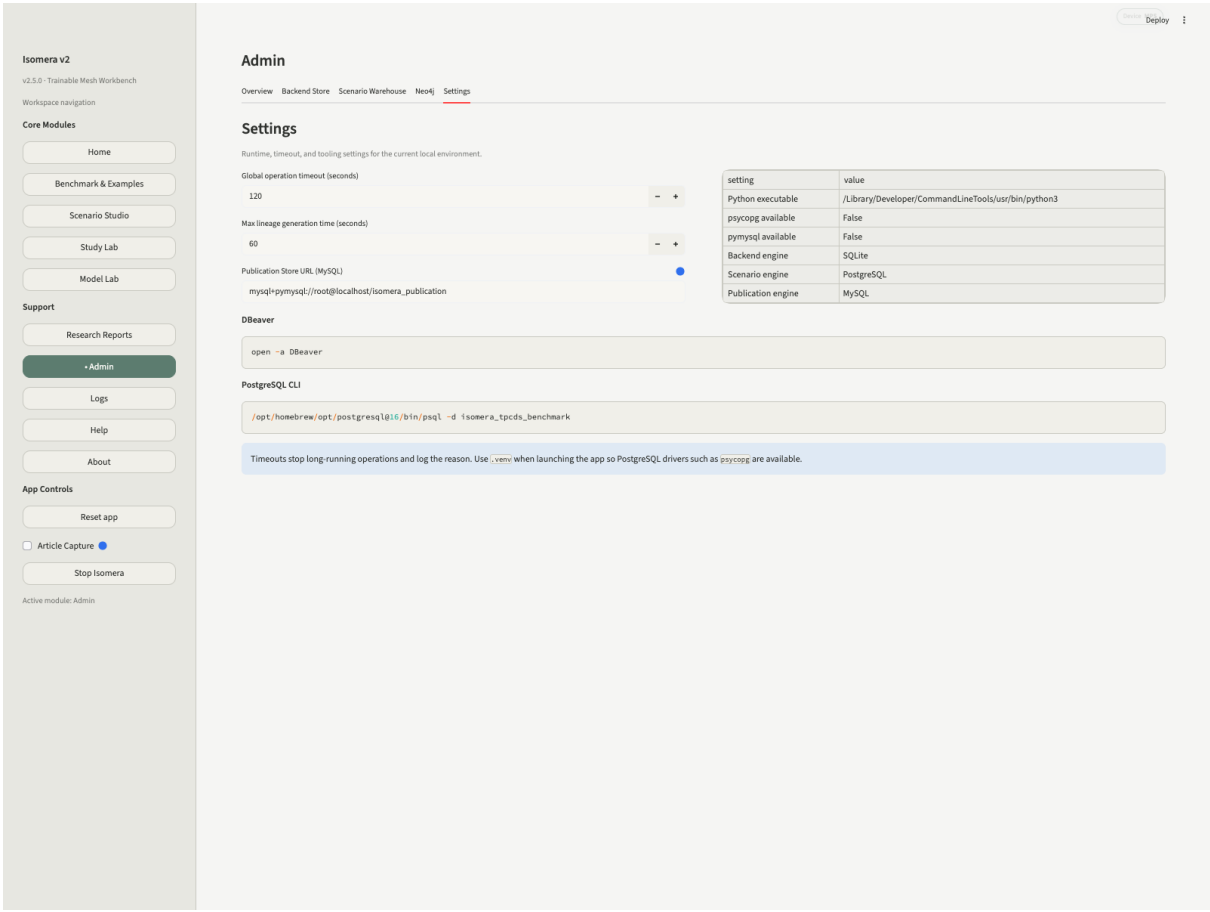


Figure 18.24: Ficha visual: Admin: Settings

18.25 Logs: Session Log

Onde clicar: Logs → Session Log.

Quando usar. Use para auditar ações do app.

O que observar na tela. Observe eventos, timestamps, payloads e registros estruturados.

Teoria ou algoritmo por trás. O Session Log registra a sequência de ações de interface e execução, ajudando a reproduzir ou depurar fluxo.

Checklist de validação. Procurar evento esperado, payload coerente e ausência de erros silenciosos.

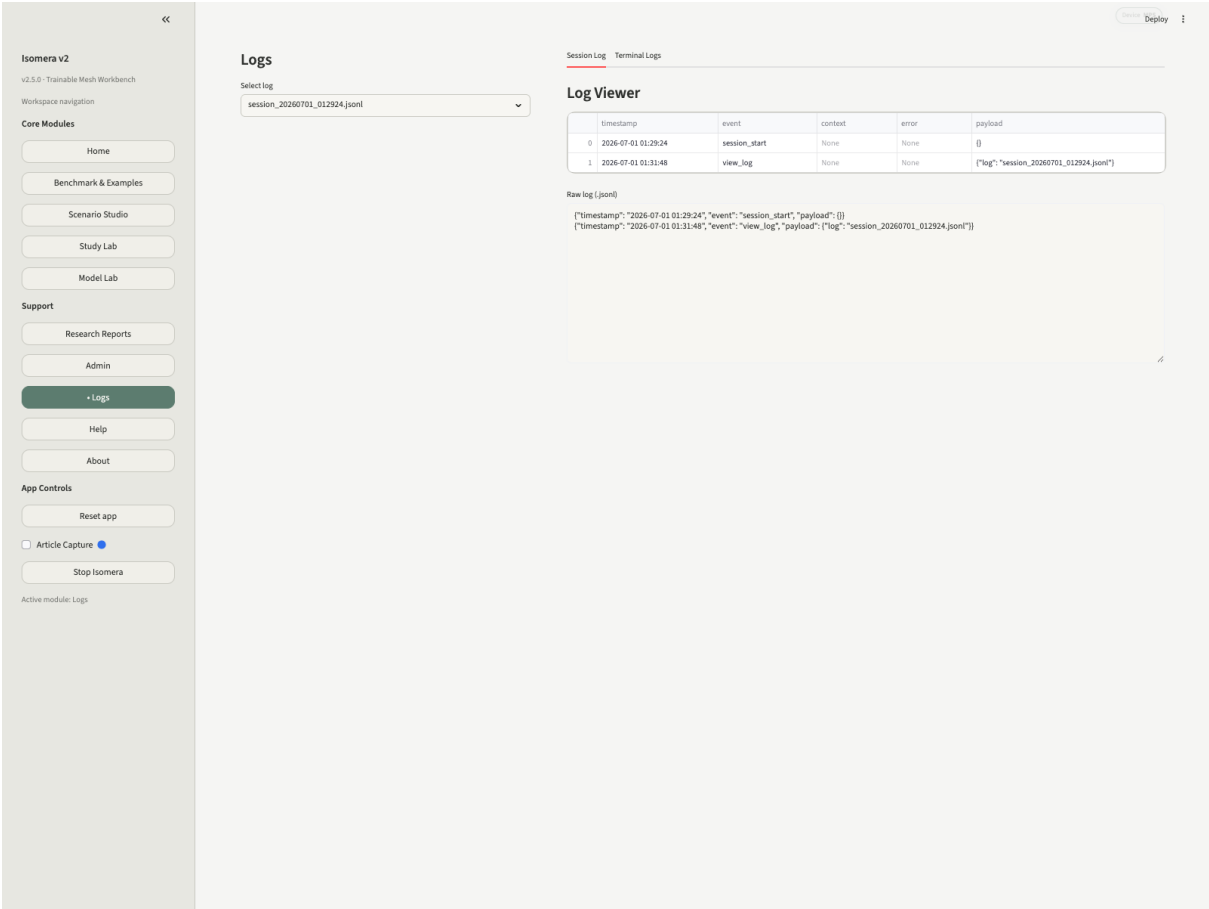


Figure 18.25: Ficha visual: Logs: Session Log

18.26 Logs: Terminal Logs

Onde clicar: Logs → Terminal Logs.

Quando usar. Use quando o erro precisa ser lido no nível de processo.

O que observar na tela. Observe saída bruta de terminal, warnings, traceback e logs de launcher.

Teoria ou algoritmo por trás. Terminal Logs complementam o log estruturado; são úteis para dependências, falhas de import e processos longos.

Checklist de validação. Verificar mensagens recentes, dependências ausentes e fim correto de execução.

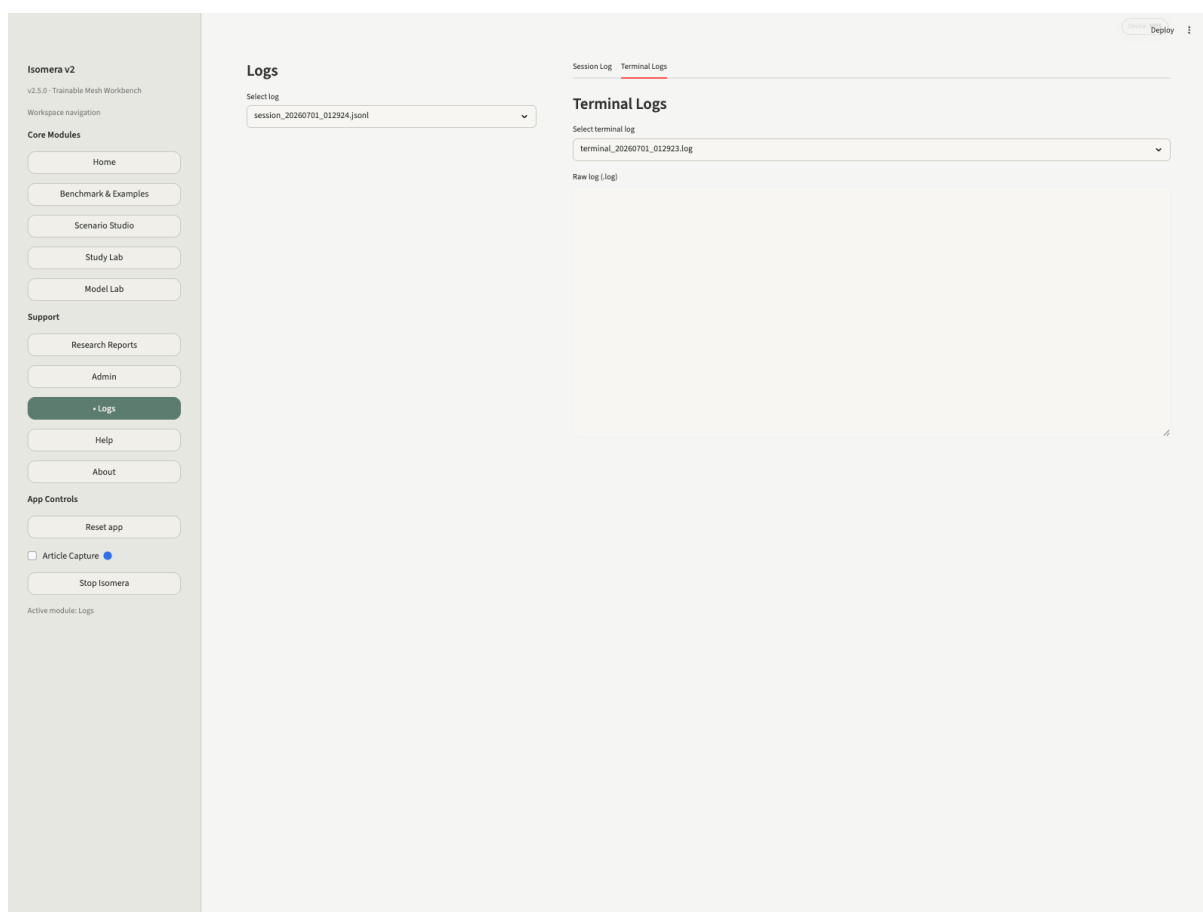


Figure 18.26: Ficha visual: Logs: Terminal Logs

18.27 Help: Presentation

Onde clicar: Help → VMamba-Mesh Presentation.

Quando usar. Use para apresentação guiada ou ensaio de vídeo.

O que observar na tela. Observe roteiro, imagens e botões que levam para Study Lab, Benchmark e Reports.

Teoria ou algoritmo por trás. A apresentação embutida conecta narrativa e ferramenta, evitando trocar entre muitos arquivos durante a fala.

Checklist de validação. Confirmar se imagens carregam e se botões de navegação levam às rotas corretas.

The screenshot shows the 'VMamba-Mesh Presentation' interface within the Isomera v2 application. The sidebar on the left contains navigation links for Home, Benchmark & Examples, Scenario Studio, Study Lab, Model Lab, Research Reports, Admin, Logs, Help (highlighted), and About. The main content area is titled 'VMamba-Mesh Presentation' and includes a brief description, a central message, a presentation strategy, and a download button for the HTML presentation. It also features live demo switches for 'Open Deep Learning Workbench', 'Open Article Reproducibility', and 'Open Research Reports'. A 'Context' section provides background information on the problem and context, and a 'Problema' section explains the challenge of Data Mesh architectures. A diagram at the bottom illustrates the 'Isomera pipeline: graph materialization, model execution, reproducible reports'.

Figure 18.27: Ficha visual: Help: Presentation

18.28 Help: Tech Docs

Onde clicar: Help → Tech Docs.

Quando usar. Use para abrir documentação técnica Markdown dentro do app.

O que observar na tela. Observe lista de documentos, viewer e caminhos de origem.

Teoria ou algoritmo por trás. Tech Docs mantém arquitetura, algoritmos e notas próximas da execução.

Checklist de validação. Abrir documentos principais e verificar se o conteúdo aparece sem links quebrados.

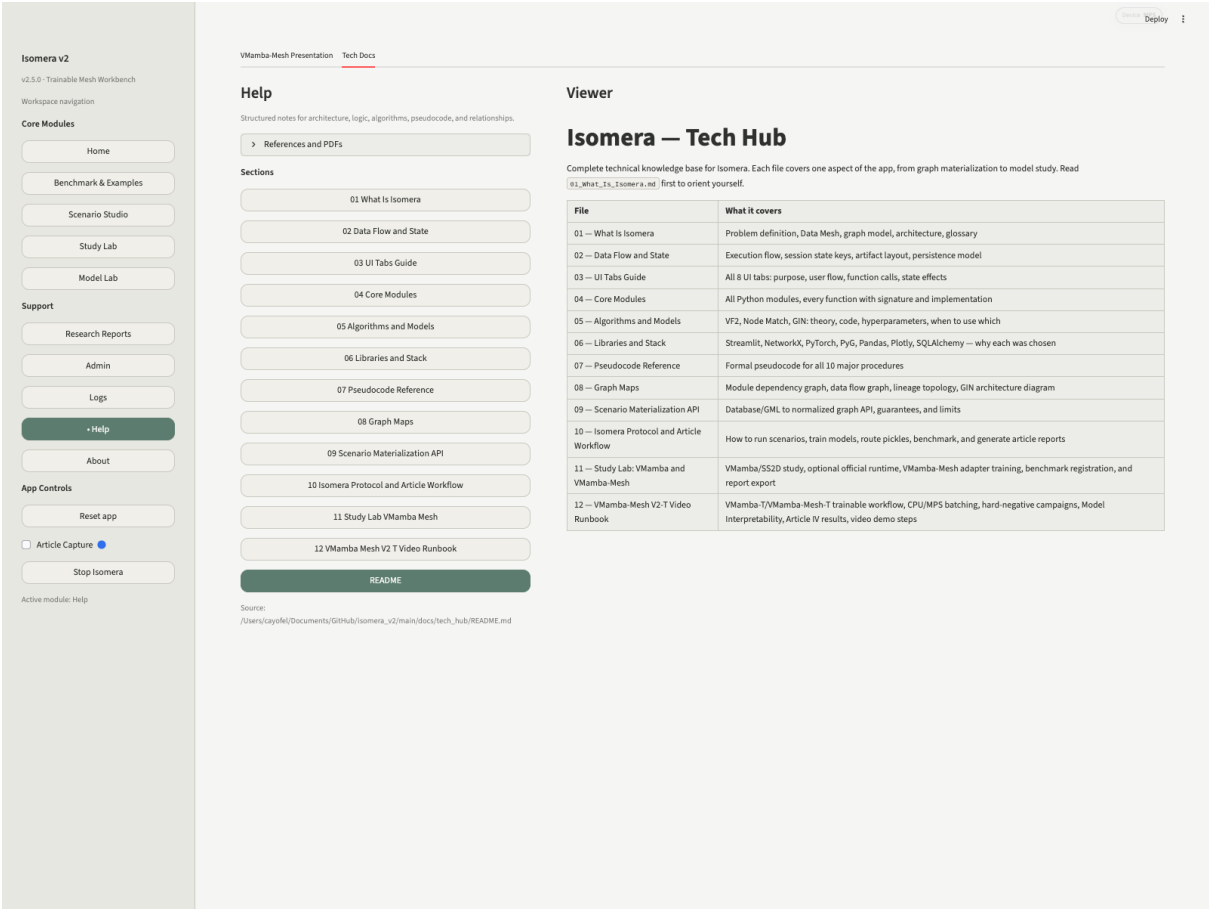


Figure 18.28: Ficha visual: Help: Tech Docs

18.29 About

Onde clicar: About.

Quando usar. Use para identificar versão, autoria e contexto do projeto.

O que observar na tela. Observe versão, codinome, release date, autores, links e notas de funcionalidades.

Teoria ou algoritmo por trás. About é parte da rastreabilidade. Em gravações e entregas, ele confirma qual build está sendo mostrado.

Checklist de validação. Conferir versão, autores, contatos e notas antes de publicar ou gravar.

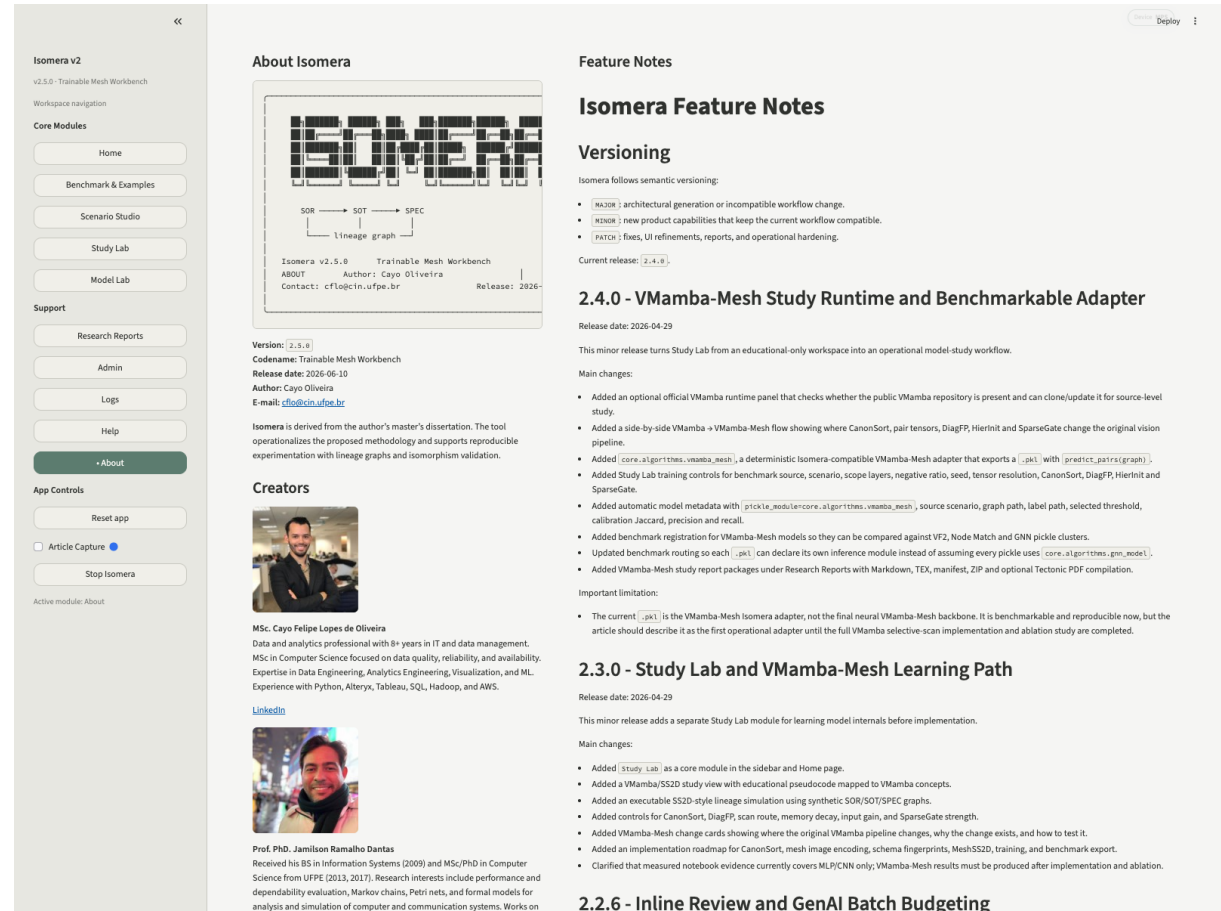


Figure 18.29: Ficha visual: About

Chapter 19

Interpretabilidade

O objetivo da interpretabilidade no Isomera não é provar causalidade absoluta. O objetivo é transformar uma decisão neural em uma evidência local que pode ser revisada. Para um par selecionado, a saliency mostra quais regiões do tensor mais influenciaram o score.

Essa distinção precisa ser repetida: interpretabilidade não é verdade absoluta. Ela é uma ferramenta de inspeção. Se a saliency aparece forte em regiões coerentes com a linhagem, isso aumenta a confiança de que o modelo está olhando para sinais relevantes. Se aparece forte apenas em regiões vazias ou sem significado, isso sinaliza problema de representação, treino ou threshold.

No fluxo ideal, a interpretabilidade vem depois da decisão. Primeiro o modelo identifica pares. Depois o usuário escolhe um par relevante. Em seguida abre a análise local. Essa sequência evita olhar para saliency sem pergunta. O objetivo não é gerar uma imagem bonita, mas responder: por que este par recebeu este score?

As cores devem ser lidas como intensidade de sensibilidade. Regiões mais fortes indicam que pequenas mudanças ali teriam maior impacto no score. Quando essa intensidade se concentra em canais como adjacência, rota ou máscara, o usuário consegue discutir se o modelo está usando estrutura de linhagem, densidade, camada ou grau.

Fique atento! Mapas de saliency são diagnósticos locais. Eles não substituem validação estatística, revisão de falsos positivos ou análise de ground truth. Use-os para investigar uma decisão, não para declarar que o modelo aprendeu uma regra causal universal.

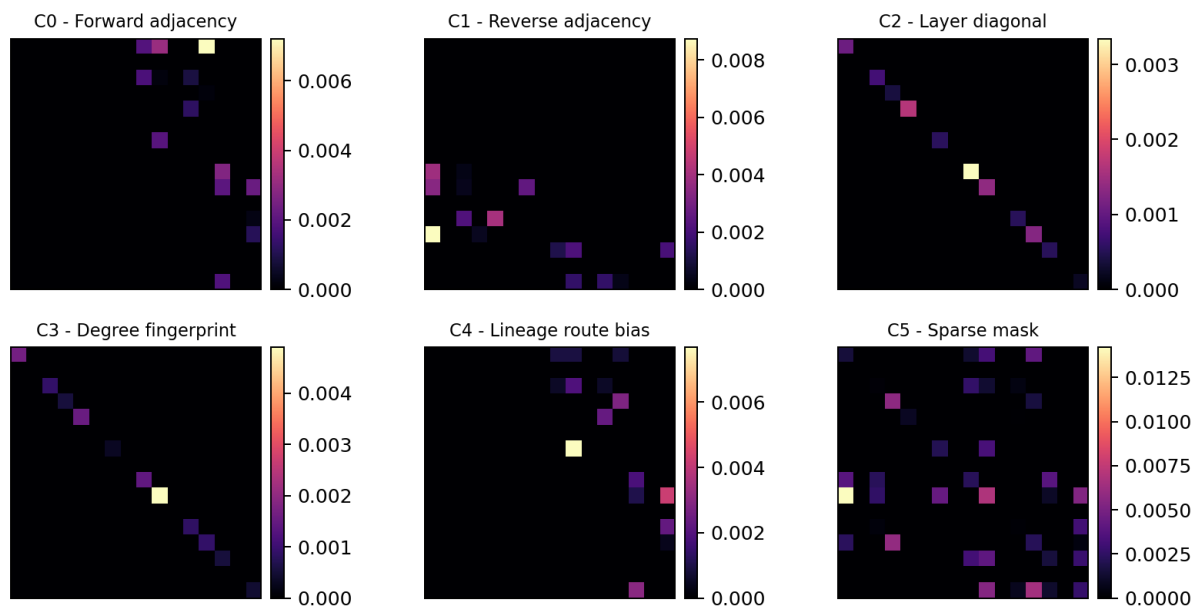


Figure 19.1: Saliency neural para o par SOR16-D1. Cores mais fortes indicam maior sensibilidade local do score.

O fluxo recomendado é:

1. abrir **Study Lab**;
2. entrar em **Model Interpretability**;
3. escolher benchmark e cenário;
4. escolher o par candidato;
5. selecionar modelo e artefato;
6. rodar a geração;
7. comparar grafo, matriz, canais, score, threshold e saliency.

Chapter 20

Como aplicar em uma empresa

Dica! Comece pequeno: um domínio, poucas tabelas, alguns pares rotulados e baselines simples. Expanda só depois que os critérios de duplicidade estiverem claros.

Para aplicar a ideia fora do benchmark, o time precisa exportar a linhagem da empresa como grafo. A origem pode ser catálogo de dados, dbt, Airflow, logs de warehouse, OpenLineage, banco relacional ou ferramenta interna. O contrato mínimo é:

1. cada tabela vira nó;
2. cada dependência vira aresta dirigida;
3. pares candidatos são definidos por domínio, nome, camada, semântica ou sugestão LLM;
4. um conjunto pequeno de pares é rotulado como duplicado ou não duplicado;
5. o grafo é convertido para matriz/tensor;
6. detectores são executados;
7. scores e decisões são revisados;
8. manifests são salvos para reprodução.

Se a empresa já possui pickles treinados, eles podem ser registrados no Model Lab. Se precisa treinar novos modelos, o caminho passa por Scenario Studio e Study Lab.

20.1 Roteiro corporativo recomendado

O primeiro passo é mapear fontes de linhagem. Se a empresa usa dbt, o manifesto pode fornecer dependências. Se usa Airflow, DAGs podem indicar transformações. Se usa catálogo de dados, relacionamentos e owners podem virar metadados. Se usa warehouse, views e queries podem revelar dependências. O Isomera não exige uma origem única; ele exige que, ao final, seja possível montar nós e arestas.

O segundo passo é escolher um escopo pequeno. Não comece por toda a empresa. Escolha um domínio, uma camada ou um conjunto de produtos analíticos. Gere o grafo, revise visualmente e rotule alguns pares. Esse primeiro escopo serve para calibrar nomes, camadas, regras de candidatura e critérios de duplicidade.

O terceiro passo é rodar detectores determinísticos. VF2 e Node Match dão uma linha de base auditável. Eles ajudam a encontrar casos óbvios e a entender o formato do grafo. Depois entram GNN, VMamba, VMamba-Mesh e versões treináveis. A comparação só faz sentido quando todos usam o mesmo ground truth.

O quarto passo é transformar achados em ação de governança. Um par duplicado pode virar recomendação de consolidação, revisão de owner, documentação cruzada, alerta de custo ou criação de regra no catálogo. O Isomera identifica e explica; a decisão de remover, consolidar ou manter duplicidade controlada continua sendo de governança.

20.2 Cuidados antes de produção

Não use o primeiro modelo treinado como decisão automática. Use como apoio de triagem. Revise falsos positivos e falsos negativos. Meça estabilidade por domínio. Verifique se o threshold faz sentido para o risco da empresa. Em alguns ambientes, falso positivo é mais caro porque pode sugerir remoção indevida. Em outros, falso negativo é mais caro porque deixa redundância continuar.

Também é importante separar ambiente de pesquisa e ambiente de operação. Benchmarks servem para comparar modelos. Produção exige dados atuais, monitoramento, governança de acesso, trilha de auditoria e integração com processos. O Isomera oferece a base técnica, mas a implantação precisa respeitar políticas internas.

Chapter 21

Checklist de demonstração

Dica! Para uma demonstração curta, não treine ao vivo. Mostre o fluxo, abra relatórios salvos e use interpretabilidade para explicar um par concreto.

1. Abrir o Isomera e mostrar a Home.
2. Explicar que o problema é duplicidade por linhagem, não apenas nome.
3. Mostrar grafo SOR16 e matriz de adjacência.
4. Abrir `Benchmark & Examples` e mostrar `Article Reproducibility`.
5. Abrir `Study Lab` e mostrar `Deep Learning Workbench`.
6. Explicar C0–C5 e VMamba-Mesh.
7. Abrir `Model Reports` para resultados salvos.
8. Abrir `Model Interpretability` para saliency.
9. Abrir `Model Lab` para pickles e roteamento.
10. Abrir `Research Reports` para pacotes.
11. Fechar com `Help` e `Tech Docs`.

21.1 Checklist de estudo individual

Para estudar a ferramenta sem pressa, use este caminho. Primeiro leia os capítulos de problema, grafo, matriz e tensor. Depois abra a Home e entenda o menu. Em seguida abra `Concepts` no `Benchmark` para ver os cenários. Vá para `Study Lab` e leia `Original VMamba`, `SS2D` e `VMamba-Mesh Changes`. Depois abra `Deep Learning Workbench` e compare dois modelos no mesmo par. Só então vá para `Model Reports`, `Interpretability` e `Research Reports`.

Esse caminho evita decorar telas sem entender o motivo. O Isomera tem muitas rotas porque o problema tem várias camadas. O usuário precisa entender dados, grafo, modelo, benchmark, métrica, relatório e interpretabilidade. Quando essa ordem é respeitada, a ferramenta fica mais simples.

Chapter 22

Referências internas e materiais de apoio

Este manual foi construído a partir dos materiais finais do Isomera, das bases de conhecimento, dos relatórios de campanha, dos materiais de apresentação, dos documentos de estudo e das capturas reais da ferramenta. Os artigos e relatórios entram aqui como evidência e documentação complementar, não como ponto de partida do manual.

Dica! Use estas referências para rastrear a origem dos materiais do manual: capturas, bases de conhecimento, relatórios e documentação técnica.

- Workspace final do Article IV no TeXLab: pacote v2_t.
- `main/docs/tech_hub`
- `.github/knowledge_bases`
- `main/docs/manuals/isomera_complete_manual`

22.1 Referências conceituais recomendadas

Para aprofundar a parte de aprendizado de máquina e redes neurais, leia Goodfellow, Bengio e Courville, *Deep Learning*, MIT Press, 2016. O livro é a referência clássica para logits, funções de ativação, otimização, gradientes, representação e regularização.

Para classificação supervisionada, decisão estatística e fundamentos probabilísticos, leia Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006. Ele ajuda a entender por que score, threshold, função de perda e incerteza precisam ser tratados separadamente.

Para aprendizado em grafos, leia Hamilton, *Graph Representation Learning*, Morgan & Claypool, 2020. Ele fornece a base para pensar em vizinhança, embeddings de nós, mensagem em grafos e limites de representações puramente estruturais.

Para uma visão aplicada de treinamento, validação, desbalanceamento, pipelines e uso prático de deep learning, leia Géron, *Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow*, O'Reilly, 3rd ed., 2022.